

Empirical Analysis for “Predictive conditional alphas” (Distaso et al. 2026)

Paper authors: Valentina Corradi, Walter Distaso,
Marcelo Fernandes, and Asger Lunde
Research assistant: Ricardo Semião e Castro

31 Aug 2026

Table of contents

1	Empirical Analysis for “Predictive Conditional Alphas” (Distaso et al. 2026)	3
1.1	Repository organization	3
2	Data processing: factors	4
2.1	Data and methodology	4
2.2	Code setup	5
2.3	Importing and dates handling	5
2.4	Log-returns and 5-minute aggregation	8
2.5	Trading days	11
3	Data processing: stocks	13
3.1	Data and methodology	13
3.2	Code setup	14
3.3	Initial data cleaning	15
3.4	Dates analysis	18
3.5	Aggregation of 1-minute log returns	26
3.6	Appendix	31
4	Factor PCA	35
4.1	Methodology	35
4.2	Code implementation	38
4.3	Calculating the PCA	42
4.4	Visualizing results	44
4.5	Appendix	48
5	Realized betas and risk-adjusted returns	50
5.1	Methodology	50
5.2	Code implementation	53
5.3	Calculating the betas and risk-adjusted returns	56
5.4	Visualizing results	59
5.5	Appendix	66
6	Conditional alphas	70
6.1	Methodology	70
6.2	The data	72

6.3	The algorithm	72
6.4	Code setup	73
6.5	Data validation	75
6.6	Code implementation	78
6.7	Calculating alphas	83
6.8	Visualizing results	84
6.9	Appendix	86

7	References	90
----------	-------------------	-----------

1 Empirical Analysis for “Predictive Conditional Alphas” (Distaso et al. 2026)

This project contains the empirical analysis for the paper “Predictive Conditional Alphas” by Distaso et al. (2026). The relevant version of the paper can be found in [bibliography/distaso2026_JASA.pdf](#).

The general steps of the empirical analysis are:

- Download and process factor and stock data.
- Extract high-frequency principal components from the factors.
- Estimate realized betas from the stock returns and the principal components.
- Estimate conditional alphas from the risk-adjusted returns.

The detailed methodology for each step is described in the associated report chapter or `.ipynb` notebook in `src/`.

This project has an associated repository and PDF report. Both are explained in this text. Some links point to files in the repository and will not work in the PDF report.

The project runs on Python 3.14 and DuckDB. The `uv` config files allow for running the code with the correct package versions. To generate the report, run `src/quarto_render.ps1` at the project root, or similar commands in Linux or MacOS.

1.1 Repository organization

The repository is organized as follows:

- `data`: not committed due to size.
 - `data/stocks_raw` and `stocks_1min_returns.parquet`: raw stocks from the TAQ database plus intermediate transformations, and the stock log-returns at the 1-minute frequency.
 - `data/factors_raw/ff6_1min_returns.csv`, `data/factors_1min_returns.csv`, and `data/factors_5min_returns.csv`: factor returns at the 1-minute and 5-minute frequencies. The raw data is available at www.sakethaleti.com/data.
 - `data/pca_1min.csv` and `data/pca_5min.csv`: principal components of the factors at the 1-minute and 5-minute frequencies.
- `src`: Most files here are sections of the report.
 - `src/data_factors.ipynb`, `src/data_stocks.ipynb`: data checking and processing.
 - `src/pca.ipynb`: high-frequency PCA of the factors estimation.
 - `src/betas.ipynb`, `src/betas_mp.py`: realized betas and risk-adjusted returns estimation.
 - `src/alphas.ipynb`, `src/alphas_mp.py`: conditional alphas estimation.
 - `src/parameters.py`, `src/utls.py`: utility scripts.
- `bibliography`: reference papers.
- `docs`: `the full report`, a `summary`, and other documentation assets.
- Configuration files:
 - Docs and repository: `__quarto.yml`, `README.md`, `.gitignore`
 - Python: `.python-version`, `pyproject.toml`, `uv.lock`

2 Data processing: factors

This notebook mines and processes the factor data needed. Read [README.md](#)/Chapter 1 for context about the project.

2.1 Data and methodology

2.1.1 Raw data

Sources: Fama-French 6 factors from saketshaheti.com/data, or [this Google Drive link](#). Documentation can be found in Section 1 of the [online appendix](#).

Format: Already at the 1-minute level. One column for the minute, and one for each factor. The factors are in returns (not log).

Filters:

- Sample period (same as stocks):
 - Range: 2001-01-02 to 2017-12-29. Stocks that are listed or delisted during this period are included.
 - Hours: regular trading hours (09:30 - 16:00 ET).
 - Days: regular trading days, removing days with market-wide halts and diminished trading hours.

Qualities:

- The data should be ordered by `datetime`.
- There should not be data for non-trading days, and there should be all 390 minutes of a day with data.
- The minutes should be left labelled as 9:30 - 9:30:00 -> 9:30:59, 9:31 - 9:31:00 -> 9:31:59, ..., 15:59 - 15:59:00 -> 15:59:59, and no additional 16:00 data point.

All are guaranteed by the source and checked informally. The last one is not, and we might need to transform it.

2.1.2 Methodology

The steps to process the data are as follows.

- Download the data into `data/factors_raw/ff6_1min_returns.csv`. Alternatively, it could be read directly from Google Drive.
- Apply the filters above.
- Check if the minutes are labelled correctly and transform them if needed.
- Transform the data into log-returns.
- Aggregate the data into left-labelled 5-minute blocks (09:30, 09:35, ..., 15:55), by summing the returns of the label minute and the 4 minutes before it. Save it into `data/ff6_5min_returns.csv`.

2.2 Code setup

Code for debugging:

```
%reset -f
import importlib
```

Core modules:

```
import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import polars as pl
import numpy as np

import src.utils as ut
import src.graphs as plot
from src.parameters import PARAMETERS as PARS
importlib.reload(ut)
importlib.reload(plot)

ut.set_seed()
```

2.3 Importing and dates handling

Importing and formatting dates as datetimes. A first look at the data is below. The names of the factors are:

- `ff__mkt`: market factor (MKT)
- `ff__smb`: size factor (SMB)
- `ff__hml`: value factor (HML)
- `ff__rmw`: profitability factor (RMW)
- `ff__cma`: investment factor (CMA)
- `ff__umd`: momentum factor (UMD)

```
data_raw = (
    pl.read_csv("data/factors_raw/ff6_1min_returns.csv")
    .with_columns(
        pl.col("datetime")
        .str.strptime(pl.Datetime, "%Y-%m-%d %H:%M:%S")
    )
)

cols_fct = {
    "ff__mkt": "market",
    "ff__smb": "size",
    "ff__hml": "value",
    "ff__rmw": "profitability",
    "ff__cma": "investment",
    "ff__umd": "momentum",
}
```

```
print(f"Data shape: {data_raw.shape}")
print("First rows:")
data_raw.head(3).rename(cols_fct)
```

Data shape: (2454420, 7)

First rows:

datetime	market	size	value	profitability	investment	momentum
datetime[s]	f64	f64	f64	f64	f64	f64
1996-01-02 09:30:00	-0.002509	0.001133	-0.000592	-0.000541	-0.000922	-0.003535
1996-01-02 09:31:00	-0.000061	0.000184	0.000053	0.000002	-0.000021	0.000204
1996-01-02 09:32:00	-0.000123	0.000121	-0.000045	0.000123	-0.000098	-0.000083

Checking sorting:

```
if not data_raw["datetime"].is_sorted():
    raise ValueError("Data is not sorted by datetime.")
```

Filtering by date limits:

```
data_filt = data_raw.filter(
    (pl.col("datetime").dt.date() >= PARS.date_start.item())
    & (pl.col("datetime").dt.date() <= PARS.date_end.item())
)
```

All trading days from the NYSE calendar must be present, and all days in the data must be trading days:

```
dates_trading_all_exist = np.isin(
    PARS.trading_days,
    data_filt["datetime"].dt.date()
)
if np.any(~ dates_trading_all_exist):
    raise ValueError(f"There are {np.sum(dates_trading_all_exist)} missing days.")

dates_all_is_trading = np.isin(
    data_filt["datetime"].dt.date(),
    PARS.trading_days
)
if np.any(~ dates_all_is_trading):
    raise ValueError(f"There are {np.sum(dates_all_is_trading)} non-trading days.")
```

Checking non-full days (fewer than 391 minutes of data), and data where stocks have systematically few observations (see [src/data_stocks.ipynb](#) / Chapter 3). All such days are special days on the NYSE calendar and will be removed.

```

dates_non_full = PARS.trading_days[
    ~ np.isin(PARS.trading_days, PARS.trading_days_final)
]
print(
    f"{len(dates_non_full)} non-full trading days:" +
    ut.format_days_byyear(dates_non_full)
)

data_filt = data_filt.filter(
    np.isin(data_filt["datetime"].dt.date(), PARS.trading_days_final)
)

```

52 non-full trading days:

```

- 2001: 06-08, 07-03, 11-23, 12-24,
- 2002: 07-05, 09-11, 11-29, 12-24,
- 2003: 07-03, 08-15, 11-28, 12-24, 12-26,
- 2004: 11-26,
- 2005: 11-25, 12-23,
- 2006: 07-03, 11-24,
- 2007: 07-03, 11-23, 12-24,
- 2008: 07-03, 11-28, 12-24, 12-26,
- 2009: 11-27, 12-24,
- 2010: 11-26, 12-27,
- 2011: 08-15, 11-25, 12-23, 12-27,
- 2012: 07-03, 11-12, 11-23, 12-24,
- 2013: 07-03, 08-22, 11-29, 12-24,
- 2014: 07-03, 11-28, 12-24, 12-26,
- 2015: 10-12, 11-27, 12-24,
- 2016: 11-25, 12-23,
- 2017: 07-03, 11-24,

```

Similarly, let's check whether all days start at 09:30 and end at 16:00:

```

if not (
    data_filt
    .group_by(pl.col("datetime").dt.date())
    .agg(
        test = (
            pl.col("datetime").dt.time()
            == pl.time_range(pl.time(9, 30), pl.time(16, 00), interval = "1m")
        ).all()
    )
    ["test"].all()
):
    raise ValueError("Some days do not start at 09:30 and end at 16:00.")

```

While the final factor data will cover only the defined window, we need warm-up data for the PCA procedure in [src/pca.ipynb](#) Chapter 4. We need 858 1-minute observations and 384 5-minute observations. We will add the raw data's start back in, perform the transformations and aggregations, and then keep only the required window.

For this warm-up data, we do not need matching with stocks or full days, so those checks are not performed.

```
dates_add = {1: 858, 5: 384}

data = pl.concat([
    data_raw.filter(data_raw["datetime"].dt.date() < PARS.date_start.item()),
    data_filt
])
```

Now let's alter the minute labels and remove overnight effects.

From Aleti (2022) online appendix: *"I only consider trades (i) between 09:30:00 and 16:00:00 [...]. I then collect low-frequency prices from CRSP to merge with the high-frequency prices from TAQ. [...] Using the adjusted close-to-close return along with the prices for each day, it is simple to infer the adjusted overnight (close-to-open) return. I can then assign this adjusted return to the 09:30:00 observation for each day. Additionally, I adjust the 16:00:00 return using the delisting return."*

Thus, the 9:30 observation is the overnight return and must be removed. The rest of the observations are right-labelled – $9:31 - 9:30:00 < x < 9:31:00$, ... – with the last instant 16:00 being adjusted with the delisting return. Thus, they must be transformed into left-labels.

```
data = (
    data
    .filter(pl.col("datetime").dt.time() != pl.time(9, 30, 0))
    .with_columns(
        pl.col("datetime") - pl.duration(minutes = 1)
    )
)
```

2.4 Log-returns and 5-minute aggregation

Data are in returns, and indeed all values are in the range $[-1, \infty[$. We can transform them into log-returns by applying the map $r \mapsto \log(1 + r)$.

```
data_1min = data.with_columns(pl.col(cols_fct.keys()).log1p())

data_5min = (
    data_1min.group_by_dynamic("datetime", every = "5m")
    .agg([pl.col(col).sum() for col in cols_fct])
)
```

Then, remove the unneeded observations:

```
dates_start_idx = (
    np.flatnonzero(data_1min["datetime"].dt.date() >= PARS.date_start.item())[0]
)

data_1min = data_1min.slice(dates_start_idx - dates_add[1], None)

if (data_1min["datetime"].dt.date() < PARS.date_start.item()).sum() != dates_add[1]:
```

```

raise ValueError("Wrong warm up size for 1-minute.")

dates_start_idx = (
    np.flatnonzero(data_5min["datetime"].dt.date() >= PARS.date_start.item())[0]
)

data_5min = data_5min.slice(dates_start_idx - dates_add[5], None)

if (data_5min["datetime"].dt.date() < PARS.date_start.item()).sum() != dates_add[5]:
    raise ValueError("Wrong warm up size for 1-minute.")

```

Having a look at and saving the 1-minute data:

```

data_1min.write_csv("data/factors_1min_returns.csv", datetime_format = "%Y-%m-%d %H:%M")
data_1min.rename(cols_fct)

```

datetime datetime[s]	market f64	size f64	value f64	profitability f64	investment f64	momentum f64
2000-12-27 14:42:00	-0.000078	0.000042	-0.000115	0.000057	0.000192	0.000248
2000-12-27 14:43:00	-0.000435	0.000376	0.000072	-0.000109	0.000194	-0.0003
2000-12-27 14:44:00	-0.000253	0.000221	0.000066	0.000196	-0.000238	-0.000078
2000-12-27 14:45:00	0.000304	-0.000367	-0.000153	0.00029	-0.000158	0.000021
2000-12-27 14:46:00	0.000057	-0.000287	-0.000133	0.000002	-0.0002	0.000121
...	...	...	...	...	...	...
2017-12-29 15:55:00	-0.00003	0.00007	-0.000064	0.000003	-0.00009	0.000055
2017-12-29 15:56:00	-0.000033	0.000039	-0.00004	0.000023	-0.000023	0.000103
2017-12-29 15:57:00	-0.000192	-0.000075	-0.000107	0.000056	0.000017	0.000075
2017-12-29 15:58:00	5.8310e-7	-0.000083	-0.000132	-0.000041	0.000018	-0.000061
2017-12-29 15:59:00	-0.001403	0.000706	-0.000426	0.000124	-0.000084	-0.00008

Having a look at and saving the 5-minute data:

```

data_5min.write_csv("data/factors_5min_returns.csv", datetime_format = "%Y-%m-%d %H:%M")
data_5min.rename(cols_fct)

```

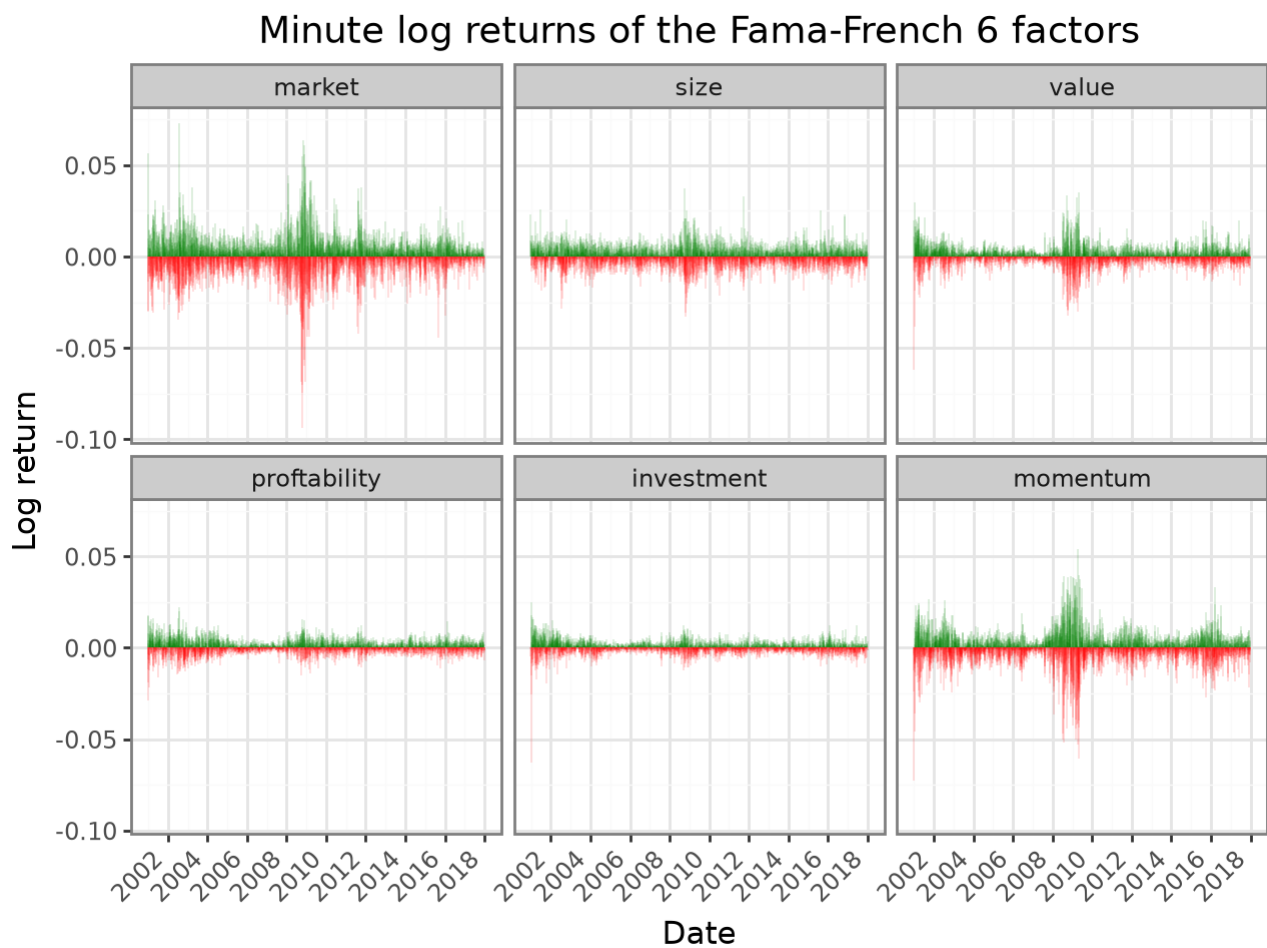
datetime datetime[s]	market f64	size f64	value f64	profitability f64	investment f64	momentum f64
2000-12-22 10:00:00	0.001716	0.000084	-0.00156	-0.001235	-0.001339	-0.000846
2000-12-22 10:05:00	-0.000355	0.001536	0.000538	-0.000224	0.00043	0.001022
2000-12-22 10:10:00	-0.001602	0.001063	0.001256	-0.000528	0.000825	0.000486
2000-12-22 10:15:00	0.000046	-0.000266	0.000518	-0.000772	-0.000284	-0.000367
2000-12-22 10:20:00	0.001853	-0.000933	-0.000773	-0.000039	-0.000956	0.000094
...	...	...	...	...	...	...
2017-12-29 15:35:00	-0.000758	0.000106	-0.000049	0.000123	-0.000138	0.000232
2017-12-29 15:40:00	-0.00028	0.000043	-0.000138	-0.000112	0.000141	0.000026
2017-12-29 15:45:00	0.000254	-0.000358	0.000196	-0.000303	0.000087	-0.000143
2017-12-29 15:50:00	-0.001415	-0.00086	0.000294	-0.000049	0.000542	-0.000387

datetime	market	size	value	profitability	investment	momentum
datetime[s]	f64	f64	f64	f64	f64	f64
2017-12-29 15:55:00	-0.001659	0.000658	-0.000769	0.000165	-0.000162	0.000091

2.4.1 Visualizing raw data and aggregation

The factors are in log returns, all centered around zero with varying variance. The aggregation into 5-minute blocks was a very simple transformation and will not be plotted here.

```
plot.factors(data_1min, cols_fct)
```



Then, we plot the basic statistics for the 1-minute data:

```
(
  data_1min
  .rename(cols_fct)
  .select(list(cols_fct.values()))
  .describe()
)
```

statistic str	market f64	size f64	value f64	profitability f64	investment f64	momentum f64
"count"	1.648218e6	1.648218e6	1.648218e6	1.648218e6	1.648218e6	1.648218e6
"null_count"	0.0	0.0	0.0	0.0	0.0	0.0
"mean"	4.7965e-8	1.4252e-7	2.1952e-7	9.2992e-7	3.4937e-7	-3.0763e-7
"std"	0.000404	0.000281	0.00018	0.000123	0.000121	0.000225
"min"	-0.019409	-0.010303	-0.012796	-0.007118	-0.005912	-0.007682
"25%"	-0.000138	-0.000108	-0.000069	-0.000056	-0.000055	-0.000078
"50%"	0.000002	7.5816e-7	2.9192e-7	4.7377e-7	2.8545e-8	8.5415e-7
"75%"	0.00014	0.000109	0.00007	0.000057	0.000055	0.000079
"max"	0.013797	0.014219	0.007995	0.003959	0.004689	0.010337

And for the 5-minute data:

```
(
    data_5min
    .rename(cols_fct)
    .select(list(cols_fct.values()))
    .describe()
)
```

statistic str	market f64	size f64	value f64	profitability f64	investment f64	momentum f64
"count"	329856.0	329856.0	329856.0	329856.0	329856.0	329856.0
"null_count"	0.0	0.0	0.0	0.0	0.0	0.0
"mean"	3.2769e-7	7.4279e-7	0.000001	0.000005	0.000002	-0.000001
"std"	0.001002	0.000624	0.000459	0.000306	0.0003	0.000611
"min"	-0.025618	-0.016354	-0.014583	-0.007156	-0.008387	-0.012706
"25%"	-0.000391	-0.00026	-0.000179	-0.000137	-0.000131	-0.000217
"50%"	0.000008	0.000004	0.000001	0.000002	2.8891e-7	0.000002
"75%"	0.000397	0.000266	0.000182	0.000143	0.000133	0.000219
"max"	0.034904	0.024543	0.01378	0.00556	0.005716	0.015051

2.5 Trading days

Here I recall the approach used to define the full trading days considered in this project:

1. The NYSE trading-days calendar is obtained from the `exchange_calendars` and `pandas_market_calendars` Python modules and checked against each other. This calendar is used to validate the raw stock and factor data.
2. Dates with fewer than 390 1-minute observations in the factor data or simultaneously across all stocks are deemed non-full trading days and removed from the final list of trading days.

At the end, the following non-weekdays are removed from the final trading-day list:

```
dates_week_all = np.arange(PARS.date_start, PARS.date_end, dtype = 'datetime64[D]')
dates_week_all = dates_week_all[np.is_busday(dates_week_all)]
dates_week_rmv = dates_week_all[~ np.isin(dates_week_all, PARS.trading_days_final)]

print("Weekdays removed:" + ut.format_days_byyear(dates_week_rmv))
```

Weekdays removed:

- 2001: 01-01, 01-15, 02-19, 04-13, 05-28, 06-08, 07-03, 07-04, 09-03, 09-11, 09-12
09-13, 09-14, 11-22, 11-23, 12-24, 12-25,
- 2002: 01-01, 01-21, 02-18, 03-29, 05-27, 07-04, 07-05, 09-02, 09-11, 11-28, 11-29
12-24, 12-25,
- 2003: 01-01, 01-20, 02-17, 04-18, 05-26, 07-03, 07-04, 08-15, 09-01, 11-27, 11-28
12-24, 12-25, 12-26,
- 2004: 01-01, 01-19, 02-16, 04-09, 05-31, 06-11, 07-05, 09-06, 11-25, 11-26, 12-24,
- 2005: 01-17, 02-21, 03-25, 05-30, 07-04, 09-05, 11-24, 11-25, 12-23, 12-26,
- 2006: 01-02, 01-16, 02-20, 04-14, 05-29, 07-03, 07-04, 09-04, 11-23, 11-24, 12-25,
- 2007: 01-01, 01-02, 01-15, 02-19, 04-06, 05-28, 07-03, 07-04, 09-03, 11-22, 11-23
12-24, 12-25,
- 2008: 01-01, 01-21, 02-18, 03-21, 05-26, 07-03, 07-04, 09-01, 11-27, 11-28, 12-24
12-25, 12-26,
- 2009: 01-01, 01-19, 02-16, 04-10, 05-25, 07-03, 09-07, 11-26, 11-27, 12-24, 12-25,
- 2010: 01-01, 01-18, 02-15, 04-02, 05-31, 07-05, 09-06, 11-25, 11-26, 12-24, 12-27,
- 2011: 01-17, 02-21, 04-22, 05-30, 07-04, 08-15, 09-05, 11-24, 11-25, 12-23, 12-26
12-27,
- 2012: 01-02, 01-16, 02-20, 04-06, 05-28, 07-03, 07-04, 09-03, 10-29, 10-30, 11-12
11-22, 11-23, 12-24, 12-25,
- 2013: 01-01, 01-21, 02-18, 03-29, 05-27, 07-03, 07-04, 08-22, 09-02, 11-28, 11-29
12-24, 12-25,
- 2014: 01-01, 01-20, 02-17, 04-18, 05-26, 07-03, 07-04, 09-01, 11-27, 11-28, 12-24
12-25, 12-26,
- 2015: 01-01, 01-19, 02-16, 04-03, 05-25, 07-03, 09-07, 10-12, 11-26, 11-27, 12-24
12-25,
- 2016: 01-01, 01-18, 02-15, 03-25, 05-30, 07-04, 09-05, 11-24, 11-25, 12-23, 12-26,
- 2017: 01-02, 01-16, 02-20, 04-14, 05-29, 07-03, 07-04, 09-04, 11-23, 11-24, 12-25,

3 Data processing: stocks

This notebook mines and processes the stock data needed. Read the [README.md](#)/Chapter 1 file for context about the project.

3.1 Data and methodology

3.1.1 Raw data

Sources: Intraday tick data from the [NYSE TAQ database](#). The relevant documentation is [Daily_TAQ_Client_Spec_v4.3.pdf](#).

Format:

The data can be easily stored as an SQLite3 database with the main table `trades` of the following schema:

```
CREATE TABLE trades (  
    permno    INTEGER NOT NULL, -- Permanent number of the stock  
    ts_ny     INTEGER NOT NULL, -- Timestamp in New York time (epoch seconds)  
    price     REAL     NOT NULL, -- Price of the trade  
    size      INTEGER NOT NULL, -- Size of the trade  
    ex        TEXT     NOT NULL, -- Exchange code  
    corr      TEXT     NOT NULL, -- Correction code  
    ticker    TEXT     NOT NULL, -- Stock ticker symbol  
    ambiguous INTEGER NOT NULL, -- Flag if the trade is ambiguous (0 or 1)  
    PRIMARY KEY (permno, ts_ny)  
) WITHOUT ROWID
```

Further documentation might be found by looking for the relevant websites.

For the preparation of this notebook, the data was given in a private Dropbox folder. The [src/scrape.py](#) file contains a script to download it into the `data/stocks_raw/trades.db` file.

Qualities:

The data is expected to have the qualities below. All are guaranteed by the data source.

- Within each `permno`, the data should be ordered by `ts_ny`. *Will be sorted.*
- There is one observation per minute, at the 00 second mark. Duplicates do not exist, but missing minutes do. *Will be checked.*
- Data validity: `price` should never be negative or zero; `ex` and `corr` should have only the correct levels. *Anomalies are filtered out.*
- There should be data only for trading days. *Will be checked.*

3.1.2 Methodology

First, some initial filters are applied (Section 3.3), and the raw data is saved in an optimized format. Then, the data is transformed from a `permno ~ second ~ price` data structure into a `permno ~ minute ~ log return` data structure (Section 3.5).

The calculations are done via [DuckDB](#) SQL queries.

3.2 Code setup

Code for debugging and running SQL inside notebooks:

```
%reset -f
import importlib

from IPython.core.magic import register_cell_magic
from pprint import pprint

import duckdb

queries_registered: dict[str, str] = {}

@register_cell_magic
def register_query(line: str, cell: str) -> None:
    query_name = line.strip()
    if not query_name:
        raise ValueError("You must provide a name for the query.")
    queries_registered[query_name] = cell
```

Setting up the DuckDB infrastructure. The thread and memory limits are set according to the machine's memory constraints (16 GB RAM) used to process the data.

```
%%register_query duckdb_infra
INSTALL sqlite;
LOAD sqlite;

-- DETACH db_trades;
ATTACH IF NOT EXISTS 'data/stocks_raw/trades.db' AS db_trades (TYPE SQLITE);

SET enable_progress_bar = true;
SET enable_progress_bar_print = true;

PRAGMA threads = 7;
PRAGMA memory_limit = '14GB';

duckdb.execute(queries_registered["duckdb_infra"])
```

Core modules:

```

import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import math

import polars as pl
import numpy as np

import src.utils as ut
import src.graphs as plot
from src.parameters import PARAMETERS as PARS
importlib.reload(ut)
importlib.reload(plot)

ut.set_seed()

```

3.3 Initial data cleaning

An initial filter is applied before the processing logic to remove unnecessary data.

The column types are converted to ones that occupy less space in memory. See the [DuckDB types](#) for reference.

The data is then saved into a [.parquet](#) file, a columnar storage format that is more efficient for analytical queries and compression than the original SQLite3 database. The row group size is chosen to be approximately the median number of observations per `permno`.

Filters:

- Exchanges: NYSE, NYSE-Arca, NASDAQ, and AMEX (code shares 10 and 11 at the CRSP database) (`ex IN ('P', 'N', 'Q', 'A')`).
- Types of trades: regular trades (`corr = '00'`).
- Price should be positive (`price > 0`).
- Sample period (same as factors):
 - Range: 2001-01-02 to 2017-12-29. Stocks that are listed or delisted during this period are kept.
 - Hours: regular trading hours (09:30:00 - 15:59:59 ET).

Formatting and data checking:

- `permno`: a 5-digit positive integer, stored as `UINT32`.
- `ts_ny`:
 - Is a seconds-since-1970-01-01 timestamp.
 - Our start date is 2001-01-02, which guarantees that the relevant observations are always positive.
 - There are no duplicates (`ts_ny > LAG(ts_ny)`).
 - There is only one observation per minute at the 00 second mark (`ts_ny % 60 != 0`), so it is transformed into a minute epoch.
 - Stored as `UINT32`, whose maximum date extends up to the year 10141.
- `price`: 14 integer digits and 6 decimal digits, stored as `FLOAT8`. `Decimal(9, 6)` could be used, but Python's matrix libraries work better with floats.

The two checks are stored in a validation column, and the final result is partitioned by it. Rows are partitioned into the `data/stocks_raw` directory's subfolders `valid=pass/`, `valid==fail_not_minute/`, and `valid==fail_unordered_or_dupe/` (not committed to git).

This will be implemented via the following query:

```
%%register_query raw_to_clean

-- ?1: start epoch seconds (PARS.date_start)
-- ?2: end epoch seconds (PARS.date_end)

COPY (
  SELECT
    CAST(permno AS UINT32) AS permno,
    CAST(ts_ny // 60 AS UINT32) AS ts_min_ny,
    CAST(price AS FLOAT8) AS price,
    CASE
      WHEN (COUNT(*) OVER (PARTITION BY permno, ts_min_ny)) > 1
        AS is_duplicate
      THEN 'fail_duplicate'
      ELSE 'pass'
    END AS valid
  FROM db_trades.trades
  WHERE
    ts_ny BETWEEN ?1 AND ?2
    AND corr = '00'
    AND ex IN ('P', 'N', 'Q', 'A')
    AND (ts_ny % 86400) BETWEEN 34200 AND 57599 -- 09:30:00 to 15:59:59
    AND price > 0
  ORDER BY permno, ts_min_ny -- Costly but helps with querying later
) TO 'data/stocks_raw' (
  FORMAT PARQUET,
  COMPRESSION 'ZSTD',
  ROW_GROUP_SIZE 555000, -- Median of observations per permno
  PARTITION_BY (valid)
);
```

Then, we check whether there are no data in the `valid==fail_duplicate/` folders, and move the `valid=pass/` data into the `data/stocks_raw` folder:

```
if False: # Only execute once
    duckdb.execute(
        queries_registered["raw_to_clean"],
        [
            int(x.astype("datetime64[s]").astype(np.int64))
            for x in (PARS.date_start, PARS.date_end)
        ]
    )
    os.rename(
        "data/stocks_raw/valid=pass/data_0.parquet",
        "data/stocks_raw/trades_clean.parquet"
    )
```

```

if "valid==fail_duplicate/" in os.listdir("data/stocks_raw/"):
    raise ValueError(
        "There are invalid rows to be analyzed. Check the `data/stocks_raw/`'s"
        " subfolders"
    )

```

For utility, save the number of observations for each permno:

```

%%register_query counts_permno
COPY (
    SELECT
        permno,
        COUNT(*) as obs_count
    FROM 'data/stocks_raw/trades_clean.parquet'
    GROUP BY permno
    ORDER BY permno
) TO 'data/stocks_raw/counts_permno.csv' (HEADER, DELIMITER ',');

```

```

if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno"])

data_counts_permno = pl.read_csv("data/stocks_raw/counts_permno.csv")

```

The number of observations (1-minute blocks) for each (permno, day):

```

%%register_query counts_permno_day_1min
COPY (
    SELECT
        permno,
        (ts_min_ny // (60 * 24)) as d,
        COUNT(*) as obs_count
    FROM 'data/stocks_raw/trades_clean.parquet'
    GROUP BY permno, d
) TO 'data/stocks_raw/counts_permno_day_1min.csv' (HEADER, DELIMITER ',');

```

```

if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno_day_1min"])

```

And the number of 5-minute blocks with data, for each (permno, day):

```

%%register_query counts_permno_day_5min
COPY (
    SELECT
        permno,
        (ts_min_ny // (60 * 24)) as d,
        COUNT(DISTINCT ts_min_ny // 5) as obs_count
    FROM 'data/stocks_raw/trades_clean.parquet'
    GROUP BY permno, d
) TO 'data/stocks_raw/counts_permno_day_5min.csv' (HEADER, DELIMITER ',');

```

```
if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno_day_5min"])
```

Saving the data and filling missing days within each permno lifetime with `obs_count = 0`:

```
data_counts_min = {
    x: pl.read_csv(f"data/stocks_raw/counts_permno_day_{x}min.csv")
    for x in [1, 5]
}
```

3.4 Dates analysis

Now, we turn to analyzing the distribution of observations within a day for a stock. We want to remove stocks that have too many days with none or too few observations.

3.4.1 Trading days

First, we need to define what constitutes a missing day, that is, what are the trading days. In [src/parameters.py](#), we calculate the full (9:30 → 16:00) trading days as the union of: days with full factor data, days as described by the `exchange_calendars` Python package, and days described by the `pandas_market_calendars` package. However, there may be some days that were not full trading days for the stocks in our sample.

First, let us check whether any stocks have data for non-trading days. It is expected that there are data for non-full trading days.

```
data_days = data_counts_min[1].with_columns(
    pl.from_epoch(pl.col("d"), time_unit = "d")
)

days_unique = np.unique(data_days["d"].to_numpy().astype("datetime64[D]"))

if not np.isin(days_unique, PARS.trading_days).all():
    raise ValueError("There are days in the data that are not trading days.")

if not np.isin(days_unique, PARS.trading_days_factors).all():
    print("There are days in the data that are not full trading days.")
```

There are days in the data that are not full trading days.

The river plot below shows, for each day, the quantiles of the number of minutes with data for each stock, divided by the total number of minutes in a trading day (390).

The blue dots show days when the factor data did not have all minutes with data, which is, at first, our truth source for non-full days. The gray dots show the days where the median is 0.1 points smaller than its 5-day rolling mean, that is, possible extraordinary trading days.

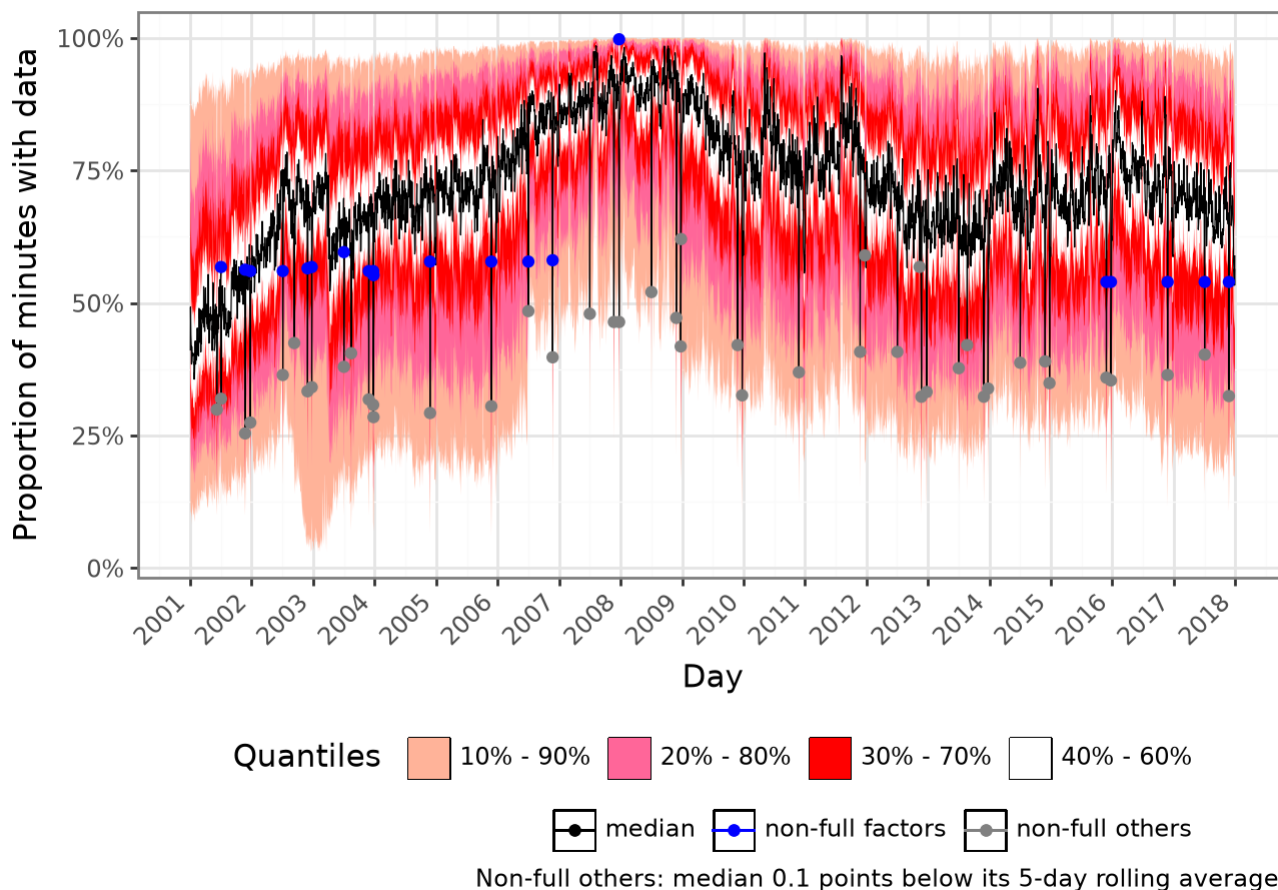
```

data_days_fill = (
    data_days
    .with_columns(pl.col("obs_count") / PARS.blocks_1min)
    .group_by("d")
    .agg(
        median = pl.median("obs_count"),
        **{
            f"q{q}": pl.quantile("obs_count", q / 10)
            for q in [4, 6, 3, 7, 2, 8, 1, 9]
        }
    )
    .sort("d")
)

plot.day_fullness_river(data_days_fill, data_counts_permno.shape[0])

```

Proportion of minutes with data within each day
Quantiles calculated each day across all 2364 stocks



The average median is ≈ 0.72 . We can see many outlier dates with very low coverage, which are likely to be extraordinary trading days. Still, some of these days are not within the list of full trading days, as shown below. Individually checking the dates, they still seem to be extraordinary trading days but have data present for the factors. They will be included in the non-full trading days list.

```

days_low_obs = (
    data_days_fill

```

```

        .with_columns(median_rw = pl.col("median").rolling_mean(8))
        .filter(pl.col("median") - pl.col("median_rw") <= - 0.1)
        ["d"].to_numpy()
    )

print(f"Number of days with low observations: {len(days_low_obs)}")
print(ut.format_days_byyear(days_low_obs[
    np.isin(days_low_obs, PARS.trading_days_factors)
]))

```

Number of days with low observations: 52

```

- 2001: 06-08,
- 2002: 09-11,
- 2003: 08-15,
- 2005: 12-23,
- 2007: 07-03, 11-23,
- 2008: 07-03, 11-28, 12-24, 12-26,
- 2009: 11-27, 12-24,
- 2010: 11-26, 12-27,
- 2011: 08-15, 11-25, 12-23, 12-27,
- 2012: 07-03, 11-12, 11-23, 12-24,
- 2013: 07-03, 08-22, 11-29, 12-24,
- 2014: 07-03, 11-28, 12-24, 12-26,
- 2015: 10-12,
- 2016: 12-23,

```

3.4.2 Days with missing data

Now we turn to checking which days have missing data. Missing days within a permno's lifetime are filled with 0. For each 1-minute and 5-minute block, we plot:

1. The histogram of the number of observed blocks for each day across all days of all stocks.
2. Boxplots of the proportion of days with more observed blocks than some threshold τ , across all stocks, for multiple thresholds.

```

data_filled_min = {
    x: ut.fill_missing_days(data_counts_min[x])
    for x in [1, 5]
}

```

3.4.3 1-Minute blocks

For $\tau = 78$, there are 1,292 stocks with at most 2.5% non-full days.

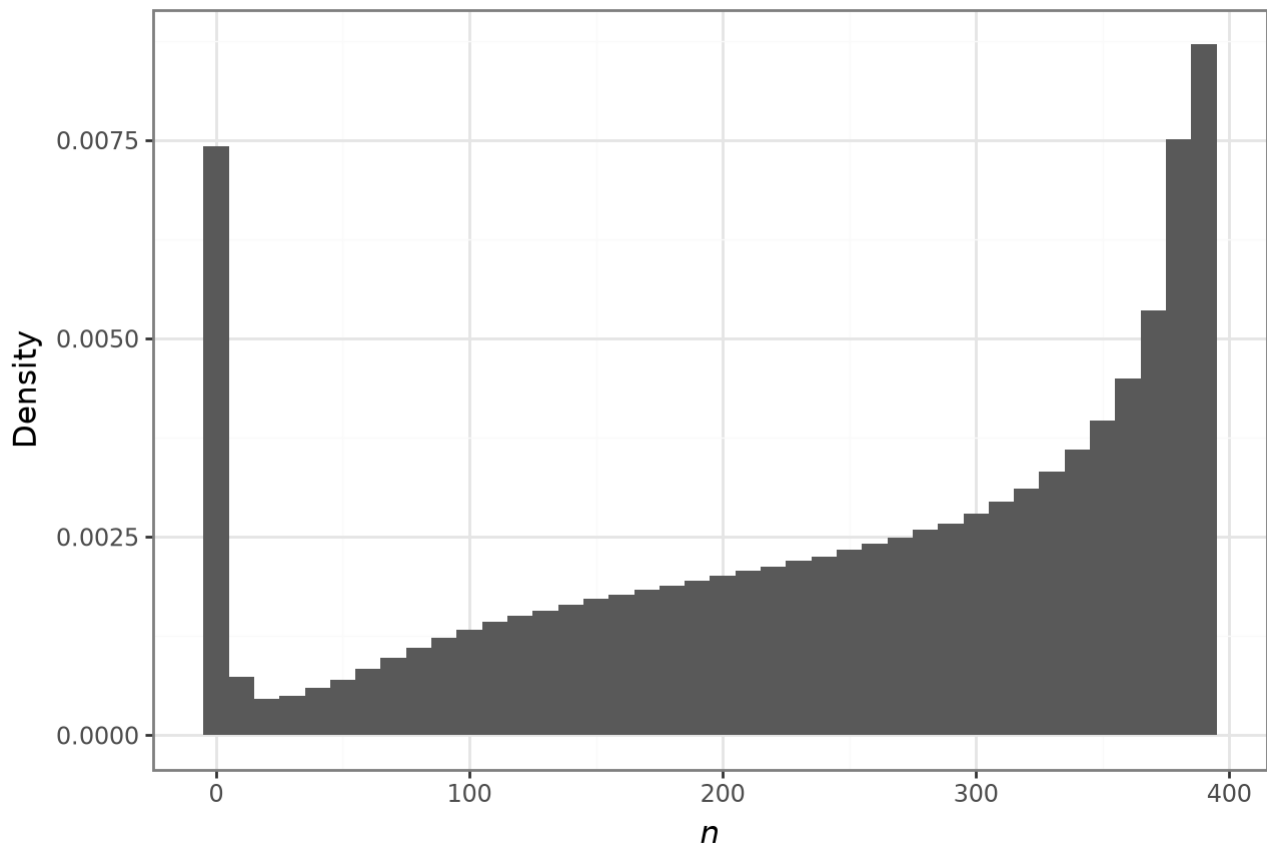
```

plot.day_fullness_hist(data_filled_min[1])

```

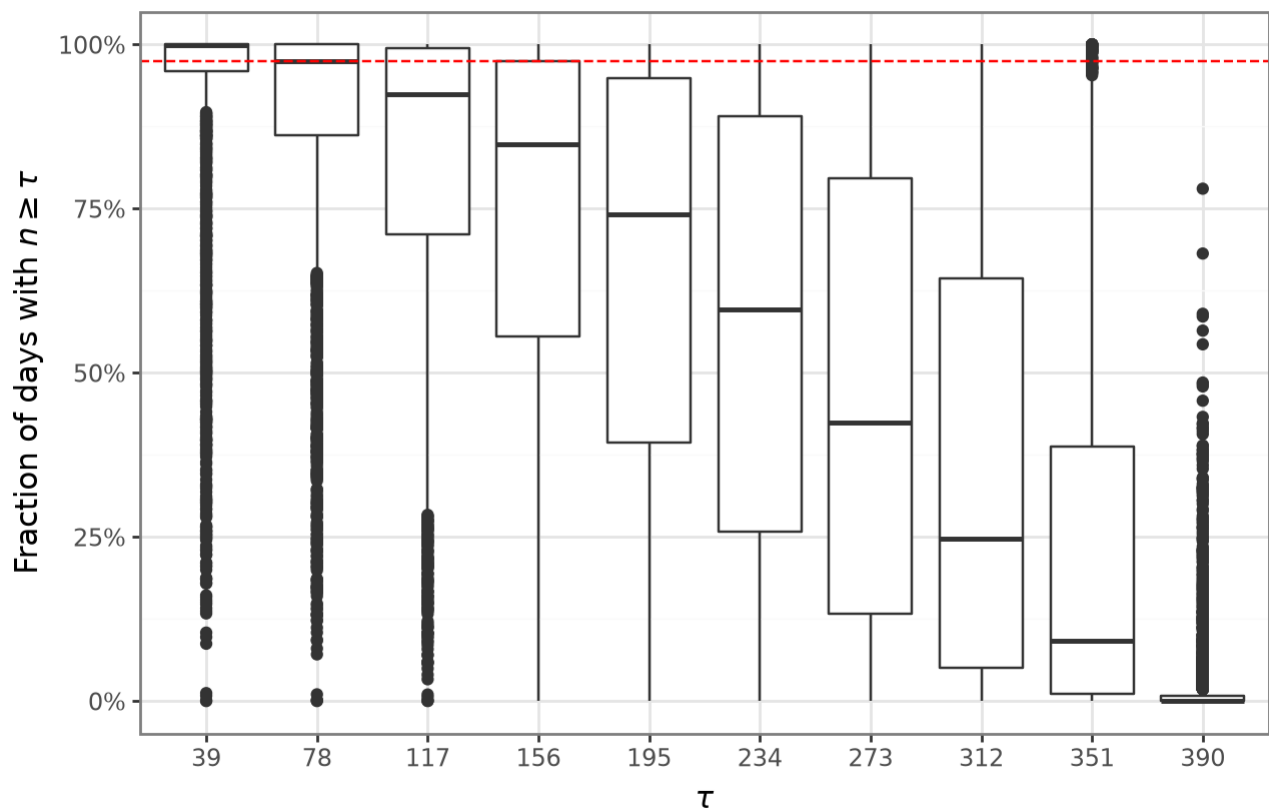
Distribution of days' observation count (n)

Across all 6,397,249 stocks-day pairs



```
plot.day_fullness_box(data_filled_min[1], 1, data_counts_permno.shape[0])
```

Distribution of days' observation count (n)
Across all 2364 stocks

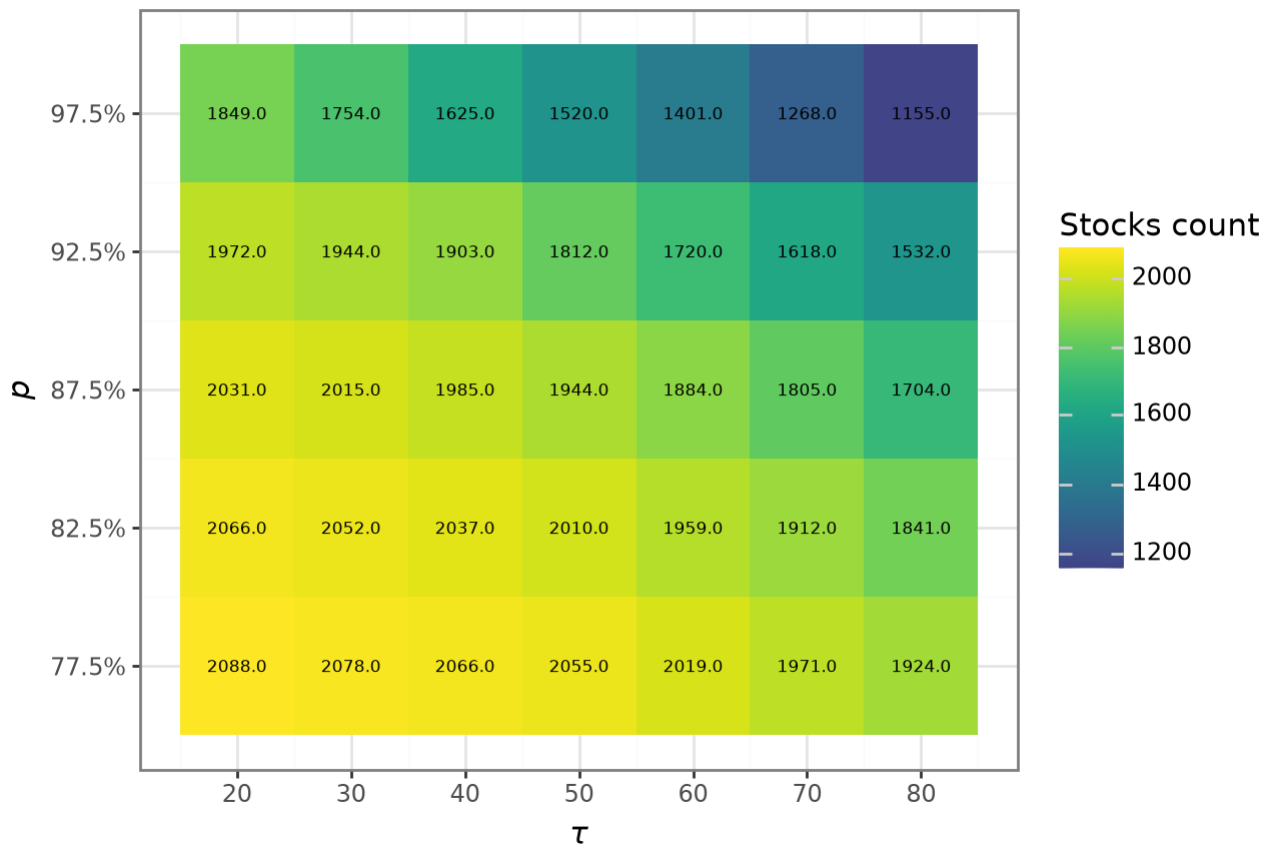


Red line: 97.5%.

To better understand how to filter stocks, consider the following heatmap:

```
plot.day_fullness_matrix(
    data_filled_min[1],
    [20, 30, 40, 50, 60, 70, 80],
    [0.775, 0.825, 0.875, 0.925, 0.975],
    data_counts_permno.shape[0]
)
```

Stocks with $p\%$ days where τ minutes had trades Across all 2364 stocks

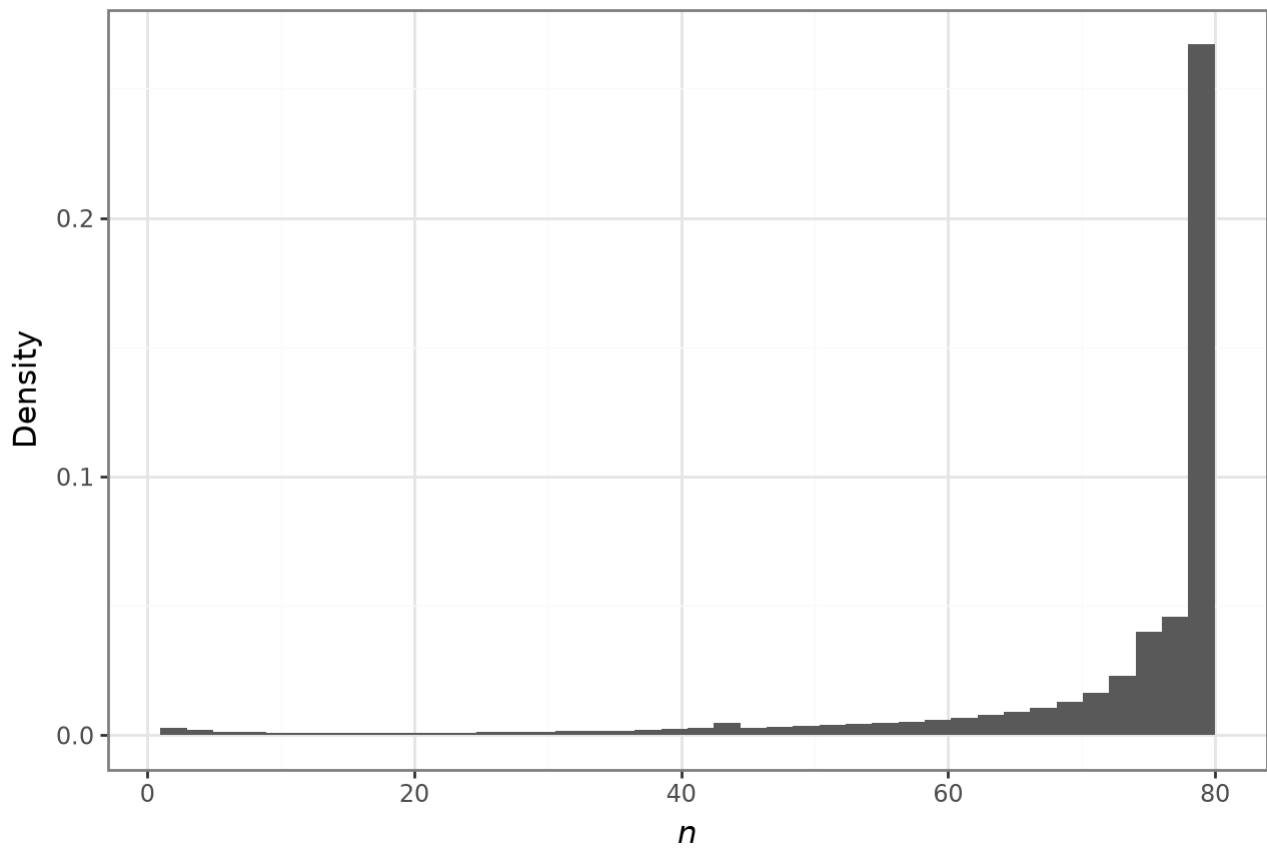


3.4.4 5-minute blocks

```
plot.day_fullness_hist(data_counts_min[5])
```

Distribution of days' observation count (n)

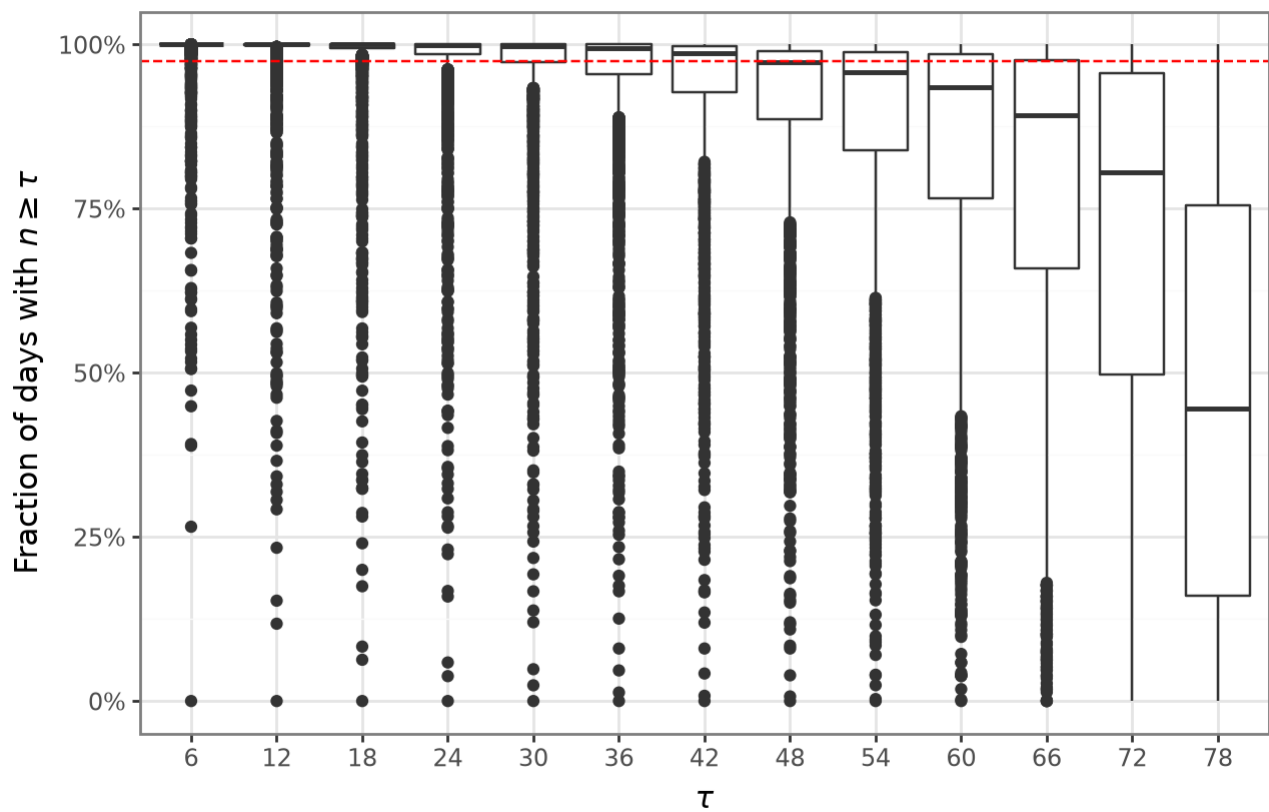
Across all 6,054,309 stocks-day pairs



```
plot.day_fullness_box(data_counts_min[5], 5, data_counts_permno.shape[0])
```

Distribution of days' observation count (n)

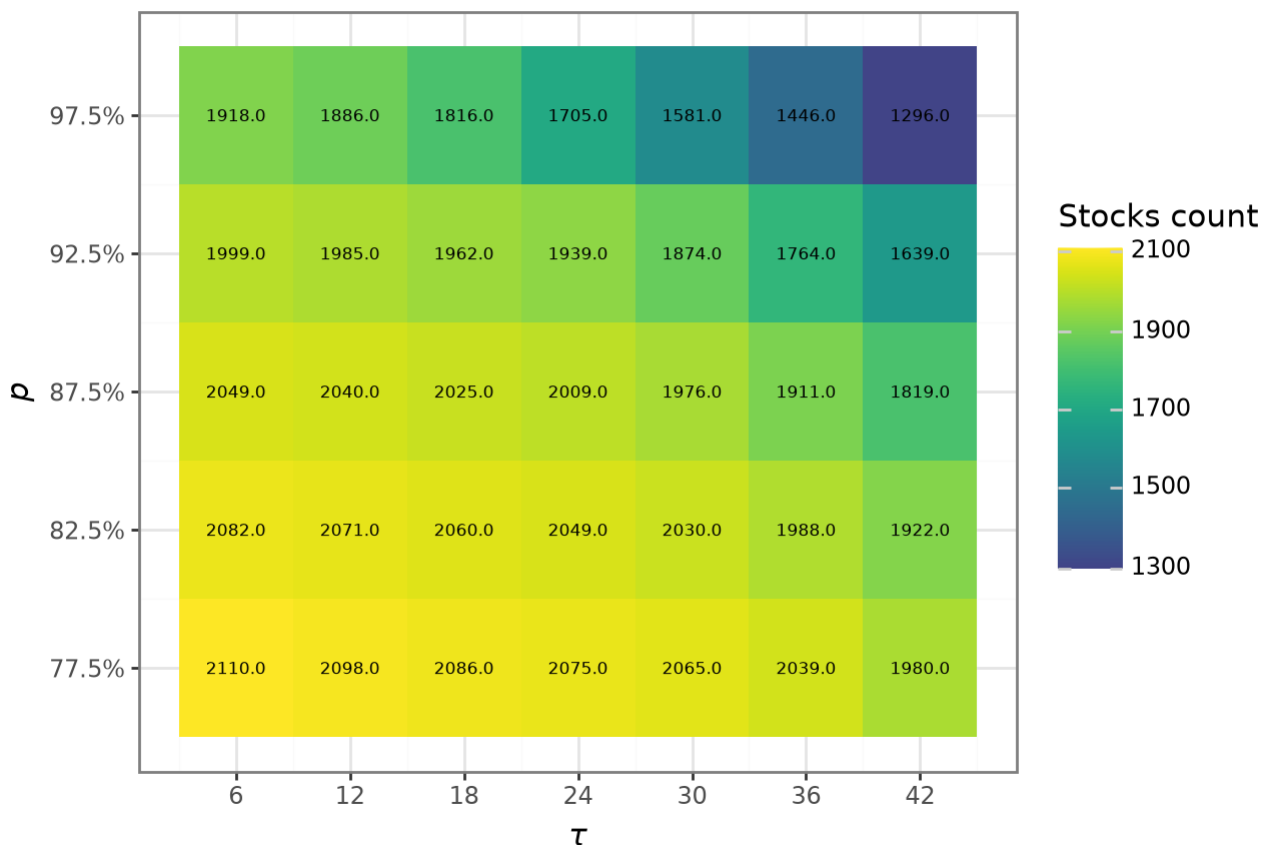
Across all 2364 stocks



Red line: 97.5%.

```
plot.day_fullness_matrix(
    data_filled_min[5],
    [6, 12, 18, 24, 30, 36, 42],
    [0.775, 0.825, 0.875, 0.925, 0.975],
    data_counts_permno.shape[0]
)
```

Stocks with $p\%$ days where τ minutes had trades Across all 2364 stocks



3.5 Aggregation of 1-minute log returns

To aggregate the data, for each `permno`:

1. For each 1-minute block m : get the last seen price before the end of m 's block, p_m .
 2. Minimal liquidity filter: at most 2.5% of the trading days (excluding pre-listing and post-delisting days) have less than 78 minutes with data.
 3. Calculate $r_m^1 = \ln(p_t/p_{m-1})$ (while not including overnight returns).
 4. Exclude non-full trading days.
- Save a [data/stocks_1min_returns.parquet](#) file with the columns `permno`, `ts_min_ny`, and `log_return`.

Some notes:

- As checked in the raw data, the observations are already at the minute level with no duplicates, so we do not actually need to do any aggregation.
- The “last seen price” is not the undisputed correct price to use for calculating returns. Other methods include the midpoint of the bid-ask spread and the volume-weighted average price.
- We could save the information about how many observations were within each minute block, but that information is not needed at the moment.

The transformation could be done with either DuckDB or Polars, both multithreaded. It could also be done in chunks if the data is too big. Polars seems to deal better with the latter, but the following DuckDB code is written to process the data in one go. See the appendix for alternatives.

```

%%register_query trade_returns_aggregation

COPY (
  WITH
  trades_by_min AS (
    SELECT
      permno,
      ts_min_ny,
      price,
      (ts_min_ny // (60 * 24)) AS d
    FROM 'data/stocks_raw/trades_clean.parquet'
  )
  SELECT
    permno,
    ts_min_ny,
    LN(price / LAG(price, 1) OVER w) AS log_return
  FROM trades_by_min
  WINDOW w AS (PARTITION BY permno, d ORDER BY ts_min_ny) -- No overnight returns
  QUALIFY
    LAG(price, 1) OVER w IS NOT NULL -- First return of the day (overnight) is NULL
)
TO 'data/stocks_1min_returns.parquet' (
  FORMAT PARQUET,
  COMPRESSION 'ZSTD',
  ROW_GROUP_SIZE 555000
);

```

```

if False: # Only execute once
    duckdb.execute(queries_registered["trade_returns_aggregation"])

```

The data will be forward filled with 0 return on minutes with no trades and will also be aggregated to the 5-minute level. However, storing these to disk is unnecessary, and it is better to create the variations on the fly.

3.5.1 Filtering stocks

Given the missing-days graphs, we can choose a filter to remove illiquid observations. The code below removes stocks that have less than $p = 97.5\%$ of days in their life with more than 20% observations missing, that is, $\tau = 78$ for one-minute data. Note that filtering the one-minute data also guarantees at least 20% observations ($\text{ceiling}(78/5) = 16$) for five-minute data.

Additionally, we remove stocks that have less than 500 days, as that is the minimum number of days needed to estimate the alphas.

```

permnos_liquid = (
  data_filled_min[1]
  .group_by("permno")
  .agg(
    is_liquid = ((pl.col("obs_count") >= 78).sum() / pl.len()) >= 0.975,
    n_obs = pl.len()
  )
)

```

```

        .filter(
            pl.col("is_liquid"),
            pl.col("n_obs") >= 500 # Alphas' window warmup size
        )
        ["permno"].to_numpy()
    )
    # TODO: remove hardcoded

print(f"Number of liquid permnos: {len(permnos_liquid)}")
np.save("data/permnos_liquid.npy", permnos_liquid)

```

Number of liquid permnos: 1141

We can save a version of the dataset filtered without the stocks, as the code below does. However, for flexibility, the PCA and beta code uses the full dataset. We only drop stocks at the time of calculating alphas, as that loop takes longer.

```

%%register_query filter_liquidity

-- ?1: array of permnos to filter

COPY (
    SELECT
        permno,
        ts_min_ny,
        log_return
    FROM 'data/stocks_1min_returns.parquet'
    WHERE permno = ANY(?1)
    ORDER BY permno, ts_min_ny
)
TO 'data/stocks_1min_returns_filtered.parquet' (
    FORMAT PARQUET,
    COMPRESSION 'ZSTD',
    ROW_GROUP_SIZE 555000
);

```

```

if False: # Only execute once
    duckdb.execute(queries_registered["filter_liquidity"], [permnos_liquid])

```

3.5.2 Visualizing raw data and aggregation

Let us visualize the work to get a first look at the data and check whether everything is as expected. First, let us sample some random stocks.

```

vis_permnos = data_counts_permno["permno"].sample(4).to_numpy()

```

We could visualize the original raw data, but very few operations were done on it. Still, one could run the following SQL and Polars code to generate a dataset to visualize:

```

%%register_query raw_permno_sample

-- ?1: list of integer permnos to filter

SELECT
    CAST(permno AS UINT32) AS permno,
    CAST(ts_ny // 60 AS UINT32) AS ts_min_ny,
    CAST(price AS FLOAT8) AS price,
FROM db_trades.trades
WHERE
    permno IN ?1

```

```

data_permno_raw = (
    duckdb.sql(queries_registered["raw_permno_sample"], params = [vis_permnos.tolist()])
    .pl()
    .with_columns(
        datetime = pl.from_epoch(pl.col("ts_min_ny") * 60, time_unit = "s")
    )
    .select(["permno", "datetime", "price"])
)

```

The code below plots the clean prices and the returns aggregation. Both are aggregated to the 30-day level to make the relationship between the two clearer.

The relationship between the two looks as expected.

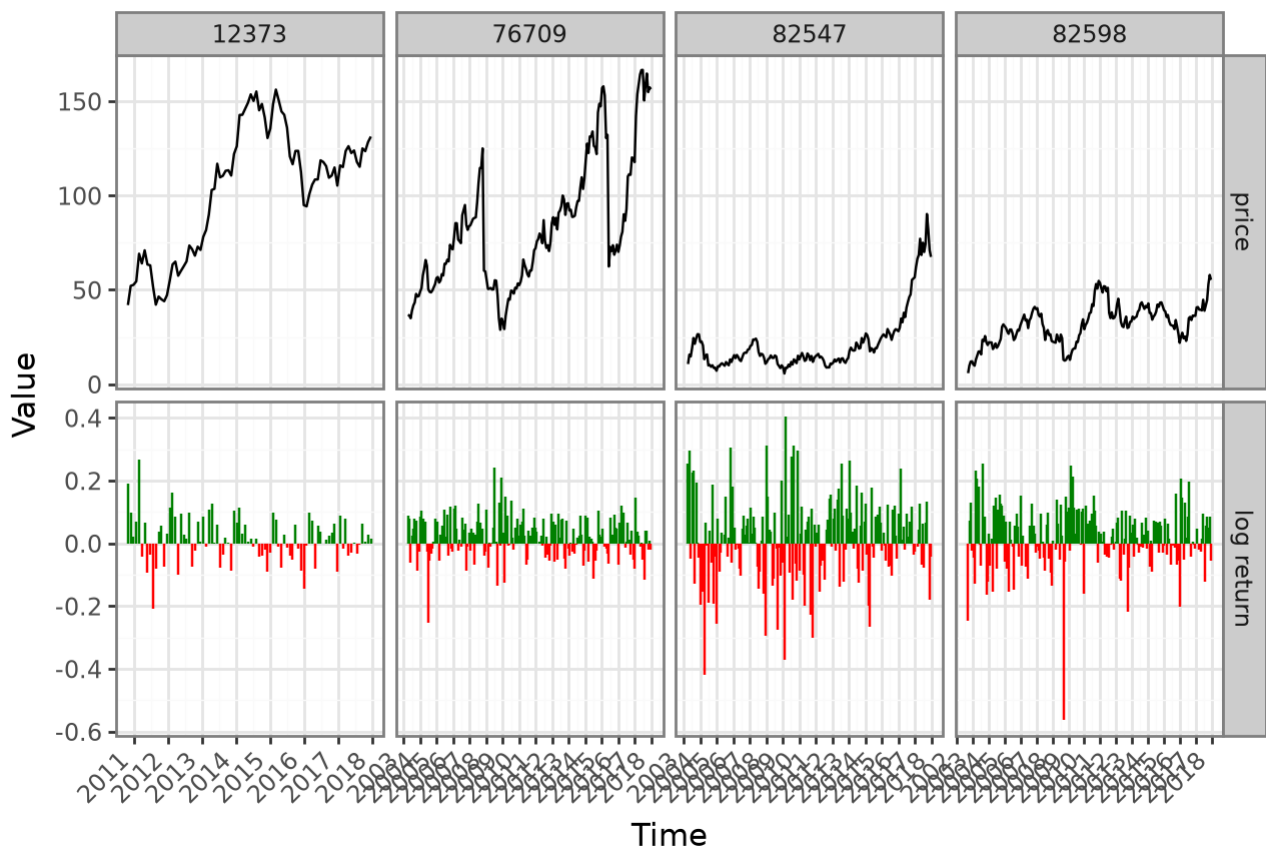
```

plot.prices_returns(
    pl.scan_parquet("data/stocks_raw/trades_clean.parquet")
    .filter(pl.col("permno").is_in(vis_permnos))
    .collect(),
    pl.scan_parquet("data/stocks_1min_returns.parquet")
    .filter(pl.col("permno").is_in(vis_permnos))
    .collect()
)

```

Price and log returns across time

Each column is a randomly selected PERMNO

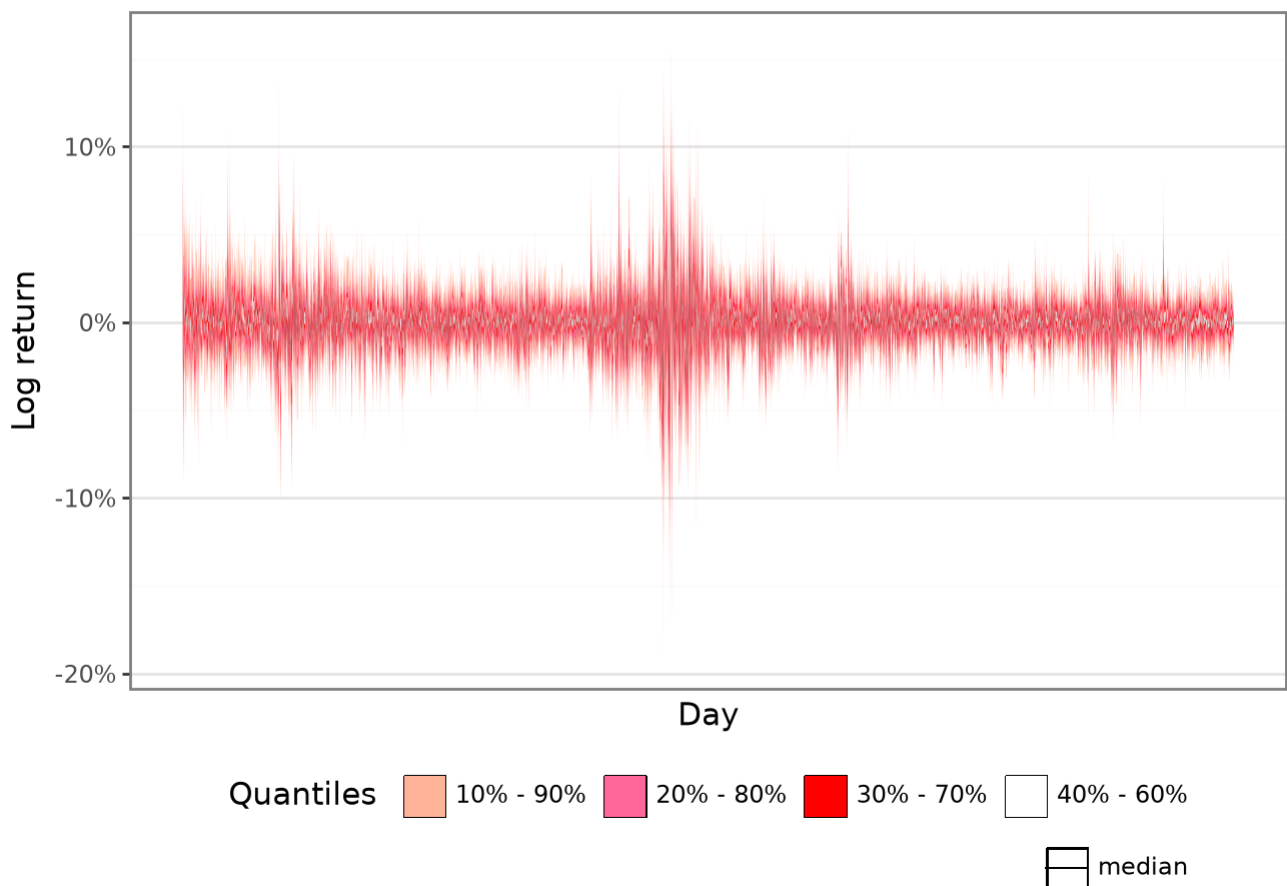


Then, we plot the river graph of the daily aggregated returns:

```
plot.returns_river(
  (
    pl.scan_parquet("data/stocks_1min_returns.parquet")
    .filter(
      pl.col("permno").is_in(
        data_counts_permno["permno"].sample(100).to_numpy()
      )
    )
    .group_by(pl.col("ts_min_ny") // (60 * 24), "permno")
    .agg(pl.col("log_return").sum())
  ),
  x = "ts_min_ny", y = "log_return",
  labs = {
    "title": "Log returns across time", "subtitle": "For 100 random stocks",
    "y": "Log return", "x": "Day", "color": ""
  }
)
```

Log returns across time

For 100 random stocks



3.6 Appendix

If the calculations above are too harsh for the available machine's memory, one can do them in chunks of `permno` values. The chunks are defined below, striving for a balance in the number of rows in each chunk. Each chunk has 7% of the rows, a number defined by the memory constraints (16 GB RAM) of the machine used to process the data.

```
agg_chunk_fraction = 0.07
agg_chunk_cutoff = math.floor(
    int(data_counts_permno["obs_count"].sum()) * agg_chunk_fraction * 0.95
) # 0.95 avoids last chunk being too small
agg_total_chunks = math.ceil(1 / agg_chunk_fraction)

chunks = []
current_stock = 0

for i in range(agg_total_chunks):
    c = []
    c_rows = 0
    while c_rows < agg_chunk_cutoff and current_stock < len(data_counts_permno):
        c.append(data_counts_permno["permno"][current_stock])
        c_rows += data_counts_permno["obs_count"][current_stock]
        current_stock += 1
```

```

    if i == agg_total_chunks - 1:
        c.extend(data_counts_permno["permno"][current_stock:].to_list())
    chunks.append(c)

if sum(len(c) for c in chunks) != len(data_counts_permno):
    raise ValueError("Some stocks are missing from the chunks.")

print(f"Number of chunks: {len(chunks)}")
print("Number of stocks in each chunk: ", end = "")
pprint([len(c) for c in chunks], width = 50, compact = True)
print("Rows/1M per chunk (approx): ", end = "")
pprint([
    round(
        sum(data_counts_permno.filter(pl.col("permno").is_in(c))["obs_count"])
        / 1_000_000,
        1
    )
    for c in chunks
], width = 50, compact = True)

```

```

Number of chunks: 15
Number of stocks in each chunk: [168, 328, 103, 106, 111, 110, 106, 141, 142, 150,
166, 164, 163, 190, 216]
Rows/1M per chunk (approx): [107.0, 108.1, 107.7, 107.6, 107.1, 107.5, 107.0,
106.9, 107.3, 107.7, 107.9, 106.9, 107.1, 107.0,
103.3]

```

```

data_trades_lazy = pl.scan_parquet(
    "data/stocks_raw/trades_clean.parquet",
    parallel = "row_groups",
    low_memory = False
)

def lazy_pipeline_agg(chunk: list[int]) -> pl.LazyFrame:
    trading_days_extra = PARS.trading_days[
        ~ np.isin(PARS.trading_days, PARS.trading_days_final)
    ].astype("int32")

    pipeline = (
        data_trades_lazy
        .filter(pl.col("permno").is_between(chunk[0], chunk[-1]))
        #.sort(["permno", "ts_min_ny"]) # Already sorted
        .with_columns(
            d = (pl.col("ts_min_ny") // (60 * 24))
        )
        .with_columns(
            log_return = (
                pl.col("price") / (
                    pl.col("price")
                    .shift(1).over(["permno", "d"]) # Never cross days
                )
                # order_by = "ts_min_ny" is unneeded
            )
        )
    )

```

```

        )
        ).log(),
    )
    .filter(
        pl.col("log_return").is_not_null() # Excludes first null return of day
        & (~ pl.col("ts_min_ny").is_in(trading_days_extra))
    )
    .select(["permno", "ts_min_ny", "log_return"])
)
return pipeline

```

For each chunk, the pipeline is executed and the results are saved in a parquet file such as [data/stocks_1min_returns/chunk_1.parquet](#):

```

if False: # Only execute once
    for i, chunk in enumerate(chunks[-1:]):
        print(f"Processing chunk {i+1}/{len(chunks)} with {len(chunk)} permnos")
        lazy_pipeline_agg(chunk).sink_parquet(
            f"data/stocks_1min_returns/chunk_{i}.parquet",
            compression = "zstd",
            engine = "streaming",
            row_group_size = 555_000
        )

```

An alternative to the Polars code is the DuckDB code below.

```

%%register_query trade_returns_aggregation_chunked

-- ?1: list of permnos to filter
-- ?2: chunk file path

COPY (
    WITH
    trades_by_min AS (
        SELECT
            permno,
            ts_min_ny,
            price,
            (ts_min_ny // (60 * 24)) AS d
        FROM 'data/stocks_raw/trades_clean.parquet'
        WHERE
            permno BETWEEN ?1[0] AND ?1[-1]
    )
    SELECT
        permno,
        ts_min_ny,
        LN(price / LAG(price, 1) OVER w) AS log_return
    FROM trades_by_min
    WINDOW w AS (PARTITION BY permno, d ORDER BY ts_min_ny)
    QUALIFY
        LAG(price, 1) OVER w IS NOT NULL

```

```
)
TO ?2 (
    FORMAT PARQUET,
    COMPRESSION 'ZSTD',
    ROW_GROUP_SIZE 555000
);
```

```
if False: # Run polars instead
    for i, chunk in enumerate(chunks[-1:]):
        print(f"Processing chunk {i+1}/{len(chunks)} with {len(chunk)} permnos")
        duckdb.execute(
            queries_registered["trade_returns_aggregation_chunked"],
            [chunk, f"data/stocks_1min_returns/chunk_{i}.parquet"]
        )
```

4 Factor PCA

Now we need to get the principal components of the Fama-French 6 factors.

Standard PCA suffers from:

- **Curse of dimensionality:** Estimating covariance matrices for large cross-sections requires long time horizons to avoid singularity.
- **Non-stationarity:** The hypothesis of a constant covariance matrix across time can be too strong in some scenarios, including this one.
- **Non-linearities:** PCA on a single covariance matrix, a linear measure of relationships, for the whole process cannot properly capture non-linear relationships.

Here we apply Aït-Sahalia and Xiu (2015), which uses high-frequency data to estimate spot covariance matrices and their principal components. With it, we relax the stationarity assumptions needed and approximate non-linear relationships given Itô's Lemma.

The following math and commentary are largely taken as-is from the paper. First, we present the theory, then the implementation in Python, and finally the results.

4.1 Methodology

4.1.1 Dynamics of the variable

The d -dimensional (in our case $d = 6$) process X of the factors is a general Itô-semimartingale, defined on a filtered probability space:

$$X_t = X_0 + \int_0^t b_s ds + \int_0^t \sigma_s dW_s + J_t \quad (2)$$

- X_t is the d -dimensional log-price (or cumulative factor) process.
- b_s is the d -dimensional locally bounded drift vector.
- σ_s is the spot volatility matrix, linking to the spot covariance via $c_s = \sigma_s \sigma_s^T$.
- W_s is a d -dimensional standard Brownian motion.
- J_t is a pure jump process accounting for discrete anomalous shocks. (More information in the paper).

4.1.2 Parameters of interest

The PCA goal is to find instantaneous linear combinations of X , orthogonal to each other, that maximize some measure of variation. In the paper's case, the continuous part of the quadratic variation.

Let $\mathcal{M}_d^+$ denote the set of $d \times d$ positive semi-definite symmetric matrices. For any $A \in \mathcal{M}_d^+$, we define:

- The d -dimensional vector of its eigenvalues sorted in non-increasing order: $\lambda(A) = (\lambda_1(A), \lambda_2(A), \dots, \lambda_d(A))^T$.

- The eigenvector corresponding to the g -th eigenvalue $\lambda_g(A)$ as $\gamma_g(A)$, which satisfies: $A\gamma_g = \lambda_g\gamma_g$ and $\gamma_g^T\gamma_g = 1$.

Our interest is calculating the $T \times d$ matrix of principal components, where T is the total number of (high-frequency) observations. This is related to the paper's derivation of the integrated/realized principal component of the g -th eigenvalue λ_g , $\int_0^t \gamma_{g,s}^T dX_s$.

The paper focuses on the estimation of the realized eigenvalues ($\int_0^t \lambda_s ds$) and eigenvectors ($\int_0^t \gamma_{g,s} ds$), while deriving their asymptotic properties and bias corrections. We focus on the PCs, but at Section 4.5 we summarize the assumptions for the asymptotic properties and the unused estimators.

4.1.3 Principal components estimator

4.1.3.1 Spot covariance matrix

The first step for all is estimating a spot covariance matrix and its associated eigenvalues. To capture non-stationary dynamics, the time window $[0, t]$ in increments of Δ_n is divided into non-overlapping blocks of length $k_n\Delta_n$. Within each small block, the continuous covariance c_s is assumed to be roughly constant. By dynamically estimating instantaneous matrices and summing them, the procedure continuously adapts to shifting correlations, capturing non-linearities without requiring global parameter estimates.

At each $ik_n\Delta_n$, we estimate $c_{ik_n\Delta_n}$ using the block spot covariance estimator $\hat{c}_{ik_n\Delta_n}$ defined as:

$$\hat{c}_{ik_n\Delta_n} = \frac{1}{k_n\Delta_n} \sum_{j=1}^{k_n} (\Delta_{ik_n+j}^n X) (\Delta_{ik_n+j}^n X)^T \mathbf{1} \{ \|\Delta_{ik_n+j}^n\| \leq u_n \} \quad (5)$$

Where the return increment is: $\Delta_l^n X = X_{l\Delta_n} - X_{(l-1)\Delta_n}$, and u_n is the jump truncation threshold. The k and u parameters can be chosen with the shorthands proposed by the literature:

- **Block Size** (k_n): the divisor of $[t/\Delta_n]$ closest to $\theta\Delta_n^{-1/2} \sqrt{\log(d)}$, with $\theta = 0.5$. Other choices of θ and functions of d deliver similar results. It scales inversely with the sampling frequency (Δ_n) and grows slightly with dimensionality (d) to ensure stable covariance estimates without smoothing over true time-varying dynamics.
- **Jump Truncation** ($u_{i,n}$): Set asset-by-asset as $u_{i,n} = 3 \left(\int_0^t c_{ii,s} ds / t \right)^{0.5} \Delta_n^{0.47}$. Where the internal integral can be estimated with Bipower Variation. This removes discrete jumps while preserving continuous diffusion.

The instantaneous eigenvalues $\hat{\lambda}_{ik_n\Delta_n} = \lambda(\hat{c}_{ik_n\Delta_n})$ are obtained from the standard determinant equation:

$$\det(\hat{c}_i - \hat{\lambda}\mathbb{I}) = 0$$

4.1.3.2 Integrated principal component

The paper focuses on the integrated principal components, whose estimator is defined as:

$$\widehat{PC}_g = \sum_{i=1}^{\lceil t/(k_n \Delta_n) \rceil - 1} \hat{\gamma}_{g, (i-1)k_n \Delta_n}^T (X_{(i+1)k_n \Delta_n} - X_{ik_n \Delta_n}) \quad (\text{Proposition 3})$$

Because the estimator is a stochastic integral, the weight errors are uncorrelated with the forward returns. The estimation noise naturally cancels out, removing the need for a Hessian bias correction. PCs use the eigenvectors estimated from the immediately preceding block to project the current block's returns, ensuring that our dynamic portfolio weights are strictly “adapted” (determined in-sample before trading) and free from microscopic look-ahead bias.

However, we want a $T \times d$ matrix of principal components, not just the integrated one. So we basically do not want to run the summation and instead store the intermediate results. The pseudocode below outlines the algorithm.

4.1.4 Estimator algorithm

To estimate the $T \times d$ matrix of principal components, we must project each individual high-frequency return observation onto the previous block's eigenvector. The Pascal-like pseudocode below outlines the procedure.

This code was written in matrix form, as it will be a much more effective way to implement in Python. Still, it should be equivalent to the original paper's algorithm.

```
procedure pca_high_freq(  
  X : Matrix;           // Shape: (T, d) | high-frequency return matrix  
  k : Integer;          // Scalar      | observations per block  
  u : Matrix;           // Shape: (d,)  | jump truncation thresholds  
  evecs_prev : Matrix // Shape: (d, d) | initial eigenvectors  
);  
  
  // 1. Parameter Setup & Initialization  
  T = number of rows in X  
  d = number of cols in X  
  Delta = 1 / T  
  N = Floor(T / k) // Total number of full blocks  
  
  // Initialize empty arrays to store outputs:  
  PC = empty Matrix of size T x d  
  E_vals = empty Matrix of size N x d  
  E_vecs = empty Matrix of size N x d x d  
  
  // 2. Process Data Block by Block  
  for i = 0 to N - 1 do  
    begin  
      // Define the current block  
      block_start = i * k  
      block_end = (i + 1) * k  
      X_block = X[block_start to block_end]
```

```

// Calculate PCs for the current block using the previous eigenvectors:
PC[block_start to block_end] = X_block * evecs_prev

// Truncate X with boolean mask and calculate the spot covariance matrix:
X_trunc := X_block * (Abs(X_block) <= u)
cov_spot = (Transpose(X_trunc) * X_trunc) / (k * Delta)

// Eigen decomposition:
evalues, evecs = eigen decomposition of cov_spot, descending evalues

// Store results and update eigenvectors for the next block:
E_vals[i, :] = evalues
E_vecs[i, :, :] = evecs
evecs_prev = evecs
end;

// 3. Handling the last block if (T % k != 0)
block_start = N * k
if block_start < T then
begin
    X_block = X[block_start to T]
    PC[block_start to T] = X_block * evecs_prev
end;

return PC, E_vals, E_vecs
end;

```

4.1.5 Specifics of our setup

We implement the above algorithm for the Fama-French 6 factors. One important thing to note is that we treat the timeline with gaps on non-business days, non-trading hours, and the removed 9:30 return as a continuous timeline for the PCA. While this is standard, it is still important to note.

In our case, t was normalized to 1. We thus have T minute-level observations in the interval $[0, t]$, spanning the whole time series window. The same is true for the 5-minute-level exercise (with a different T value).

4.2 Code implementation

Here we implement the algorithm described above in Python.

First the setup, with debugging:

```

%reset -f
import importlib

```

And core modules:

```

import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import polars as pl
import numpy as np

from tqdm import tqdm

from numpy.typing import NDArray

import src.utils as ut
import src.graphs as plot
from src.parameters import PARAMETERS as PARS
importlib.reload(ut)
importlib.reload(plot)

ut.set_seed()

```

First, we implement the rules of thumb used in the literature to set the parameters k and u :

```

def default_k(d: int, T: int, Delta: float) -> int:
    theta = 0.5

    k_target = int(np.round(theta * (Delta**-0.5) * np.sqrt(np.log(d))))
    divisors = np.array([i for i in range(1, T + 1) if T % i == 0])
    closest_idx = np.argmin(np.abs(divisors - k_target))

    return int(divisors[closest_idx])

def default_u(X: NDArray, T: int, Delta: float) -> NDArray:
    X_abs = np.abs(X)
    bpv = (np.pi / 2) * np.sum(X_abs[1:] * X_abs[:-1], axis = 0)
    u = 3 * np.sqrt(bpv / 1) * (Delta ** 0.47) # bpv/1:  $\$t\$$  normalized to 1
    return u

```

Then, to start the PCA algorithm, we need to estimate the first block's eigenvectors. We can use the warm-up data selected in [src/data_factors.ipynb](#) / Chapter 2 to achieve that. Otherwise, the first block would be left with `np.nan`.

```

def pca_start_evectors(
    X: NDArray, k: int | None = None, u: NDArray | None = None,
    quiet: bool = True
) -> NDArray:
    ut.pca_high_freq_validate(X, k, u, None)

    T: int = X.shape[0]
    d: int = X.shape[1]
    Delta: float = 1 / T

    if k is None:
        k = default_k(d, T, Delta)

```

```

        if not quiet: print(f"Calculated `k` as {k}.")
    if u is None:
        u = default_u(X, T, Delta)
        if not quiet: print(f"Calculated `u` as {u}.")

    _, evecs = pca_truncated(X, k * Delta, u)

    return evecs

```

Finally, we implement the main algorithm. First, the simple PCA with truncation engine, for the stationary data within each block:

```

def pca_truncated(
    X: NDArray, # Shape: (k, d) a block of the original X
    n: float, # k * Delta
    u: NDArray # Shape: (d,)
):
    X_trunc = X * (np.abs(X) <= u) # x*0 for big values, else x*1
    cov_spot = (X_trunc.T @ X_trunc) / n # No need to demean following the paper

    evals, evecs = np.linalg.eigh(cov_spot)
    evals_order = np.argsort(evals)[::-1]

    return evals[evals_order], evecs[:, evals_order]

```

Then, the main loop that iterates over the blocks, runs the PCA, and stores results:

```

def pca_high_freq_core(
    N: int, k: int, T: int, Delta: float,
    u: NDArray, # Shape: (d,)
    X: NDArray, # Shape: (T, d)
    evecs_prev: NDArray, # Shape: (d, d)
    PC: NDArray, # Shape: (T, d)
    E_vals: NDArray, # Shape: (N, d)
    E_vecs: NDArray, # Shape: (N, d, d)
    quiet: bool = False
):
    for i in tqdm(range(N), total = N, disable = quiet): # Last block after loop
        block_start, block_end = i * k, (i + 1) * k

        X_block = X[block_start:block_end, :]

        PC[block_start:block_end, :] = X_block @ evecs_prev

        evals_prev, evecs_prev = pca_truncated(X_block, k * Delta, u)
        E_vals[i, :] = evals_prev
        E_vecs[i, :, :] = evecs_prev

    block_start = N * k
    if block_start < T:
        X_block = X[block_start:T, :]

```

```

        PC[block_start:T, :] = X_block @ evecs_prev

    return PC, E_vals, E_vecs

```

Finally, the main orchestrator that validates the inputs, sets the default parameters, and runs the main loop:

```

def pca_high_freq(
    X: NDArray, # Shape: (T, d)
    k: int | None = None,
    u: NDArray | None = None, # Shape: (d,)
    evecs_start: NDArray | None = None, # Shape: (d, d)
    quiet: bool = False
) -> dict[str, NDArray]:
    # Setup:
    ut.pca_high_freq_validate(X, k, u, evecs_start)
    X.flags.writeable = False

    T: int = X.shape[0] # Number of observations (rows)
    d: int = X.shape[1] # Number of variables (columns)
    Delta: float = 1 / T

    if k is None:
        k = default_k(d, T, Delta)
        if not quiet: print(f"Calculated `k` as {k}.")
    if u is None:
        u = default_u(X, T, Delta)
        if not quiet: print(f"Calculated `u` as {u}.")

    # Warm up:
    N: int = T // k # Number of blocks - 1
    if evecs_start is None:
        evecs_prev = np.full((d, d), np.nan)
    else:
        evecs_prev = evecs_start # Shape: (d, d)

    PC = np.full((T, d), np.nan) # Shape: (T, d)
    E_vals = np.full((N, d), np.nan) # Shape: (N, d)
    E_vecs = np.full((N, d, d), np.nan) # Shape: (N, d, d)

    # Main loop:
    if not quiet: print(f"Starting PCA with {N} blocks of size {k}.")
    PC, E_vals, E_vecs = pca_high_freq_core(
        N = N, k = k, T = T, Delta = Delta, u = u,
        X = X, evecs_prev = evecs_prev,
        PC = PC, E_vals = E_vals, E_vecs = E_vecs,
        quiet = quiet
    )

    return {"pc": PC, "evals": E_vals, "evecs": E_vecs}

```

Some performance considerations:

- The `pca_high_freq_core()` must be run sequentially.
- Given the low dimensionality of the factor data, using GPU with e.g. `cupy` would add unnecessary overhead. Still, the code could be translated basically as-is with `import cupy as np` instead.
- It could be beneficial to lock multithreading of NumPy's BLAS/LAPACK matrix engines.
- Ensuring that `X` is a contiguous array in memory could help performance.
- Finally, the code could be just-in-time compiled with `numba`, and the separation of `pca_high_freq_core()` helps with that. One would only need to decorate it with `@numba.jit`.

The code runs super fast, so these complications are not needed.

4.3 Calculating the PCA

4.3.1 Loading data

First, I load the data processed in [src/data_factors.ipynb](#) / Chapter 2.

```
blocks_size = {1: 858, 5: 384}

data_raw = {
    1: pl.read_csv("data/factors_1min_returns.csv"),
    5: pl.read_csv("data/factors_5min_returns.csv")
}

data_raw = {
    min: df.with_columns(
        pl.col("datetime").str.strptime(pl.Datetime, format = "%Y-%m-%d %H:%M")
    )
    for min, df in data_raw.items()
}

cols_fct = {
    "ff__mkt": "market",
    "ff__smb": "size",
    "ff__hml": "value",
    "ff__rmw": "profitability",
    "ff__cma": "investment",
    "ff__umd": "momentum",
}
```

Then, we separate the added warm-up data from the main part from which we want the PCs:

```
data_warmup = {
    min: (
        df
        .filter(pl.col("datetime").dt.date() < PARS.date_start.item())
        [list(cols_fct.keys())]
        .to_numpy()
    )
    for min, df in data_raw.items()
}
```

```
data_main = {
    min: df.filter(pl.col("datetime").dt.date() >= PARS.date_start.item())
    for min, df in data_raw.items()
}
```

4.3.2 Running algorithm

First, let us inspect the default k and u parameters that will be generated. It only cuts the higher quantiles of the data.

```
pca_start_evecors(data_main[1][list(cols_fct.keys())].to_numpy(), quiet = False)
u_default = [0.00182358, 0.0012769, 0.00082141, 0.00055697, 0.00055211, 0.00102862]

print("\nCalculated `u` and quantiles of the absolute factors:")
(
    pl.concat(
        [
            pl.DataFrame({
                col: val * 100 for col, val in zip(cols_fct.values(), u_default)
            })
        ] + [
            data_main[1].select(cols_fct).select(pl.all().abs().quantile(q) * 100)
            for q in [0.995, 0.99, 0.5]
        ]
    )
    .with_columns(
        statistic = pl.Series(["u"] + [f"factor's Q{q}" for q in [0.995, 0.99, 0.5]])
    )
    .select(["statistic"] + list(cols_fct.values()))
    .rename(cols_fct)
)
```

Calculated `k` as 858.

Calculated `u` as [0.00182358 0.0012769 0.00082141 0.00055697 0.00055211 0.00102862].

Calculated `u` and quantiles of the absolute factors:

stat	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
str	f64	f64	f64	f64	f64	f64
"u"	0.182358	0.12769	0.082141	0.055697	0.055211	0.102862
"factor's Q0.995"	0.191411	0.127203	0.08511	0.052444	0.052304	0.110336
"factor's Q0.99"	0.149494	0.101947	0.067479	0.042692	0.042699	0.086969
"factor's Q0.5"	0.013916	0.010854	0.00696	0.005668	0.005523	0.007894

Then, we run the results for each minute-block-size:

```
pca_1min = pca_high_freq(
    X = data_main[1][list(cols_fct.keys())].to_numpy(),
    evectors_start = pca_start_evector(data_warmup[1]),
    quiet = True
)
```

```
pca_5min = pca_high_freq(
    X = data_main[5][list(cols_fct.keys())].to_numpy(),
    evectors_start = pca_start_evector(data_warmup[5]),
    quiet = True
)
```

Saving results:

```
n_pcs = pca_1min["pc"].shape[1]
cols_pc = {f"column_{i}": f"pc_{i + 1}" for i in range(n_pcs)}
```

```
pcas = {1: pca_1min, 5: pca_5min}

data_pcs = {
    k: (
        pl.DataFrame(res["pc"])
        .rename(cols_pc)
        .with_columns(
            datetime = data_main[k]["datetime"],
            block = pl.row_index() // blocks_size[k]
        )
        .select(["datetime", "block" + list(cols_pc.values())])
    )
    for k, res in pcas.items()
}
```

```
data_pcs[1].write_csv("data/pca_1min.csv", datetime_format = "%Y-%m-%d %H:%M")
data_pcs[5].write_csv("data/pca_5min.csv", datetime_format = "%Y-%m-%d %H:%M")
```

4.4 Visualizing results

Let us visualize the results as a first look and to check their reasonableness. Both visualizations are useful for comparing with the raw factors plotted in [src/data_factors.ipynb](#) / Chapter 2.

To start, let us visualize the resulting datasets.

One-minute results:

```
data_pcs[1].rename(lambda x: x.replace("_", " "))
```

datetime	block	pc_1	pc_2	pc_3	pc_4	pc_5	pc_6
datetime[s]	u32	f64	f64	f64	f64	f64	f64
2001-01-02 09:30:00	0	-0.000154	0.000016	-0.00051	0.000063	0.000336	-0.000114

datetime datetime[s]	block u32	pc_1 f64	pc_2 f64	pc_3 f64	pc_4 f64	pc_5 f64	pc_6 f64
2001-01-02 09:31:00	0	0.000537	0.000525	-0.000502	-0.000011	0.000106	-0.000005
2001-01-02 09:32:00	0	0.000371	-0.00026	-0.000674	-0.000094	-0.000139	-0.000131
2001-01-02 09:33:00	0	0.001591	0.000637	0.000092	0.000128	-0.000198	-0.000318
2001-01-02 09:34:00	0	0.001138	0.000373	-0.000498	0.000055	-0.000262	-0.000116
...	...	...	...	...	...	...	...
2017-12-29 15:55:00	1919	0.000099	-0.000078	-0.000025	0.000016	0.000013	-0.000065
2017-12-29 15:56:00	1919	0.000108	-0.000052	-0.000007	-0.000003	0.000034	0.000017
2017-12-29 15:57:00	1919	0.000129	-0.000089	0.000145	-0.000099	-0.00007	0.000054
2017-12-29 15:58:00	1919	-0.000002	-0.000009	0.000012	0.000012	-0.000171	0.000012
2017-12-29 15:59:00	1919	0.000184	-0.001405	0.000664	-0.00039	0.000102	-0.000261

And five-minute results:

```
data_pcs[5].rename(lambda x: x.replace("_", " "))
```

datetime datetime[s]	block u32	pc_1 f64	pc_2 f64	pc_3 f64	pc_4 f64	pc_5 f64	pc_6 f64
2001-01-02 09:30:00	0	0.003726	0.002074	-0.000139	0.000391	0.00043	0.000366
2001-01-02 09:35:00	0	-0.000655	0.003066	-0.002053	0.000727	0.000623	-0.000289
2001-01-02 09:40:00	0	-0.00405	0.000519	-0.002211	-0.000739	-0.001156	0.001771
2001-01-02 09:45:00	0	0.001775	0.000073	0.000385	-0.000416	-0.000813	0.001325
2001-01-02 09:50:00	0	0.001435	0.001594	-0.000542	-0.000514	-0.001209	0.000588
...	...	...	...	...	...	...	...
2017-12-29 15:35:00	857	-0.000216	0.000176	0.000693	0.000143	-0.000309	0.000039
2017-12-29 15:40:00	857	0.000002	0.00005	0.000292	0.000139	0.000093	-0.00013
2017-12-29 15:45:00	857	0.00021	0.000259	-0.000432	0.000171	0.000151	-0.000029
2017-12-29 15:50:00	857	0.000579	0.001401	0.000888	0.000196	-0.000146	-0.000361
2017-12-29 15:55:00	857	-0.000324	-0.000053	0.001916	0.000188	-0.00002	0.000153

And the related statistics:

```
(
  data_pcs[1]
  .select(cols_pc)
  .rename(lambda x: x.replace("_", " "))
  .describe()
)
```

```
(
  data_pcs[5]
  .select(cols_pc)
  .rename(lambda x: x.replace("_", " "))
  .describe()
)
```

Now, before plotting some graphs, let us store intermediate results for further analysis below.

```

n_fcts = len(cols_fct)
n_blocks = {k: res["evalues"].shape[0] for k, res in pcas.items()}

cols_evalue = {f"column_{i}": f"evalue_{i + 1}" for i in range(n_pcs)}
cols_evector = {f"column_{i}": f"evect_{i + 1}" for i in range(n_pcs)}

data_evals = {
    k: (
        pl.DataFrame(res["evalues"])
        .with_columns(block = pl.row_index())
        .rename(cols_evalue)
    )
    for k, res in pcas.items()
}

data_evecs = {
    k: (
        pl.DataFrame(res["evectors"].transpose(0, 1, 2).reshape((-1, n_fcts)))
        .rename(cols_evector)
        .with_columns(
            fct = pl.repeat(cols_fct, n_blocks[k]).explode(),
            block = pl.arange(0, n_blocks[k]).repeat_by(n_fcts).list.explode()
        )
    )
    for k, res in pcas.items()
}

```

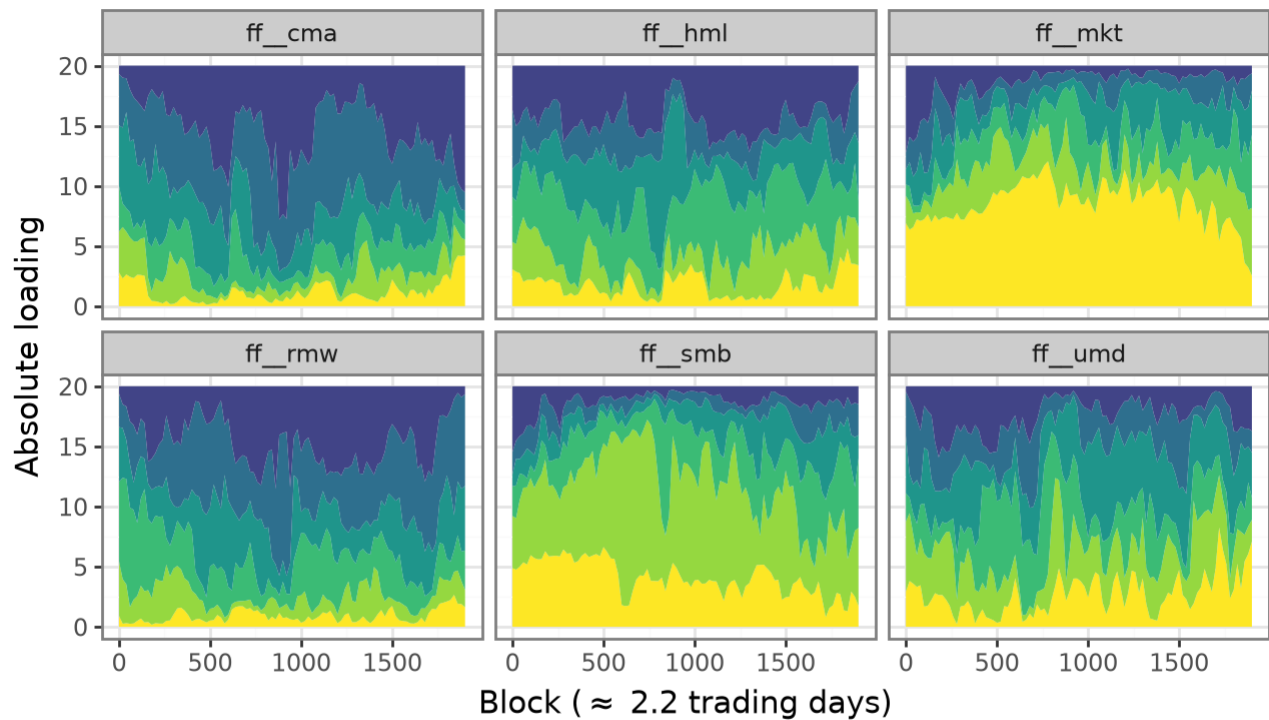
The factor loadings for each factor for all PCs across time:

```

plot.factor_loadings(
    data_evecs[1].with_columns(pl.col("fct").replace(cols_fct)),
    list(cols_evector.values()),
    blocks_size[1] / PARS.blocks_1min,
    exp = 1
)

```

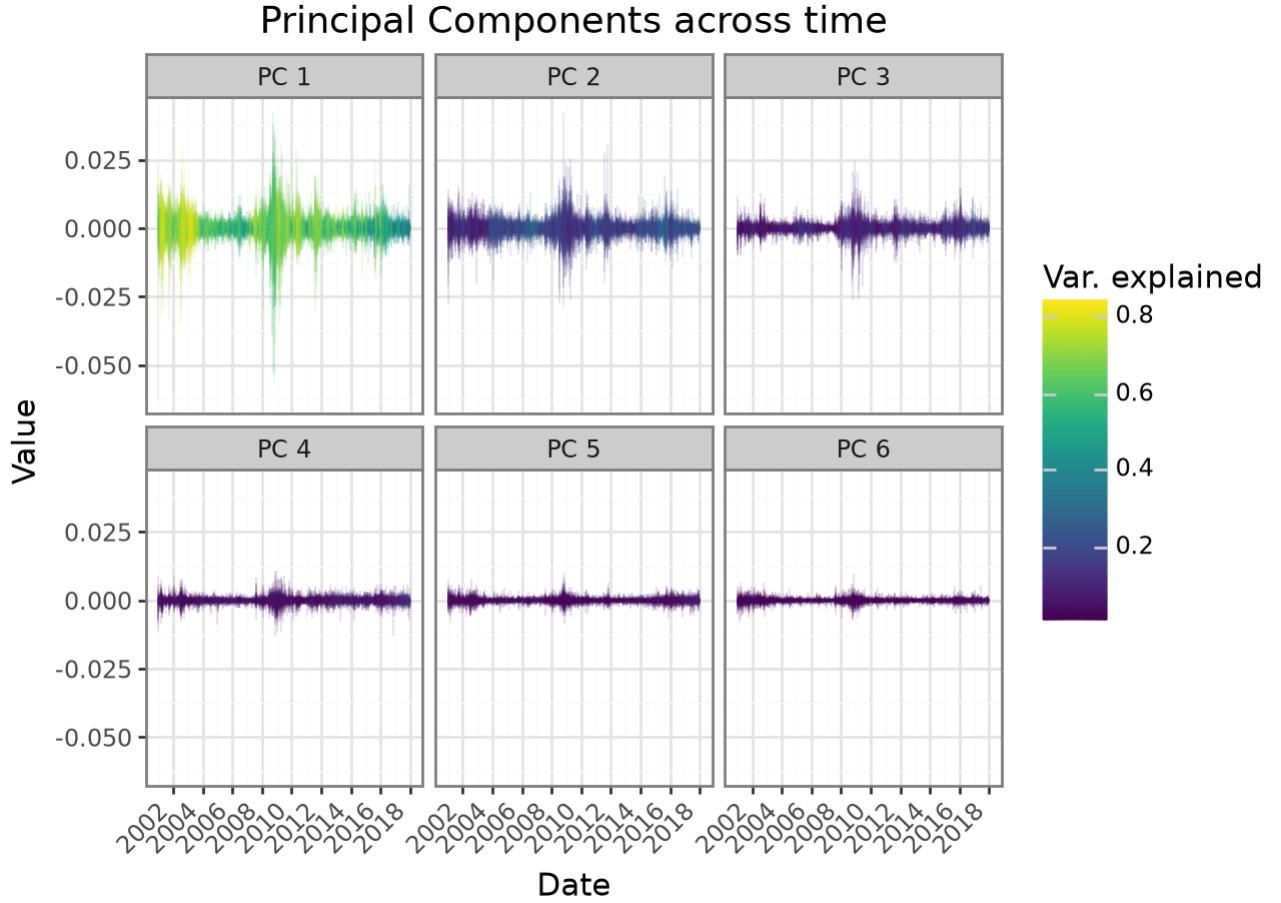
Factors' absolute loadings on PCs across time



PC: principal component, ordered by variance explained

Second, the principal components themselves, colored by the variance they explained at each block.

```
plot.principal_components(
    data_evals[1], data_pcs[1],
    blocks_size[1], list(cols_evalue.values()), list(cols_pc.values())
)
```



4.5 Appendix

4.5.1 Inference and spectral functions

The estimators rely on the inference for $\int_0^t \lambda(c_s) ds$. When eigenvalues can be repeated, this is non-differentiable, and the authors introduce a “trick” with spectral functions. This allows them to prove consistency and find bias-correction terms. The bias arises from the fact that eigenvalues are non-linear functions of the covariance matrix. In reality, our measure of such a matrix comes from noisy high-frequency data, and when this noise is applied to non-linear operations, it creates a bias.

In short, a spectral function $F(c_s)$ is simply a matrix function that is invariant to rotations. Associated with it is a symmetric function $(f \circ \lambda)(c_s)$ that takes the vector of eigenvalues. Studying the asymptotics of $\int_0^t F(c_s) ds$ allows the authors to prove consistency of their proposed estimators and add bias correction where needed. See the paper for more details; here, only the estimators are shown.

The authors make the four main assumptions below, although not all are needed for all results and their versions. They should be read in the context of the paper.

1. **Semimartingale Dynamics (Assumption 1):** The drift term b_t is progressively measurable and locally bounded, and the spot covariance matrix c_t is itself an Itô semimartingale. This standard assumption, along with limits on jump activity, ensures the underlying processes are mathematically well-behaved.
2. **Spectral Regularity for Consistency (Assumption 2):** The symmetric function f (where $F = f \circ \lambda$) is continuous and satisfies a polynomial growth condition $\|f(x)\| \leq K(1 + \|x\|^\zeta)$.

This guarantees that the target integral $\int_0^t F(c_s)ds$ is strictly well-defined, making the proposed estimators consistent.

3. **Non-Crossing Eigenvalues for Inference (Assumption 3):** To establish asymptotic distributions (like standard errors via the Central Limit Theorem), different groups of eigenvalues must not cross over within the time window $[0, t]$ (i.e., $c_s \in M^*(g_1, \dots, g_r)$). This prevents non-differentiable “kinks” from forming, ensuring the eigenvalue function remains sufficiently smooth (C^3) so the mean-value theorem can be applied.
4. **Weakened Non-Crossing (Assumption 4):** If we only care about a specific simple eigenvalue (e.g., isolating just the 1st Principal Component), Assumption 3 can be heavily relaxed. We only need to ensure that this specific target eigenvalue does not cross its immediate neighbor (e.g., $\lambda_1(c_s) > \lambda_2(c_s)$) during $[0, t]$. The crossover behavior of the remaining eigenvalues does not matter.

4.5.2 Integrated eigenvalues and eigenvectors

Integrated vector of eigenvalues $\int_0^t \lambda_s ds$: The estimator is a sum of the instantaneous eigenvalues, minus the bias correction term.

$$\hat{\lambda}_g = \underbrace{k_n \Delta_n \sum_i \hat{\lambda}_{g, ik_n \Delta_n}}_{\text{raw estimator}} - \underbrace{\Delta_n \text{Tr} \left(\left(\hat{\gamma}_{g, ik_n \Delta_n} - \hat{c}_{ik_n \Delta_n} \right)^+ \hat{c}_{ik_n \Delta_n} \right)}_{\text{bias correction}} \hat{\gamma}_{g, ik_n \Delta_n} \quad (17)$$

Integrated eigenvector $\int_0^t \gamma_{g,s} ds$: The estimator is a sum of the instantaneous eigenvectors, minus the bias correction term.

$$\hat{\gamma}_g = \underbrace{k_n \Delta_n \sum_{i=0}^{\lfloor t/(k_n \Delta_n) \rfloor} \hat{\gamma}_{i, k_n \Delta_n}}_{\text{raw estimator}} - \underbrace{\frac{1}{2k_n} \sum_{p \neq q} \frac{\hat{\lambda}_{g, ik_n \Delta_n} \hat{\lambda}_{p, ik_n \Delta_n} \hat{\lambda}_{g, ik_n \Delta_n}}{(\hat{\lambda}_{g, ik_n \Delta_n} - \hat{\lambda}_{p, ik_n \Delta_n})^2} \hat{\gamma}_{g, ik_n \Delta_n}}_{\text{bias correction}}$$

5 Realized betas and risk-adjusted returns

Now we need to get the realized betas from the stocks' returns with respect to the factors' principal components.

The following math and commentary are taken largely as-is from the paper. First, I present the theory, then the implementation in Python, and finally the results.

5.1 Methodology

The methodology is taken from Corradi et al. (2013).

5.1.1 Realized betas

Consider i as a stock's index, with $I = 2000$ total stocks; d as a factor's index, with $D = 6$ total factors; t as a day index, with $T = 4000$ total days; $\lambda \in (1, 5)$ as the number of minutes in a within-day block; m as the within-day block index, with $M_1 = 390$ or $M_5 = 78$ total blocks.

Suppose we have M equidistant observations within a day starting at time t , namely, $P_{i,t+\frac{\ell}{M}}$ and $F_{t+\frac{\ell}{M}}$ with $\ell = 0, \dots, M-1$. We define the vector of realized betas as:

$$\hat{\beta}_{i,t,M} = \left[\sum_{\ell=0}^{M-1} \left(F_{t+\frac{\ell+1}{M}} - F_{t+\frac{\ell}{M}} \right) \left(F_{t+\frac{\ell+1}{M}} - F_{t+\frac{\ell}{M}} \right)' \right]^{-1} \sum_{\ell=0}^{M-1} \left(F_{t+\frac{\ell+1}{M}} - F_{t+\frac{\ell}{M}} \right) \left(P_{i,t+\frac{\ell+1}{M}} - P_{i,t+\frac{\ell}{M}} \right) \quad (4)$$

If we define:

$$\Delta_{\ell} F_t := F_{t+\frac{\ell+1}{M}} - F_{t+\frac{\ell}{M}}$$

$$\Delta_{\ell} P_{i,t} := P_{i,t+\frac{\ell+1}{M}} - P_{i,t+\frac{\ell}{M}}$$

Equation (4) can be rewritten as:

$$\hat{\beta}_{i,t,M} = \left[\sum_{\ell=0}^{M-1} \Delta_{\ell} F_t \Delta_{\ell} F_t' \right]^{-1} \sum_{\ell=0}^{M-1} \Delta_{\ell} F_t \Delta_{\ell} P_{i,t} \quad (4')$$

5.1.2 Risk-adjusted returns

To get the daily returns and alphas, we must move to discrete time. For $t = 1, \dots, T$:

$$r_{i,t+1} = \int_t^{t+1} dP_i(s) = \mu_{i,t} + \Sigma'_{j,t} \int_t^{t+1} dW_F(s) + \int_t^{t+1} \sigma_i(s) dW_i(s) \quad (6)$$

$$f_{t+1} = \int_t^{t+1} dF(s) = \mu_{F,t} + \Sigma_{F,t} \int_t^{t+1} dW_F(s) \quad (7)$$

Where $\mu_{i,t}$ and $\mu_{F,t}$ are the drift parameters of the respective processes. Using the beta definition in (4), we get:

$$r_{i,t+1} = \alpha_{i,t} + f'_{t+1} \beta_{i,t} + \varepsilon_{i,t+1} + \eta_{i,t+1} \quad (8)$$

Where $\varepsilon_{i,t+1}$ and $\eta_{i,t+1}$ are error terms independent of the risk factor and state variables.

The alphas will be calculated based on the risk-adjusted returns, which are defined as:

$$\hat{Z}_{i,t+1,M} = r_{i,t+1} - f'_{t+1} \hat{\beta}_{i,t,M} \quad (Z)$$

5.1.3 The algorithm

5.1.3.1 Overview

The stocks and factor data were already processed into $\Delta_\ell F_t$ and $\Delta_\ell P_{i,t}$ formats.

The first step is, for each stock i and day t , to calculate the realized betas $\hat{\beta}_{i,t,M}$ via equation (4').

Then, aggregate the daily returns $r_{i,t+1} = \sum_{\ell=0}^{M-1} \Delta_\ell P_{i,t}$, and use equation (Z) to calculate the risk-adjusted returns.

All this must be done for the 1-minute and 5-minute frequencies.

5.1.3.2 Looping order

Consider i as a stock's index, with $I = 2000$ total stocks; d as a factor's index, with $D = 6$ total factors; t as a day index, with $T = 4000$ total days; $\lambda \in (1, 5)$ as the number of minutes in a within-day block; m as the within-day block index, with $M_1 = 390$ or $M_5 = 78$ total blocks.

There are many possible ways to organize the beta calculations, given that we have one result for each day, stock, factor, and frequency. Performance and readability (easily matching the code with the mathematical equations) are the main concerns.

The first concern is the memory constraint: data for all stocks and days cannot fit in memory at once. Thus, the first/outer loop must be over stocks or days. While looping over days is a possibility, it might not be as intuitive. Also, the `.parquet` data was ordered by `permno` exactly for this reason, to help query the data by stock.

The second concern is about regularizing sizes: we must explicitly loop by frequencies, as we cannot combine objects of different sizes (M_1 and M_5) into a single tensor. Happily, as there are only 2 frequencies, using a loop for it does not hurt performance much and helps with readability.

Finally, we can optimize the size of vectorized operations: if we loop by day, we perform many (T) loops of operations in blocks of D factors $\times M_1$ or M_5 observations. While if we loop by factor, we perform D loops of operations in blocks of T days $\times M_1$ or M_5 observations. The latter is more efficient, as it allows for larger vectorized operations that still fit in memory.

Each dimension can be operated over via explicit loops, via tensor dimensions, or via merging them into block-matrices. The latter is the least readable, while the first is the least efficient. Thus, the optimal structure is looping over i , then λ , then operating over the tensor (d, t, m) . Alternatively, if we were willing to sacrifice some readability, we could loop by blocks of stocks, λ , and operate with the tensor (i, d, t, m) on the GPU.

5.1.3.3 Pseudo-algorithm

The Pascal-like pseudocode below outlines the procedure.

```

procedure pca_high_freq(
  F_m1 : Matrix;           // Shape: (T*M1, d) | factor returns
  F_m5 : Matrix;           // Shape: (T*M5, d) | factor returns
  P_df : Matrix;           // Shape: (T*M1, I) | stock returns
  permnos_ordered : Vector // Shape: (I,) | list of permnos
);
  // Initialize containers for results:
  D = number of columns in F_m1;
  T = number of days in F_m1;
  betas_all = {"m1": [], "m5": []}
  adjusted_all = {"m1": [], "m5": []}

  // Stock loop:
  for each permno in permnos_ordered do
    begin
      P_m1 = query permno from P_df; // (T*M1,) + timestamps

      // Crop data to stock's window
      day_start = first day of P_m1 in F_m1;
      day_end   = last day of P_m1 in F_m1;
      T_s = number of days in (day_start, day_end);

      F_m1_s = crop F_m1 to (day_start, day_end); // (T_s*M1, D)
      F_m5_s = crop F_m5 to (day_start, day_end); // (T_s*M5, D)
      P_m1_s = crop P_m1 to (day_start, day_end), set missing to 0; // (T_s*M1,)

      // Days for dataframe storing:
      days = P_m1_idx[day_start to day_end];
      days_lead = P_m1_idx[(day_start + 1) to day_end];

      // Aggregate to different frequencies, get t+1 day returns:
      P_m5_s = aggregate P_m1_s into sum of 5-minute blocks; // (T_s*M5,)
      P_day_s = aggregate P_m5_s into sum of 1-day blocks, lead by 1; // (T_s - 1,)
      F_day_s = aggregate F_m5_s into sum of 1-day blocks, lead by 1; // (T_s - 1,)

      data_frequencies = {
        "m1": {"factors": F_m1_s, "returns": P_m1_s, "M": 390},

```

```

    "m5": {"factors": F_m5_s, "returns": P_m5_s, "M": 78}
}

// Frequency loop:
for freq in ["m1", "m5"] do
    begin
        // Setup:
        F = data_frequencies[freq]["factors"];
        P = data_frequencies[freq]["returns"];
        M = data_frequencies[freq]["M"];

        // Reshape into tensors:
        FF = rearrange(F, '(t m) d -> t m d', t = T_s, m = M); // (T_s, M, D)
        PP = rearrange(P, '(t m) -> t m 1', t = T_s, m = M); // (T_s, M, 1)

        // Compute cov. and var. across each day and factor
        cov = reduce(FF * PP, 't m d, t m 1 -> t d', 'sum'); // (T_s, D)
        var = reduce(FF * FF, 't m d -> t d', 'sum'); // (T_s, D)

        // Compute betas, with var = 0 -> beta = 0:
        betas = divide(cov, var, fill = 0); // (T_s, D)
        betas_lag = betas[0 to (T_s - 2), :]; // (T_s - 1, D)

        // Get the adjusted daily returns:
        P_day_hat = reduce(F_day_s * betas_lag, 't d -> t', 'sum'); // (T_s - 1,)
        P_day_adj = P_day_s - P_day_hat; // (T_s - 1,)

        // Store results:
        betas_all[freq] append (permno, days, betas_lag);
        adjusted_all[freq] append (permno, days_lead, P_day_adj);
    end;

end;

// Finalization:
concatenate each of betas_all into (permno, day, beta1, ..., beta 6) dataframe;
concatenate each of adjusted_all into (permno, day, adjusted_return) dataframe;
end.

```

5.2 Code implementation

The problem is embarrassingly parallel at the stock level. At the same time, the large tensor operations could be done on the GPU. Below, I present first the sequential implementation, which presents the algorithm in a more readable way. In the appendix, I present both the GPU (via CuPy) and multiprocessing implementations.

CuPy is designed to be equivalent to NumPy, such that we do not need to change almost anything. However, the tensor operations are not that large, and the overhead is not worth it. Multiprocessing adds significant setup cost (initializing workers, creating shared memory objects), and modifies heavily the main orchestrator. On the other hand, it greatly improves performance.

And core modules:

```

import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import polars as pl
import polars.selectors as cs
import numpy as np
from einops import reduce, rearrange

from tqdm import tqdm

from typing import TypedDict, Literal
from numpy.typing import NDArray

import src.utils as ut
import src.graphs as plot

ut.set_seed()

```

```

import importlib
importlib.reload(plot)

```

```
<module 'src.graphs' from 'c:\\Users\\ricar\\OneDrive\\A-Trabalho\\Freelance\\ra-distaso2026\\'
```

Let us break the algorithm into pieces, starting with the results for a given frequency (inner iteration):

```

def freq_results(
    F: NDArray, P: NDArray, # (T_s * M, D), (T_s * M,)
    M: int,
    F_day_s: NDArray, P_day_s: NDArray, # (T_s - 1, D), (T_s - 1,)
    T_s: np.int64
) -> tuple[NDArray, NDArray]: # (T_s, D), (T_s - 1,)
    # Reshape into tensors:
    FF = rearrange(F, '(t m) d -> t m d', t = T_s, m = M) # (T_s, M, D)
    PP = rearrange(P, '(t m) -> t m 1', t = T_s, m = M) # (T_s, M, 1)

    # Compute cov. and var. across each day and factor:
    cov = reduce(FF * PP, 't m d -> t d', 'sum') # (T_s, D)
    var = reduce(FF * FF, 't m d -> t d', 'sum') # (T_s, D)

    # Compute betas, with var = 0 -> beta = 0:
    betas = np.divide(
        cov, var,
        out = np.zeros_like(cov), where = var != 0
    ) # (T_s, D)
    betas_lag = betas[:-1, :] # (T_s - 1, D)

    # Get the adjusted daily returns:
    P_day_hat = reduce(F_day_s * betas_lag, 't d -> t', 'sum') # (T_s - 1,)
    P_day_adj = P_day_s - P_day_hat # (T_s - 1,)

    return betas, P_day_adj

```

Then, the outer loop:

```
class ResultsStock(TypedDict):
    permno: int
    day_range: tuple[np.int64, np.int64]
    m1: tuple[NDArray, NDArray] # (T_s, D), (T_s - 1,)
    m5: tuple[NDArray, NDArray] # (T_s, D), (T_s - 1,)

def stock_results(
    permno: int, P_df: pl.LazyFrame,
    F_m1: NDArray, F_m5: NDArray, # (T * M1, D), (T * M5, D)
    F_m1_idx: NDArray, # (T * M1,)
) -> ResultsStock:
    # Query permno from P_df (if query empty, should err):
    P_query = (
        P_df.filter(pl.col("permno") == permno)
        .select(["ts_min_ny", "log_return"])
        .collect()
    )

    P_m1 = P_query["log_return"].to_numpy() # (T_s * M1,)
    P_m1_idx = P_query["ts_min_ny"].to_numpy() # (T_s * M1,)

    # Crop data to stock's window:
    idx_start: np.int64 = np.searchsorted(F_m1_idx, P_m1_idx[0])
    idx_end: np.int64 = np.searchsorted(F_m1_idx, P_m1_idx[-1], side = "right")
    day_start = idx_start // 390 # First minute of first day (left-closed)
    day_end = np.int64(np.ceil(idx_end / 390)) # First minute of last day + 1 (right-open)
    T_s = day_end - day_start

    F_m1_s = F_m1[day_start * 390 : day_end * 390, :] # (T_s * M1, D)
    F_m5_s = F_m5[day_start * 78 : day_end * 78, :] # (T_s * M5, D)

    # Crop P_m1 to (day_start, day_end), set missing to 0:
    F_m1_s_idx = F_m1_idx[day_start * 390 : day_end * 390]

    fs_in_p = np.clip(np.searchsorted(F_m1_s_idx, P_m1_idx), 0, len(F_m1_s_idx) - 1)
    fs_in_p_exact = F_m1_s_idx[fs_in_p] == P_m1_idx
    # To solve out-of-bounds indexing (TODO: happens for permno 22859)

    P_m1_s = np.zeros(T_s * 390, dtype = np.float64) # (T_s * M1,)
    P_m1_s[fs_in_p[fs_in_p_exact]] = P_m1[fs_in_p_exact]

    # Aggregate to different frequencies, get t+1 day returns:
    P_m5_s = reduce(P_m1_s, '(t m) -> t', 'sum', m = 5) # (T_s * M5,)
    P_day_s = reduce(P_m5_s, '(t m) -> t', 'sum', m = 78)[1:] # (T_s - 1,)
    F_day_s = reduce(F_m5_s, '(t m) d -> t d', 'sum', m = 78)[1:, :] # (T_s - 1, D)

    # Frequency loop:
    results_permno: ResultsStock = {
        "permno": permno,
        "day_range": (day_start, day_end),
```

```

    "m1": freq_results(F_m1_s, P_m1_s, 390, F_day_s, P_day_s, T_s),
    "m5": freq_results(F_m5_s, P_m5_s, 78, F_day_s, P_day_s, T_s)
}

```

```

return results_permno

```

Note: it would be better to save 390, 78 as constants

Finally, the main orchestrator:

```

class ResultsRealized(TypedDict):
    permno: list[int]
    day_range: list[tuple[np.int64, np.int64]]
    m1: list[tuple[NDArray, NDArray]] # (T_s, D), (T_s - 1,)
    m5: list[tuple[NDArray, NDArray]] # (T_s, D), (T_s - 1,)

def betas_returns_realized(
    permnos_ordered: NDArray, P_df: pl.LazyFrame, # (I,)
    F_m1: NDArray, F_m5: NDArray, # (T * M1, D), (T * M5, D)
    F_m1_idx: NDArray, # (T * M1,)
    quiet: bool = False,
) -> ResultsRealized:
    I = len(permnos_ordered)

    F_m1 = np.ascontiguousarray(F_m1)
    F_m5 = np.ascontiguousarray(F_m5)
    F_m1_idx = np.ascontiguousarray(F_m1_idx)

    if not quiet: print(f"Starting stock loop for {I} stocks...")
    results: ResultsRealized = {"permno": [], "day_range": [], "m1": [], "m5": []}

    for permno in tqdm(permnos_ordered, disable = quiet, total = I):
        results_stock = stock_results(permno, P_df, F_m1, F_m5, F_m1_idx)
        results["permno"].append(results_stock["permno"])
        results["day_range"].append(results_stock["day_range"])
        results["m1"].append(results_stock["m1"])
        results["m5"].append(results_stock["m5"])

    return results

```

Some performance considerations:

- `F_m1` and `F_m5` are turned into contiguous arrays. `P_m1` and `P_m5` are already created contiguously.
- `freq_results()` could be JIT-compiled with `numba`, but it would not work with the `einops` package. The performance gain is not significant.

5.3 Calculating the betas and risk-adjusted returns

Loading data:

```

data_pc_1 = pl.read_csv("data/pca_1min.csv")
data_pc_5 = pl.read_csv("data/pca_5min.csv")

F_m1, F_m5 = (
    x.select(cs.starts_with("pc_")).to_numpy() for x in [data_pc_1, data_pc_5]
)
F_m1_idx, F_m5_idx = (
    x["datetime"]
    .str.strptime(pl.Datetime, "%Y-%m-%d %H:%M")
    .to_numpy().astype("datetime64[m]").astype(np.uint32)
    for x in [data_pc_1, data_pc_5]
)
D = F_m1.shape[1]

permnos_ordered = pl.read_csv("data/stocks_raw/counts_permno.csv")["permno"].to_numpy()

P_filepath = "data/stocks_1min_returns.parquet"
P_df = pl.scan_parquet(P_filepath, parallel = "row_groups")

```

As stated before, the code is actually run with multiprocessing. The logic is the same, and Section 5.5.1 shows the code for the multiprocessing infrastructure. Below, I import it and run the results.

```

from src import betas_mp

if 1: # Run only once
    results = betas_mp.betas_returns_realized_mp(
        permnos_ordered = permnos_ordered,
        P_filepath = P_filepath,
        F_m1 = F_m1, F_m5 = F_m5,
        F_m1_idx = F_m1_idx,
        max_workers = 6, chunksize = 20
    )

```

Finally, we save the results into parquet files. Inside `save_results()`, some basic checks are run:

- All dataframes are indexed by `permno` and `ts_day_ny`, in the exact same way, except for the missing first day in the adjusted returns.
- The index is sorted by `permno` and `ts_day_ny`, without duplicate pairs.
- There are no NaNs, Infs, or nulls in the dataframes.

```

if False: # Run only once
    ut.save_results(results, F_m1_idx, D)

```

5.3.1 Checking results

Here I present a simpler version of the code for a single stock, frequency, and day, to check whether the results are correct. It is based on matching the day stamps themselves from P and F.

```

def single_result(permno: int, freq: str) -> tuple[NDArray, NDArray]:
    P_query = (
        P_df.filter(pl.col("permno") == permno)
        .select(["ts_min_ny", "log_return"])
        .collect()
    )

    P_m1 = P_query["log_return"].to_numpy() # (T_s * M1,)
    P_m1_idx = P_query["ts_min_ny"].to_numpy() # (T_s * M1,)

    days_p = P_m1_idx // 1440
    day_min, day_max = days_p.min(), days_p.max()

    days_f = F_m1_idx // 1440
    days_f_s = np.unique(days_f[(day_min <= days_f) & (days_f <= day_max)])
    T_s = np.int64(len(days_f_s))

    F_m1_s = F_m1[np.isin(days_f, days_f_s), :]
    F_m1_s_idx = F_m1_idx[np.isin(days_f, days_f_s)]
    F_m5_s = F_m5[np.isin(F_m5_idx // 1440, days_f_s), :]

    mask_fs_in_p = np.isin(F_m1_s_idx, P_m1_idx)
    mask_p_in_fs = np.isin(P_m1_idx, F_m1_s_idx)
    P_m1_s = np.zeros(T_s * 390, dtype = np.float64)
    P_m1_s[mask_fs_in_p] = P_m1[mask_p_in_fs]

    P_m5_s = reduce(P_m1_s, '(t m) -> t', 'sum', m = 5) # (T_s * M5,)
    P_day_s = reduce(P_m5_s, '(t m) -> t', 'sum', m = 78)[1:] # (T_s - 1,)
    F_day_s = reduce(F_m5_s, '(t m) d -> t d', 'sum', m = 78)[1:, :] # (T_s - 1, D)

    if freq == "m1":
        result_freq = freq_results(F_m1_s, P_m1_s, 390, F_day_s, P_day_s, T_s)
    elif freq == "m5":
        result_freq = freq_results(F_m5_s, P_m5_s, 78, F_day_s, P_day_s, T_s)

    return result_freq

```

Then, for some random stock-frequency pairs, we check whether the results are the same, up to some numerical precision. It passes, and the results are correct.

```

def check_results(
    permno: int, freq: Literal["m1", "m5"], threshold: float = 1e-13
) -> tuple[float, float]:
    res = stock_results(permno, P_df, F_m1, F_m5, F_m1_idx)
    betas_code, adjusted_code = res[freq]

    betas_single, adjusted_single = single_result(permno, freq)

    means = (
        np.mean(np.abs(betas_code - betas_single) <= threshold),
        np.mean(np.abs(adjusted_code - adjusted_single) <= threshold)
    )

```

```

    return means

n_tests = 10
permnos = np.random.choice(permnos_ordered, size = n_tests, replace = False)
freqs = np.random.choice(["m1", "m5"], size = n_tests, replace = True)

if not all(
    check_results(permno, freq) == (1, 1)
    for permno, freq in zip(permnos, freqs)
):
    raise ValueError("`stock_results()` is not in accordance with `single_result()`.")

```

5.4 Visualizing results

Let us visualize the results, as a first look, and to check their reasonableness. The adjusted returns are interesting to compare with the base returns in [src/data_stocks.ipynb](#)/Chapter 3.

5.4.1 The datasets

First, let us select some random stocks to visualize, and print the datasets.

```

vis_permnos = (
    pl.read_csv("data/stocks_raw/counts_permno.csv")
    ["permno"].sample(4).to_numpy()
)

```

```
pl.scan_parquet("data/betas_1min.parquet").head(5).collect()
```

permno u32	ts_day_ny u32	beta_1 f64	beta_2 f64	beta_3 f64	beta_4 f64	beta_5 f64	beta_6 f64
10032	12146	-0.085846	0.561377	-0.547257	0.340261	-0.116859	0.635344
10032	12149	-0.063692	1.264317	0.103729	1.079935	-1.382808	2.39721
10032	12150	-0.175836	0.784855	0.561158	-0.174367	0.617653	0.587243
10032	12151	-0.051801	1.151908	-1.296084	-0.856966	0.615159	0.116852
10032	12152	0.007006	0.488782	-0.812786	-0.833815	-0.803676	0.326376

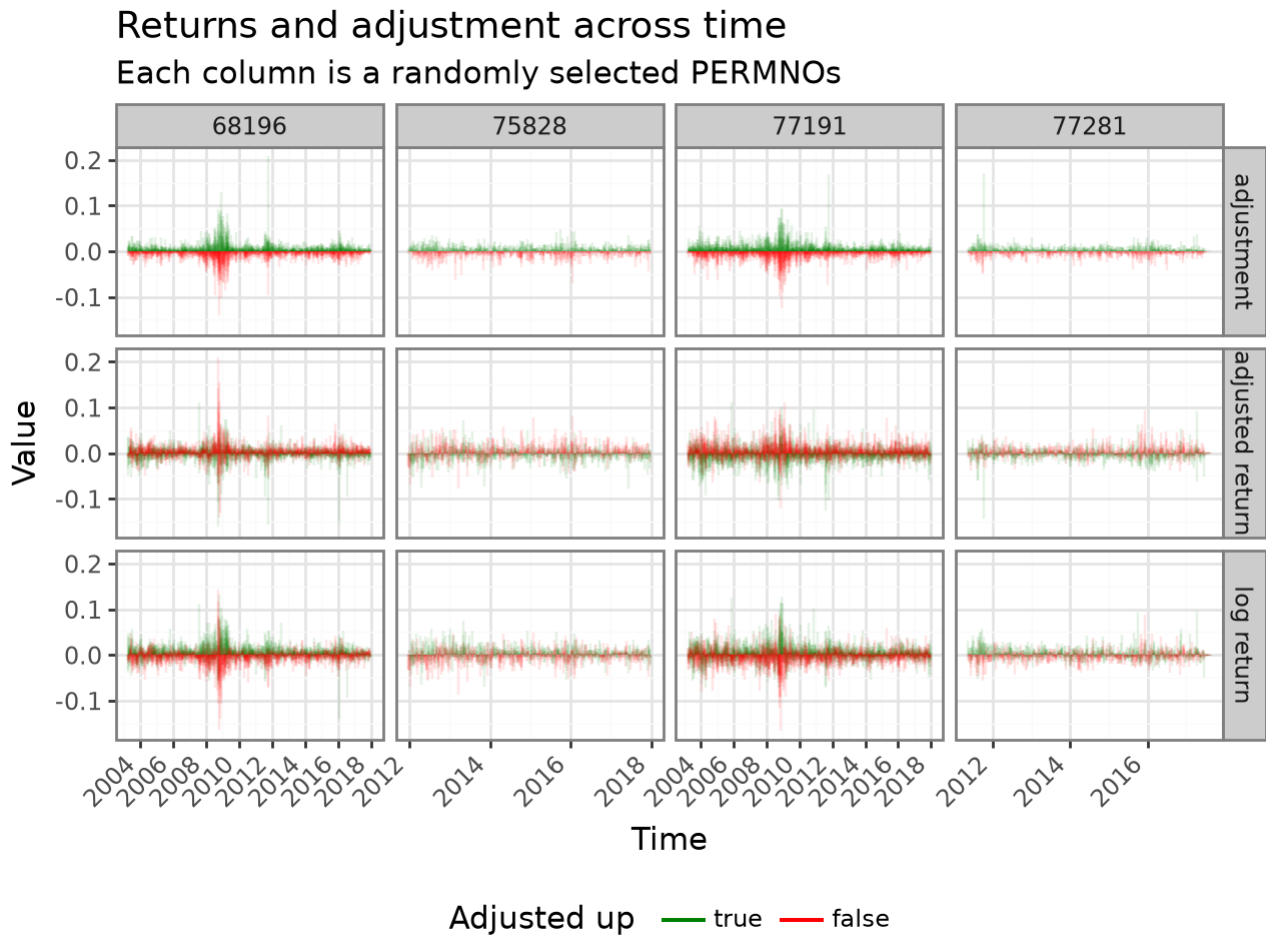
```
pl.scan_parquet("data/stocks_1min_adjusted.parquet").head(5).collect()
```

permno u32	ts_day_ny u32	adjusted_return f64
10032	12149	-0.006831
10032	12150	-0.00011
10032	12151	0.000149
10032	12152	-0.008418
10032	12153	-0.02418

5.4.2 Adjusted returns

For some random PERMNOs, we plot the log return, the adjusted return, and the difference (the adjustment). All colored by the sign of the adjustment:

```
plot.adjusted_returns(vis_permnos)
```

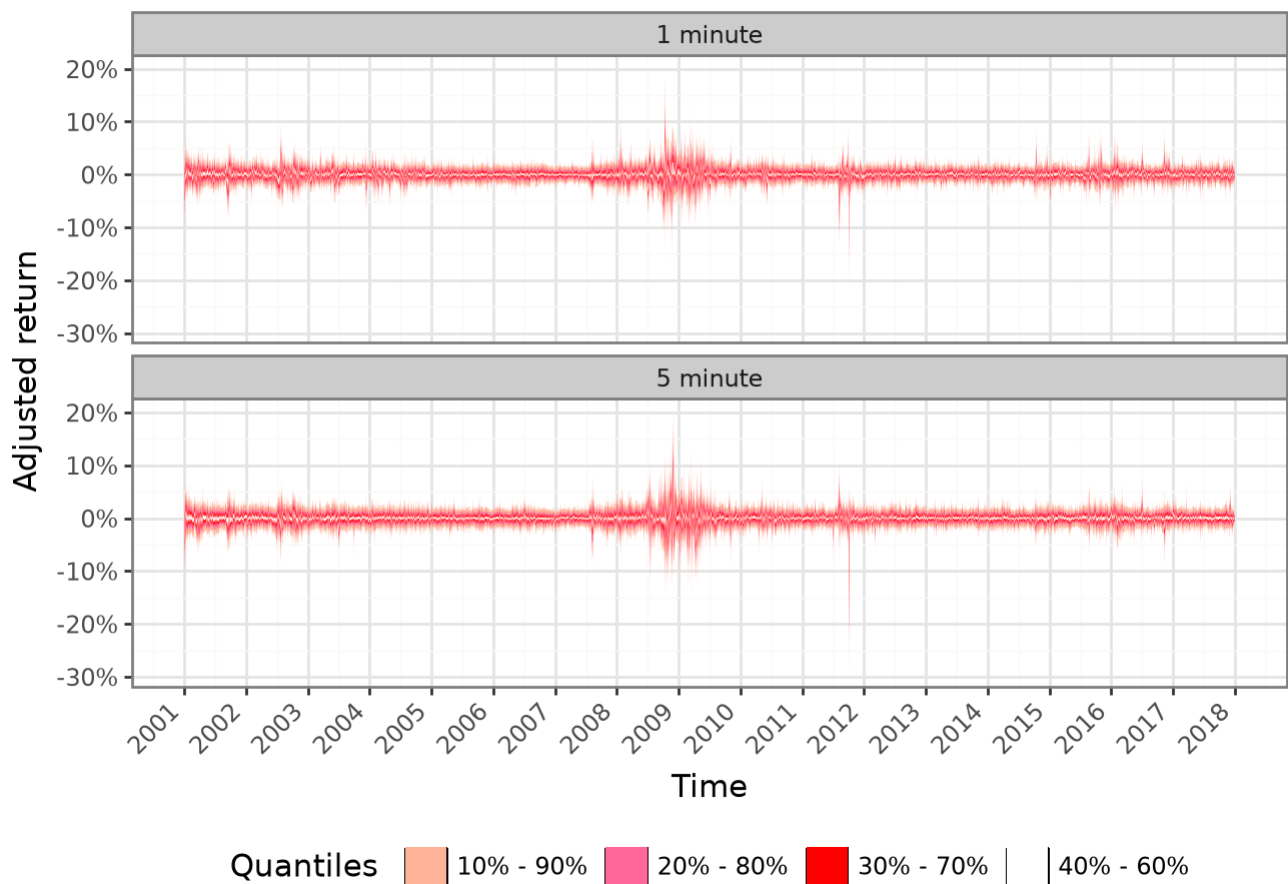


To get a more general view, we can see the river plot of the adjusted returns for all stocks. Now we also plot the 5-minute exercise.

```
plot.adjusted_returns_river()
```

Adjusted returns' distribution across time

For each frequency aggregation



Let us understand the difference between the two frequencies.

```
plot.frequency_comparison()
```

statistic	adjusted return 1min	adjusted return 5min	adjusted return diff
str	f64	f64	f64
"mean"	-0.000193	-0.00044	0.010786
"std"	0.02516	0.027331	0.017108
"min"	-6.334161	-6.344942	0.0
"25%"	-0.009201	-0.009851	0.002036
"50%"	0.0	0.0	0.005835
"75%"	0.009052	0.009349	0.013088
"max"	1.623521	5.629308	6.351686

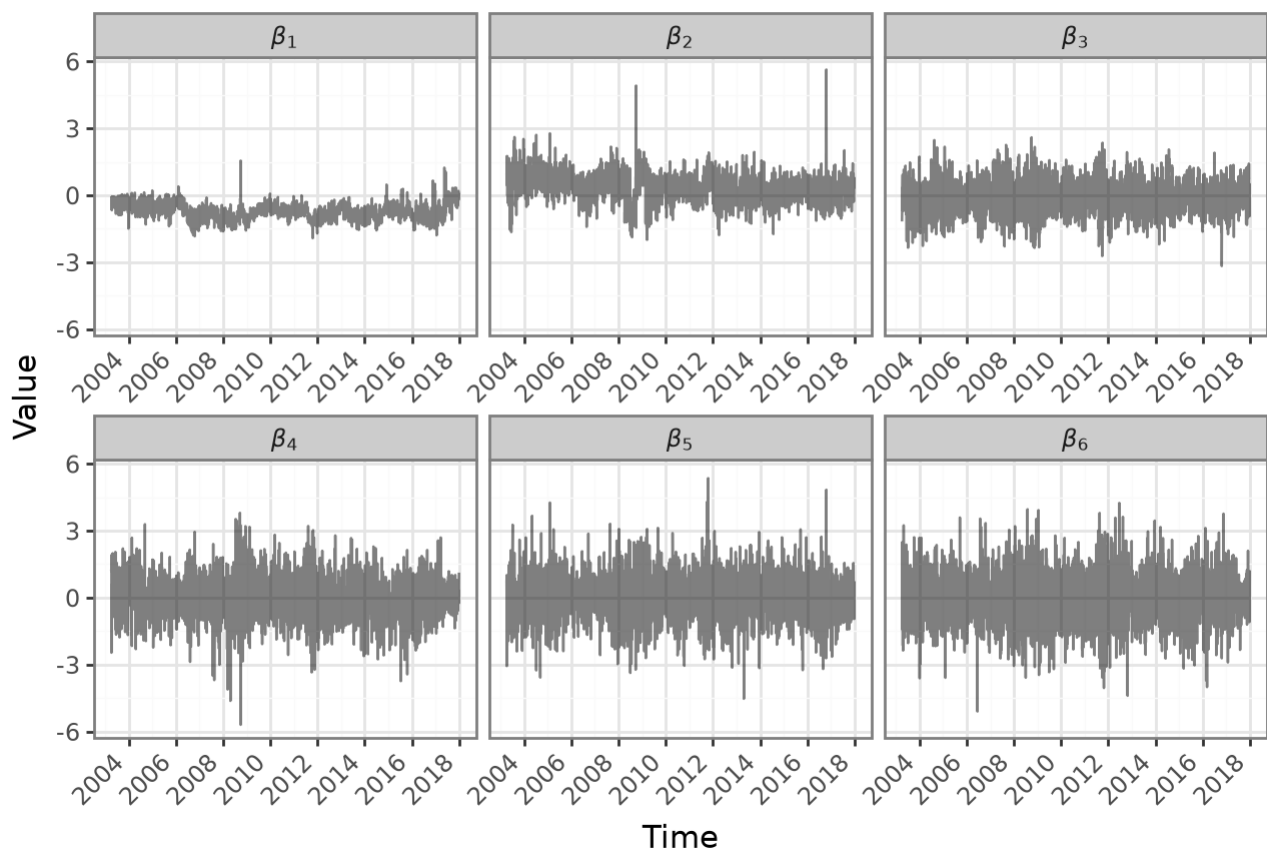
5.4.3 Betas

Now we do a similar exercise for all the betas. First for a single stock:

```
plot.betas(vis_permnos[0])
```

Realized betas across time

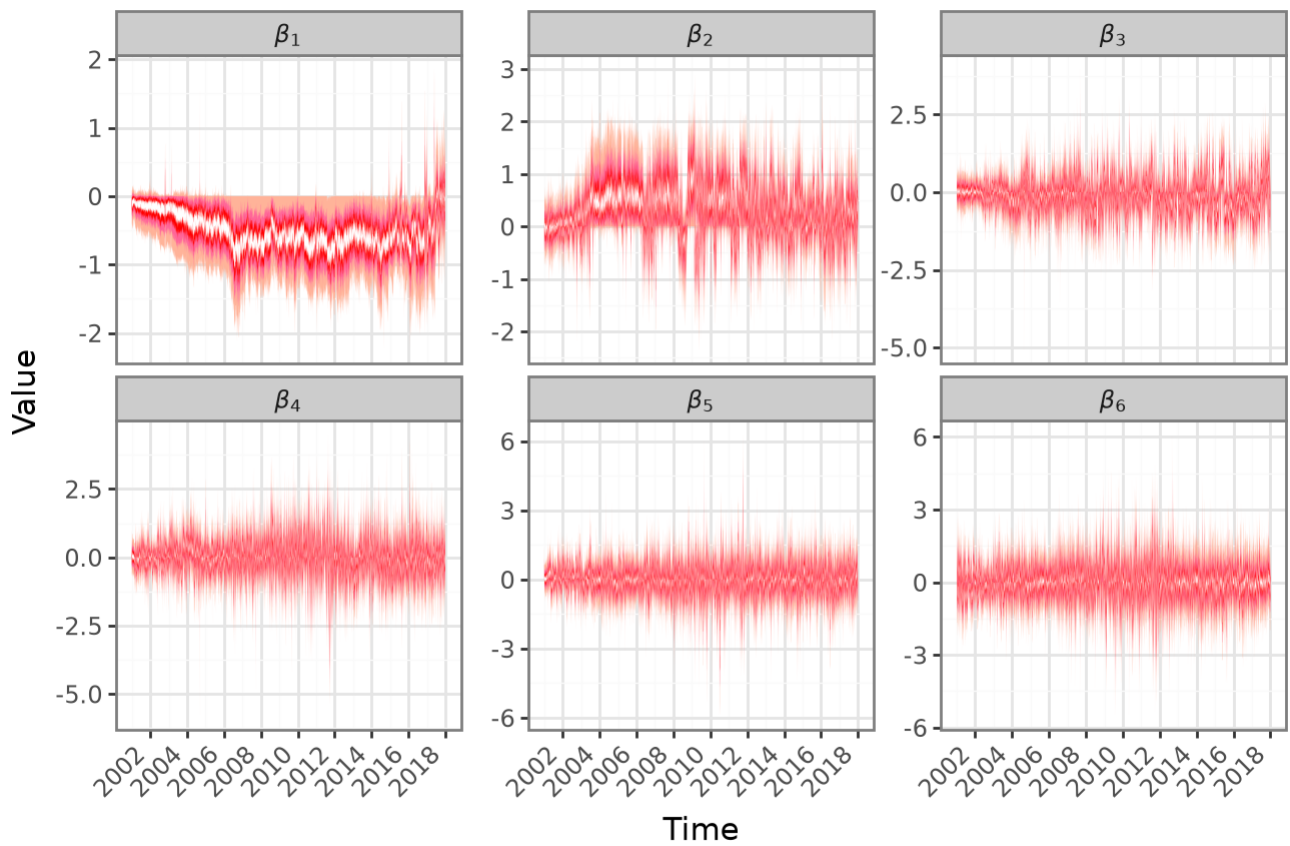
For PERMNO 68196



Then for all stocks, with river plots:

```
plot.betas_river()
```

Realized betas' distribution across time

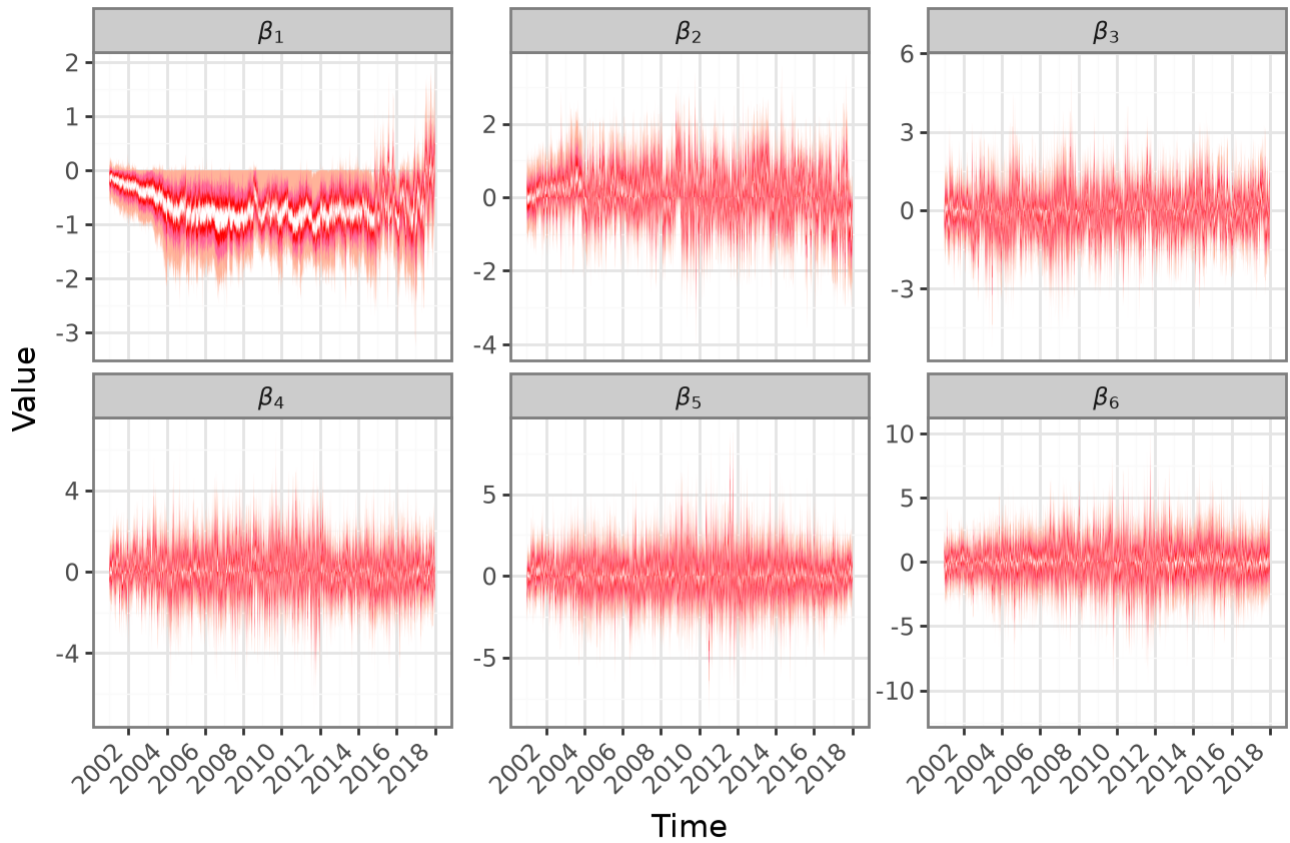


Quantiles 10% - 90% 20% - 80% 30% - 70% 40% - 60%

And again for the 5-minute exercise:

```
plot.betas_river("data/betas_5min.parquet")
```

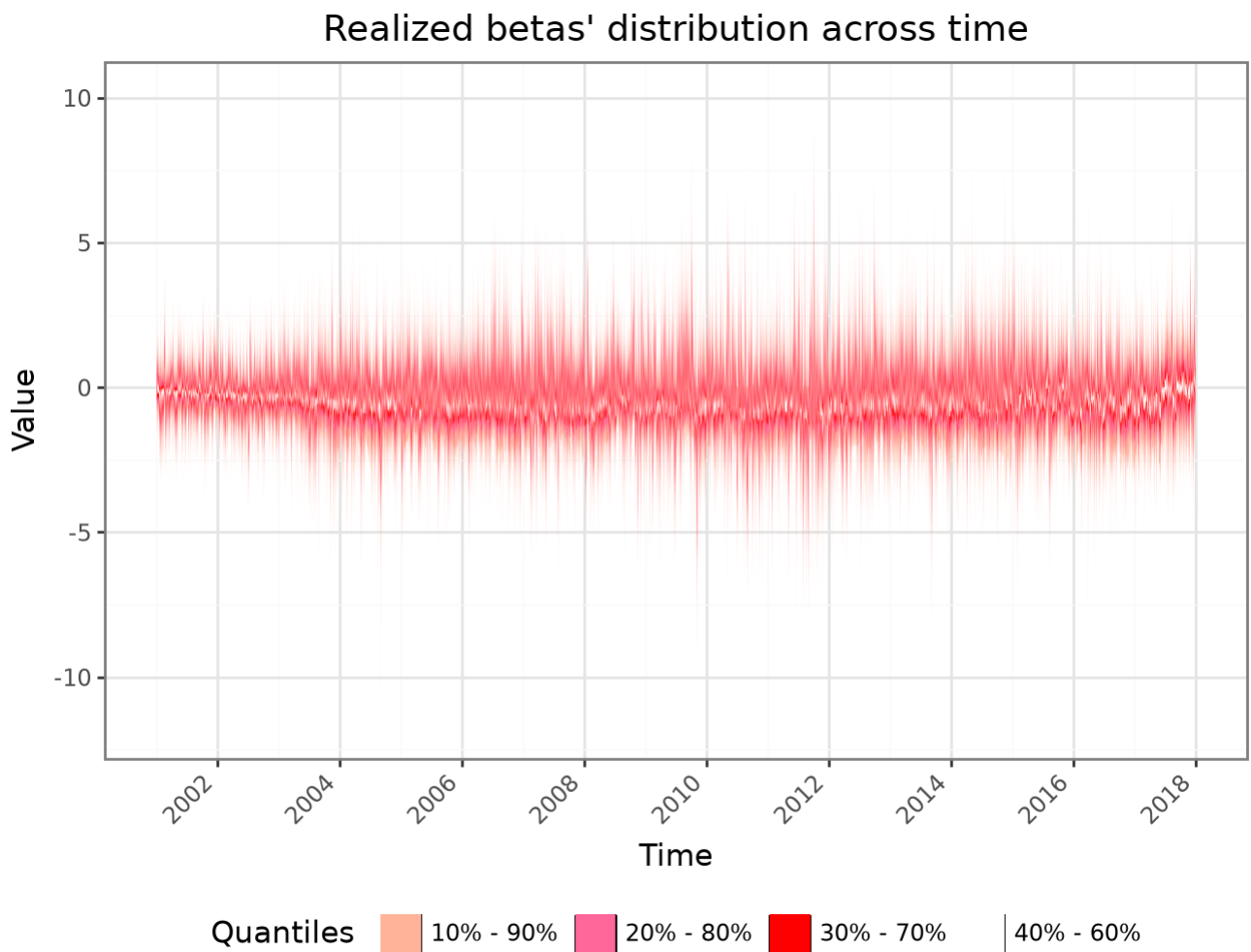
Realized betas' distribution across time



Quantiles 10% - 90% 20% - 80% 30% - 70% 40% - 60%

Finally, for all betas together:

```
plot.betas_river("data/betas_5min.parquet", facet_betas = False)
```



We can also plot some statistics:

```
(
  pl.scan_parquet("data/betas_5min.parquet")
    .select(pl.col("^beta_.$"))
    .rename(lambda x: x.replace("_", " "))
    .describe()
    [2:]
)
```

statistic	beta 1	beta 2	beta 3	beta 4	beta 5	beta 6
str	f64	f64	f64	f64	f64	f64
"mean"	-0.754928	0.085405	-0.045188	0.011261	0.006265	-0.023852
"std"	0.805245	1.184127	1.529726	2.055262	2.479256	2.984621
"min"	-141.681678	-222.804905	-416.011271	-444.871	-473.897769	-953.580566
"25%"	-1.15786	-0.446359	-0.723584	-0.875457	-1.039951	-1.251895
"50%"	-0.694373	0.004781	0.0	0.0	0.0	0.0
"75%"	-0.242462	0.624554	0.614898	0.891176	1.048369	1.207348
"max"	40.80044	71.083478	281.725165	795.09603	289.645411	559.232162

Let us understand the difference between the two frequencies. One can look at the individual betas with:

```
plot.frequency_comparison("data/betas_1min.parquet", "data/betas_5min.parquet", "beta_1")
```

But below I use the average across all betas:

```
plot.frequency_comparison_all()
```

statistic	beta 1min	beta 5min	beta diff
str	f64	f64	f64
"mean"	-0.06679	-0.120173	0.564046
"std"	0.46288	0.819444	0.699123
"min"	-92.899769	-340.730429	0.0
"25%"	-0.266689	-0.457014	0.137261
"50%"	-0.027907	-0.057572	0.371056
"75%"	0.134262	0.213699	0.754502
"max"	67.793202	179.490856	263.966128

5.5 Appendix

Here I present the multiprocessing and GPU implementations.

5.5.1 Multiprocessing code

Setup and helpers to create shared memory arrays and initialize the worker processes:

```
from multiprocessing import shared_memory
from multiprocessing.managers import SharedMemoryManager
from concurrent.futures import ProcessPoolExecutor
from threadpoolctl import ThreadpoolController

class SharedArrayMeta(TypedDict):
    name: str
    shape: tuple[int, ...]
    dtype: np.dtype

def create_shared_array(
    arr: NDArray, smm: SharedMemoryManager
) -> tuple[SharedArrayMeta, shared_memory.SharedMemory]:
    arr_c = np.ascontiguousarray(arr)
    shm = smm.SharedMemory(size = arr_c.nbytes)
    shared_arr = np.ndarray(arr_c.shape, dtype = arr_c.dtype, buffer = shm.buf)
    shared_arr[:] = arr_c[:]

    meta: SharedArrayMeta = {"name": shm.name, "shape": arr_c.shape, "dtype": arr_c.dtype}
    return meta, shm

WORKER_OBJS: dict[str, NDArray]
WORKER_SHMS: dict[str, shared_memory.SharedMemory]
```

```

def worker_init(shared metas: dict[str, SharedArrayMeta]) -> None:
    # Permanently limit thread pools on the worker's process:
    _thread_controller = ThreadpoolController()
    _thread_controller.limit(limits = 1).__enter__() # Consider user_api = "blas"

    # Attach shared memory objects:
    global WORKER_OBJS, WORKER_SHMS
    WORKER_OBJS = {}
    WORKER_SHMS = {}

    for name, meta in shared_metas.items():
        shm = shared_memory.SharedMemory(name = meta["name"])
        WORKER_SHMS[name] = shm
        WORKER_OBJS[name] = np.ndarray(
            meta["shape"], dtype = meta["dtype"], buffer = shm.buf
        )

```

Then, the outer loop:

```

def stock_results_mp(permno: int, P_filepath: str) -> ResultsStock:
    # Fetch global shared matrices directly from dictionary
    F_m1, F_m5, F_m1_idx = (WORKER_OBJS[x] for x in ["F_m1", "F_m5", "F_m1_idx"])

    # Query permno from P_df (if query empty, should error):
    P_query = (
        pl.scan_parquet(P_filepath)
        .filter(pl.col("permno") == permno)
        .select(["ts_min_ny", "log_return"])
        .collect()
    )

    P_m1 = P_query["log_return"].to_numpy() # (T_s * M1,)
    P_m1_idx = P_query["ts_min_ny"].to_numpy() # (T_s * M1,)

    # Crop data to stock's window:
    idx_start: np.int64 = np.searchsorted(F_m1_idx, P_m1_idx[0])
    idx_end: np.int64 = np.searchsorted(F_m1_idx, P_m1_idx[-1], side = "right")
    day_start = idx_start // 390 # First minute of first day (left-closed)
    day_end = np.int64(np.ceil(idx_end / 390)) # First minute of last day + 1 (right-open)
    T_s = day_end - day_start

    F_m1_s = F_m1[day_start * 390 : day_end * 390, :] # (T_s * M1, D)
    F_m5_s = F_m5[day_start * 78 : day_end * 78, :] # (T_s * M5, D)

    # Crop P_m1 to (day_start, day_end), set missing to 0:
    F_m1_s_idx = F_m1_idx[day_start * 390 : day_end * 390]

    fs_in_p = np.clip(np.searchsorted(F_m1_s_idx, P_m1_idx), 0, len(F_m1_s_idx) - 1)
    fs_in_p_exact = F_m1_s_idx[fs_in_p] == P_m1_idx
    # To solve out-of-bounds indexing (TODO: happens for permno 22859)

    P_m1_s = np.zeros(T_s * 390, dtype = np.float64) # (T_s * M1,)

```

```

P_m1_s[fs_in_p[fs_in_p_exact]] = P_m1[fs_in_p_exact]

# Aggregate to different frequencies, get t+1 day returns:
P_m5_s = reduce(P_m1_s, '(t m) -> t', 'sum', m = 5) # (T_s * M5,)
P_day_s = reduce(P_m5_s, '(t m) -> t', 'sum', m = 78)[1:] # (T_s - 1,)
F_day_s = reduce(F_m5_s, '(t m) d -> t d', 'sum', m = 78)[1:, :] # (T_s - 1, D)

# Frequency loop:
results_permno: ResultsStock = {
    "permno": permno,
    "day_range": (day_start, day_end),
    "m1": freq_results(F_m1_s, P_m1_s, 390, F_day_s, P_day_s, T_s),
    "m5": freq_results(F_m5_s, P_m5_s, 78, F_day_s, P_day_s, T_s)
}

return results_permno

```

Finally, the main orchestrator:

```

def betas_returns_realized_mp(
    permnos_ordered: NDArray, P_filepath: str, # (I,)
    F_m1: NDArray, F_m5: NDArray, # (T * M1, D), (T * M5, D)
    F_m1_idx: NDArray, # (T * M1,)
    quiet: bool = False,
    max_workers: int | None = None, chunksize: int = 1
) -> ResultsRealized:
    I = len(permnos_ordered)
    results: ResultsRealized = {"permno": [], "day_range": [], "m1": [], "m5": []}

    with SharedMemoryManager() as smm:
        metas: dict[str, SharedArrayMeta] = {}
        shms: dict[str, shared_memory.SharedMemory] = {}
        for key, arr in [("F_m1", F_m1), ("F_m5", F_m5), ("F_m1_idx", F_m1_idx)]:
            meta, shm = create_shared_array(arr, smm)
            metas[key] = meta
            shms[key] = shm

        with ProcessPoolExecutor(
            initializer = worker_init, initargs = (metas,),
            max_workers = max_workers
        ) as executor:

            map_iterator = executor.map(
                stock_results_mp,
                permnos_ordered, [P_filepath] * I,
                chunksize = chunksize
            )

            for results_stock in tqdm(map_iterator, disable = quiet, total = I):
                results["permno"].append(results_stock["permno"])
                results["day_range"].append(results_stock["day_range"])
                results["m1"].append(results_stock["m1"])

```

```

        results["m5"].append(results_stock["m5"])

    return results

```

As these functions need to be exported into the worker processes, they must be defined in a `.py` script, `src/betas_mp.py`. The notebook will import them from there.

5.5.2 GPU code

With the GPU, we simply need to import the `cupy` package, set it up (make sure we have a tensor toolkit like CUDA available), and replace it as the engine within the `freq_results()` function.

At the moment I have not implemented the safe division for the GPU, but that is not the most critical aspect.

The GPU code is not expected to be much faster, since we are working with relatively small matrices.

```

import cupy as cp

if not cp.is_available():
    raise RuntimeError("Please properly set up CuPy to use GPU acceleration.")

def freq_results_gpu(
    F: NDArray, P: NDArray, # (T_s * M, D), (T_s * M,)
    M: int,
    F_day_s: NDArray, P_day_s: NDArray, # (T_s - 1, D), (T_s - 1,)
    T_s: int
) -> tuple[NDArray, NDArray]: # (T_s, D), (T_s - 1,)
    F = cp.asarray(F)
    P = cp.asarray(P)
    F_day_s = cp.asarray(F_day_s)
    P_day_s = cp.asarray(P_day_s)

    # Reshape into tensors:
    FF = rearrange(F, '(t m) d -> t m d', t = T_s, m = M) # (T_s, M, D)
    PP = rearrange(P, '(t m) -> t m 1', t = T_s, m = M) # (T_s, M, 1)

    # Compute cov. and var. across each day and factor:
    cov = reduce(FF * PP, 't m d -> t d', 'sum') # (T_s, D)
    var = reduce(FF * FF, 't m d -> t d', 'sum') # (T_s, D)

    # Compute betas, with var = 0 -> beta = np.nan:
    betas = (cov / var) # (T_s - 1, D)
    betas_lag = betas[:-1, :] # (T_s - 1, D)

    # Get the adjusted daily returns:
    P_day_hat = reduce(F_day_s * betas_lag, 't d -> t', 'sum') # (T_s - 1,)
    P_day_adj = P_day_s - P_day_hat # (T_s - 1,)

    return cp.asnumpy(betas), cp.asnumpy(P_day_adj)

```

6 Conditional alphas

Now we need to get the conditional alphas from the risk-adjusted returns and state variables.

The following math and commentary are taken largely as-is from the paper. First, I present the theory, then the implementation in Python, and finally the results.

6.1 Methodology

The methodology is taken from Corradi et al. (2013), section 2.2.

In [src/betas.ipynb](#)/Chapter 5, we calculated the risk-adjusted returns $\hat{Z}_{i,t+1,M}$. This is useful because $Z_{i,t+1} = \alpha_{i,t} + \epsilon_{i,t+1} + \eta_{i,t+1}$. Identification of the conditional alpha stems from the fact that the conditional expectations of $\epsilon_{i,t+1}$ and $\eta_{i,t+1}$ are zero, so that $\alpha_{i,t} \equiv \mathbb{E}(Z_{i,t+1}|C_t)$.

Thus, it suffices to estimate the conditional expectation of the risk-adjusted returns, $\mathbb{E}(Z_{i,t+1}|C_t)$, to back out conditional alphas.

After estimating $\hat{Z}_{i,t+1,M}$ with the realized betas, we only need to estimate the conditional expectation $\mathbb{E}(\hat{Z}_{i,t+1,M}|C_t)$ of the realized risk-adjusted returns using nonparametric kernel methods. The paper proposes a consistent estimator with respect to the number of intraday observations M .

The high dimensionality of the state variable vector C_t clashes with the nonparametric kernel estimator convergence, so we need to summarize it with PCA. Let then $\widehat{PC}_t = (\widehat{PC}_{1,t}, \dots, \widehat{PC}_{k,t})'$ denote the $k \leq k_C$ principal component estimates of C_t .

The conditional alpha estimator is:

$$\hat{\alpha}_{i,t} = \hat{m}_{i,T,M}(\widehat{PC}_t) = \frac{\frac{1}{Th_T^k} \sum_{\ell=1}^{T-1} \hat{Z}_{i,\ell+1,M} K\left(\frac{\widehat{PC}_{\ell} - \widehat{PC}_t}{h_T}\right)}{\hat{g}_T(\widehat{PC}_t)}$$

where $\hat{g}_T(\widehat{PC}_t) = \frac{1}{Th_T^k} \sum_{\ell=1}^{T-1} K\left(\frac{\widehat{PC}_{\ell} - \widehat{PC}_t}{h_T}\right)$ is the kernel estimator of the density g of the vector of principal components.

To circumvent the difficulty of estimating $m_i(pc)$ over regions of low density, we consider a trimmed version of $\hat{m}_{i,T,M}(pc)$ as in Andrews (1995):

$$tr_T\{\hat{m}_{i,T,M}(pc)\} = \hat{m}_{i,T,M}(pc)1\{pc \in \hat{G}_T(pc)\}$$

where $\hat{G}_T(pc) = \{c : \hat{g}_T(pc) > d_T\}$.

6.1.1 Expanding window calculation

The paper proposes an expanding window to restrict attention to real-time information:

To obtain conditional alphas, we entertain an expanding window to restrict attention to real-time information. We estimate the conditional expectation of the risk-adjusted returns using either an affine specification or a kernel-regression approach using a multivariate standard Gaussian kernel with rule-of-thumb bandwidths. As for the amount of trimming in (10), uniform consistency requires $d_T = O(h_T^\zeta)$ with $0 < \zeta < 1/4$. Note that, because $h_T \rightarrow 0$ as $T \rightarrow \infty$, trimming becomes more aggressive as ζ declines. We set ζ to 0.20 to enforce only a mild trimming, though we also experiment with other values of ζ in Section 4.2.

This windowed version calculates $\widehat{PC}_t$ using data from $1 : t$ only. And the bandwidth h_T and trimming threshold d_T are calculated separately for each t using t as T . Additionally, the kernel estimator does not use data up to T , but only up to $t - 1$. To differentiate the notation, let $W = t$ be the total window size:

$$\hat{\alpha}_{i,t} = \hat{m}_{i,W,M}(\widehat{PC}_t) = \frac{\frac{1}{Wh_W^k} \sum_{\ell=1}^{W-1} \hat{Z}_{i,\ell+1,M} K\left(\frac{\widehat{PC}_\ell - \widehat{PC}_t}{h_W}\right)}{\hat{g}_W(\widehat{PC}_t)}$$

6.1.2 Gaussian Kernel

For a single real-valued standardized deviation $u \in \mathbb{R}$, the standard univariate 1D Gaussian kernel function $K(u)$ is defined as the standard normal probability density function:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}u^2\right)$$

In our nonparametric setting, for a single feature j at time ℓ evaluated at time t , $u_{\ell,j} = \frac{\widehat{PC}_{\ell,j} - \widehat{PC}_{t,j}}{h_W}$.

As established in Assumption $A(iv)$ of the paper, the k -dimensional joint kernel $K(u)$ across k independent principal components $u = (u_1, u_2, \dots, u_k)'$ is constructed as a product kernel of k univariate Gaussian kernels:

$$K(u) = \prod_{j=1}^k K(u_j) = \prod_{j=1}^k \left[\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}u_j^2\right) \right]$$

By factoring out the constant coefficient and applying the exponent product rule $\prod_{j=1}^k e^{a_j} = e^{\sum_{j=1}^k a_j}$, the joint kernel simplifies to:

$$K(u) = \left(\frac{1}{\sqrt{2\pi}}\right)^k \prod_{j=1}^k \exp\left(-\frac{1}{2}u_j^2\right) = \left(\frac{1}{\sqrt{2\pi}}\right)^k \exp\left(-\frac{1}{2} \sum_{j=1}^k u_j^2\right)$$

6.2 The data

The daily frequency of the analysis dictates the choice of state variables. In particular, we consider changes in the VIX index and in the Fed Fund rate, credit and term spreads, momentum, size and value factors, short- and long-term reversal factors, and volatility risk premium (namely, difference between the VIX index and realized volatility). To avoid the curse of dimensionality, we consider only their three first principal components.

The data is stored in `data/states_raw/FactorData_2018.mat`, where the “factors” key has a dataset of several state variables, and the “ADSdata” has many columns of the same variable, ending at different dates, with each column being used for the window that ends at the same date.

6.3 The algorithm

Similar to before, the operations are independent between stocks, so the main loop is a parallelized loop over stocks.

Calculating the PC and kernel bandwidths and trimming thresholds is done separately for each stock, considering only the state variable periods that match the stock’s trading days. This is considered more appropriate than calculating the PC and bandwidths outside the stock loop, but it is more computationally expensive.

This is the most expensive part of the whole project and could be optimized especially with numba or Cython in the innermost functions (e.g. PCA and kernel density estimation).

The main algorithm sets up the stock and state variables data and runs the following procedure for each stock, once for 1-minute data and once for 5-minute data:

```
// Parameters:
k = 3 // Number of principal components to use
scaler = 1 // Bandwidth scaling factor
trim_exp = 0.20 // Trimming exponent
trim_scaler = 1 // Trimming scaling factor

procedure get_permno_alphas(
    permno : Integer;
    data_permnos : DataFrame; // Stock adjusted returns
    f : Matrix;               // "factors"
    ads : Matrix;             // "ADSdata"
);
    // 1. Data Alignment
    Z_data = query permno from data_permnos;
    intersect timestamps to align Z_data, f, and ads;

    Z_1_T, f_1_T, ads_1_t = aligned data; // Shape: (T,)

    T = length of Z_1_T;
    Initialize res_ts, res_alpha, res_linear of size (T - warmup);

    // 2. Expanding window loop (restricting to real-time information)
    for w = warmup to T do
        begin
            wi = w - warmup;
```

```

res_ts[wi] = timestamp at w - 1;
W = w; // Current window size

// Extract windowed state variables
ads_1_w = ads_1_t column with data up to w // Shape: (W, 1)
C_1_W = concatenated f_1_T[1:W, :], ads_1_w // Shape: (W, F)

// --- Dimensionality Reduction (PCA) ---
C_norm = standardize(C_1_W); // Shape: (W, F)
eigvecs = eigen decomposition of cov(C_norm), sorted by descending eigenvalues;
PC_1_W = (C_norm * eigvecs[:, 1:k]) / 100.0; // Shape: (W, k)
PC_1_W = standardize(PC_1_W); // Shape: (W, k)

// --- Nonparametric bandwidth & thresholds ---
bandrate = (4 / (k + 2)) ** (1 / (k + 4)) * (W ^ (- 1 / (k + 4)));
h_W = scaler * bandrate; // Kernel bandwidth
d_W = (bandrate ^ trim_exp) * trim_scaler; // Trimming threshold

// --- Split current state vs historical states & targets ---
PC_W = PC_1_W[W, :]; // Shape: (k,) | Conditioning state at time W
PC_1_Wm1 = PC_1_W[1:W-1, :]; // Shape: (W-1, k) | Historical state variables
Z_2_W = Z_1_T[2:W]; // Shape: (W-1,) | Target risk-adjusted returns

// --- Linear Conditional Alpha ---
res_linear[wi] = fitted value of PC_W from OLS regression of Z_2_W on PC_1_Wm1;

// --- Nonparametric Conditional Alpha (Nadaraya-Watson with Trimming) ---
// Evaluate multivariate Gaussian kernel across the k dimensions for historical distan
K_x = gaussian_kernel((PC_1_Wm1 - PC_W) / h_W) / (h_W ** k); // Shape: (W-1,)

density = mean(K_x); // Kernel density estimator evaluated at PC_W

// Trim alpha to 0.0 in regions of low state variable density (Andrews 1995)
if density <= d_W then
    res_alpha[wi] = 0.0;
else
    res_alpha[wi] = mean(K_x * Z_2_W) / density;
end;

return (permno, res_ts, res_alpha, res_linear);
end;

```

6.4 Code setup

For debugging:

```
from importlib import reload
```

Core modules:

```

import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import numpy as np
import polars as pl

from pickle import dump
from scipy.io import loadmat, savemat
from h5py import File
from tqdm import tqdm

from typing import NamedTuple, Final
from numpy.typing import NDArray

import src.utils as ut
import src.graphs as plot
from src.parameters import PARAMETERS as PARS

class sio:
    loadmat = staticmethod(loadmat)
    savemat = staticmethod(savemat)

class h5py:
    File = staticmethod(File)

class pkl:
    dump = staticmethod(dump)

```

Parameters:

```

class AlphasParameters(NamedTuple):
    window_warmup: int # Initial burn-in window size
    scaler: float      # Bandwidth scaling factor
    trim_exp: float    # Trimming exponent zeta in (0, 1/4)
    trim_scaler: float # Trimming density threshold scaling factor
    k: int              # Number of principal components to retain
    freq: str           # Frequency of the data

PARS_ALPHAS: Final[AlphasParameters] = AlphasParameters(
    window_warmup = 500,
    scaler = 1,      # Try 1, 2, 3.5
    trim_exp = 0.2,  # Bounded by (0, 1/4)
    trim_scaler = 0, # 0.01, # Try 0.01 to 0.05
    k = 3,
    freq = "5min"
)

k = PARS_ALPHAS.k
band_const = (4 / (k + 2)) ** (1 / (k + 4))
band_exp = - 1 / (k + 4)
#trim_scaler = PARS_ALPHAS.trim_scaler * ((2.0 * np.pi) ** (- k / 2.0))

```

```
DATE_START, DATE_END = (
    pl.lit(x.astype("datetime64[D]").astype(np.uint32))
    for x in [np.datetime64("2001-01-01 00:00:00", "m"), np.datetime64("2017-12-29 23:59:59",
)])
```

6.5 Data validation

The paper states the following data sources:

The daily frequency of the analysis dictates the choice of state variables. In particular, we consider changes in the VIX index and in the Fed Fund rate, credit and term spreads, momentum, size and value factors, short- and long-term reversal factors, and volatility risk premium (namely, difference between the VIX index and realized volatility). We gather the Fama-French, momentum and reversal factors from Kenneth French's webpage at http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html. Interest rates are from the FRED database, whereas the VIX index is from the Chicago Board Options Exchange. To avoid the curse of dimensionality, we consider only their three first principal components.

This data was saved into the MATLAB data file `data/states_raw/FactorData_2018.mat`.

The timestamp variables seem to be days from epoch 0000-01-01. MATLAB has 1-indexing for days, so we need to subtract 719_529 to align with the epoch 1970-01-01 being used in Python.

```
def load_state_data(path: str) -> pl.DataFrame:
    state_mat = sio.loadmat(path)

    state_raws = []
    for key in ["factors", "ADSdata"]:
        mat = state_mat[key]
        data = (
            pl.DataFrame(mat[:, 1:])
            .with_columns(ts_day_ny = (mat[:, 0] - 719_529).astype(np.uint32))
            .select(pl.col("ts_day_ny"), pl.all().exclude("ts_day_ny"))
        )
        state_raws.append(data)

    data_state = (
        state_raws[0]
        .join(state_raws[1], on = "ts_day_ny", how = "inner")
        .filter(
            (pl.col("ts_day_ny") >= DATE_START)
            & (pl.col("ts_day_ny") <= DATE_END)
        )
        .with_columns(pl.all().fill_nan(None))
        .pipe(lambda df:
            df.select(
                col for col in df.columns
                if not (df[col].is_null().any() or df[col].is_nan().any())
            )
        )
        .tail(-1) # To match with adjusted returns' dropped 1st observation
```

```
)
```

```
return data_state
```

```
data_state = load_state_data("data/states_raw/FactorData_2018.mat")  
data_state.write_parquet("data/states_raw/states_clean.parquet")
```

A first look at the data is as below. Note that it has different levels of decimal precision, something that comes from the different sources.

```
with pl.Config(tbl_cols = 7):  
    display(data_state)
```

ts_day_ny	column_0	column_1	...	column_894	column_895	column_896
u32	f64	f64	...	f64	f64	f64
11325	-3.39	-0.32	...	-0.790969	-0.790969	-0.790969
11326	0.37	-0.43	...	-0.808035	-0.808035	-0.808035
11327	1.7	-0.09	...	-0.825037	-0.825037	-0.825037
11330	1.17	0.15	...	-0.875067	-0.875067	-0.875067
11331	-1.85	-0.03	...	-0.891353	-0.891353	-0.891353
...	...	...	...	...	...	...
17522	0.28	0.0	...	-0.04509	-0.04509	-0.045083
17526	0.35	0.0	...	-0.105166	-0.105166	-0.105156
17527	0.22	0.0	...	-0.120432	-0.120432	-0.120422
17528	-0.29	0.0	...	-0.135768	-0.135768	-0.135757
17529	0.86	-0.09	...	-0.151176	-0.151176	-0.151164

Returns loading:

```
data_adjusted = pl.scan_parquet("data/stocks_1min_adjusted.parquet")
```

The state variables dataset must be validated:

- The date ranges in both datasets should be the same. Ideally both date vectors should be identical, but it is not a problem if not.
- There should be no duplicate values in the date vectors.
- There should be no nulls or NaNs in the data.
- All variables should be float64.

```
range_states = tuple(f(data_state["ts_day_ny"]) for f in [min, max])  
  
range_returns = (  
    data_adjusted  
    .select(  
        min = pl.col("ts_day_ny").min(),  
        max = pl.col("ts_day_ny").max()  
    )  
    .collect()  
    .row(0)
```

```

)

if range_states != range_returns:
    raise ValueError(
        f"State and returns data have different date ranges: "
        f"{range_states} vs {range_returns}"
    )

```

Some days are only present in the state variables or in the returns data. While it would be ideal to not have that, it is not a problem to drop them.

```

days_state = pl.from_epoch(data_state["ts_day_ny"], "d").to_numpy()

days_only_state = ~ np.isin(days_state, PARS.trading_days_final)
print(
    f"{np.sum(days_only_state)} days in state but not in returns:",
    ut.format_days_byyear(days_state[days_only_state])
)

days_only_returns = ~ np.isin(PARS.trading_days_final, days_state)
print(
    f"\n{n{np.sum(days_only_returns)} days in returns but not in state:",
    ut.format_days_byyear(PARS.trading_days_final[days_only_returns])
)

```

49 days in state but not in returns:

- 2001: 06-08, 07-03, 11-23,
- 2002: 07-05, 09-11, 11-29, 12-24,
- 2003: 07-03, 08-15, 11-28, 12-24, 12-26,
- 2004: 11-26,
- 2005: 11-25, 12-23,
- 2006: 07-03, 11-24,
- 2007: 07-03, 11-23, 12-24,
- 2008: 07-03, 11-28, 12-24, 12-26,
- 2009: 11-27, 12-24,
- 2010: 11-26, 12-27,
- 2011: 08-15, 11-25, 12-23, 12-27,
- 2012: 07-03, 11-23, 12-24,
- 2013: 07-03, 08-22, 11-29, 12-24,
- 2014: 07-03, 11-28, 12-24, 12-26,
- 2015: 11-27, 12-24,
- 2016: 11-25, 12-23,
- 2017: 07-03, 11-24,

40 days in returns but not in state:

- 2001: 01-02, 03-08, 03-21, 10-08, 11-12,
- 2002: 10-14, 10-31, 11-11,
- 2003: 01-17, 01-21, 10-13, 11-11,
- 2004: 01-12, 01-13, 10-11, 10-12, 11-11,
- 2005: 10-10, 11-11,
- 2006: 10-09,

```

- 2007: 10-08, 11-12,
- 2008: 10-13, 11-11,
- 2009: 10-12, 11-11,
- 2010: 10-11, 11-11,
- 2011: 10-10, 11-11,
- 2012: 10-08,
- 2013: 10-14, 11-11,
- 2014: 10-13, 11-11,
- 2015: 11-11,
- 2016: 10-10, 11-11,
- 2017: 10-09, 11-10,

```

Checking date duplicates:

```

if data_state["ts_day_ny"].is_duplicated().any():
    raise ValueError("Duplicate dates in state data")

```

Checking date types and nulls/NaNs:

```

if not all(
    x == pl.Float64
    for x in data_state.select(pl.all().exclude("ts_day_ny")).dtypes
):
    raise ValueError("State data contains non-float64 columns")

(
    any(data_state.select(f(pl.all().exclude("ts_day_ny")).any()).row(0))
    for f in [pl.Expr.is_nan, pl.Expr.is_null]
)

if any(
    any(data_state.select(f(pl.all().exclude("ts_day_ny")).any()).row(0))
    for f in [pl.Expr.is_nan, pl.Expr.is_null]
):
    raise ValueError("State data contains nulls or NaNs")

```

6.6 Code implementation

6.6.1 Data loading

After checking the data, it is better to save it into a different format.

The ADS data is turned into a map of W to the column with data up to W . For every column, it finds the date of the last non-NaN observation. For a window's W date, we want to use the column with such date being $\leq W$. It also removes columns with duplicate last observation dates:

```

def get_ads_data(ads: NDArray, ads_idx: NDArray):
    ads_last_obs = np.where(
        np.isnan(ads).any(axis = 0),
        np.argmax(np.isnan(ads), axis = 0) - 1,
        len(ads) - 1
    )

```

```

)

_, idx_unique = np.unique(ads_last_obs, return_index = True)
idx_unique = np.sort(idx_unique)

ads = ads[:, idx_unique]
ads_last_obs = ads_last_obs[idx_unique]

#return {ads_idx[ads_last_obs[i]]: ads[:, i] for i in range(ads.shape[1])}
return (ads_idx[ads_last_obs], ads)

```

Using that, we align all the data sources:

```

state_mat = loadmat("data/states_raw/FactorData_2018.mat")
state_raws = []
for key in ["factors", "ADSdata"]:
    mat = state_mat[key]
    data = (
        pl.DataFrame(mat[:, 1:])
        .with_columns(ts_day_ny = (mat[:, 0] - 719_529).astype(np.uint32))
        .select(pl.col("ts_day_ny"), pl.all().exclude("ts_day_ny"))
        .filter(
            (pl.col("ts_day_ny") >= DATE_START)
            & (pl.col("ts_day_ny") <= DATE_END)
        )
        .tail(-1) # To match with adjusted returns' dropped 1st observation
    )
    state_raws.append(data)

data_factors = state_raws[0]
f_idx = data_factors["ts_day_ny"].to_numpy()
f = data_factors.drop("ts_day_ny").to_numpy()

data_ads = state_raws[1]
ads_idx = data_ads["ts_day_ny"].to_numpy()
ads = data_ads.drop("ts_day_ny").to_numpy()

# Align factors and ADS data:
_, ads_inf, f_in_ads = np.intersect1d(
    ads_idx, f_idx, return_indices = True
)

f, f_idx = f[f_in_ads], f_idx[f_in_ads]
ads, ads_idx = ads[ads_inf], ads_idx[ads_inf]

ads_last_idx, ads = get_ads_data(ads, ads_idx)

```

6.6.2 Estimators

The kernel and affine estimators are implemented in separate functions.

```

INV_SQRT_2PI_K: float = (
    (1.0 / np.sqrt(2.0 * np.pi))
    ** PARS_ALPHAS.k
)

def kernel_gauss(
    x: NDArray[np.float64]
) -> NDArray[np.float64]:
    return INV_SQRT_2PI_K * np.exp(-0.5 * np.sum(x**2, axis = 1)) # Across k

def cae_gauss_trim(
    PC_W: NDArray[np.float64],      # (k,) conditioning state variables at time t (PC_W)
    Z_2_W: NDArray[np.float64],    # (W-1,) target risk-adjusted returns (Z_{2:W})
    PC_1_Wm1: NDArray[np.float64], # (W-1, k) historical state variables (PC_{1:W-1})
    h_W: np.float64,               # Kernel bandwidth (h_W)
    d_W: np.float64                 # Trimming density threshold (d_W)
) -> np.float64:
    K_1_Wm1 = kernel_gauss((PC_1_Wm1 - PC_W) / h_W) # (T-1,)
    K_x = K_1_Wm1 / (h_W ** k)
    deno = np.mean(K_x) # Across t

    if np.isnan(deno):
        raise ValueError("Denominator is NaN in cae_gauss_trim.")
    if deno <= d_W:
        return np.float64(0.0)

    num = np.mean(K_x * Z_2_W) # Across t
    return num / deno
# NOTE: We could use denominator cancellation to avoid some np.mean()

def cae_linear(
    PC_W: NDArray[np.float64],      # (k,)
    Z_2_W: NDArray[np.float64],    # (T-1,)
    PC_1_Wm1: NDArray[np.float64]  # (T-1, k)
) -> np.float64:
    X = np.column_stack([np.ones(PC_1_Wm1.shape[0]), PC_1_Wm1])

    beta, _, _, _ = np.linalg.lstsq(X, Z_2_W)
    return beta[0] + PC_W @ beta[1:]

```

A helper standard PCA function is also created:

```

def pca_window(
    C_1_W: NDArray[np.float64], # (W, k) historical state variables
    k: int = 3
) -> NDArray[np.float64]:
    # Standardization:
    mean = np.mean(C_1_W, axis = 0) # (k,) Across t
    std = np.std(C_1_W, axis = 0, ddof = 1) # (k,) Across t
    if np.any(std == 0.0):
        raise ValueError("Constant feature detected in PCA window.")

```

```

C_1_W_norm = (C_1_W - mean) / std

# Eigendecomposition:
cov_mat = np.cov(C_1_W_norm, rowvar = False, ddof = 1) # (k, k)
eigvals, eigvecs = np.linalg.eigh(cov_mat)

# Principal components:
idx_sort = np.argsort(eigvals)[::-1]
eigvecs = eigvecs[:, idx_sort]
scores = C_1_W_norm @ eigvecs[:, :k] # (T, k)

# Rescaling:
return scores / 100.0

```

6.6.3 Orchestration

The stock iteration function is `get_permno_alphas`:

```

def get_permno_alphas(
    permno: int
):
    data_permno = (
        data_permnos
        .filter(pl.col("permno") == permno)
        .select(["ts_day_ny", "adjusted_return"])
        .collect()
    )

    Z_idx = data_permno["ts_day_ny"].to_numpy()
    Z_1_T = data_permno["adjusted_return"].to_numpy()

    # Align stock and factor data (and thus also with ADS data):
    zc_idx, z_in_f, f_in_z = np.intersect1d(
        Z_idx, f_idx, return_indices = True
    )

    Z_1_T = Z_1_T[z_in_f]
    f_1_T = f[f_in_z]
    ads_1_T = ads[f_in_z]

    n = len(Z_1_T)
    if n <= PARS_ALPHAS.window_warmup:
        # Can happen as permnos are filtered before aligning with state data
        return (None, None, None, None)

    n_out = n - PARS_ALPHAS.window_warmup
    res_ts = np.full(n_out, 0, dtype = np.uint32)
    res_alpha = np.full(n_out, np.nan, dtype = np.float64)
    res_linear = np.full(n_out, np.nan, dtype = np.float64)

    for w in range(PARS_ALPHAS.window_warmup, n): # CHECK: add +1 to start?

```

```

wi = w - PARS_ALPHAS.window_warmup
res_ts[wi] = zc_idx[w - 1] # CHECK: w-1?

# Window data:
w_idx = zc_idx[:w]
W = len(w_idx) - 1

# if W < k: continue # Unneeded because of warmup window size

f_1_W = f_1_T[:w, :]
ads_current_col = np.searchsorted(ads_last_idx, zc_idx[w - 1])
ads_1_W = ads_1_T[:w, ads_current_col:(ads_current_col + 1)]

# PCA and its standardization:
PC_1_W = pca_window(np.hstack([f_1_W, ads_1_W]), k)

PC_std = np.std(PC_1_W, axis = 0)
if np.any(PC_std == 0.0): # Surely never happens
    raise ValueError("Constant feature detected in PCA window.")
PC_1_W = (PC_1_W - np.mean(PC_1_W, axis = 0)) / PC_std

# Bandwidth and trimming:
bandrate = band_const * (W ** band_exp)
# smx = np.sqrt(np.mean(np.std(PC_1_W, axis = 0, ddof = 1)**2)) # Should be = 1 and un
smx = 1.0

h_W = PARS_ALPHAS.scaler * (bandrate * smx)
d_W = (bandrate * smx)**PARS_ALPHAS.trim_exp * INV_SQRT_2PI_K * PARS_ALPHAS.trim_scale

# Compute alphas:
PC_W = PC_1_W[-1, :]
PC_1_Wm1 = PC_1_W[:-1, :]
Z_2_W = Z_1_T[1:w]

res_linear[wi] = cae_linear(PC_W, Z_2_W, PC_1_Wm1)
res_alpha[wi] = cae_gauss_trim(PC_W, Z_2_W, PC_1_Wm1, h_W, d_W)

return (permno, res_ts, res_alpha, res_linear)

```

And the main orchestration runs all the stocks in parallel. The `worker_init` sets up the global variables (i.e. datasets and constants) for each worker and can be seen in `data/alphas_mp.py`.

```

from concurrent.futures import ProcessPoolExecutor

def alphas_mp(
    permnos: list[int],
    freq: str,
    max_workers: int | None = None,
    chunksize: int = 1,
    quiet: bool = False
) -> pl.DataFrame:
    with ProcessPoolExecutor(

```

```

        initializer = worker_init,
        initargs = (freq,),
        max_workers = max_workers
    ) as executor:

        map_iterator = executor.map(
            get_permno_alphas,
            permnos,
            chunksize = chunksize
        )

        results = {"permno": [], "ts_day_ny": [], "alpha": [], "linear_alpha": []}

        for permno, t_arr, a_arr, l_arr in tqdm(map_iterator, disable = quiet, total = len(per
            if permno is None:
                continue
            results["permno"].append(np.full(len(t_arr), permno, dtype = np.uint32))
            results["ts_day_ny"].append(t_arr)
            results["alpha"].append(a_arr)
            results["linear_alpha"].append(l_arr)

        # Single bulk C-level concatenation and Polars wrapping at the end
        result_data = pl.DataFrame({k: np.concatenate(v) for k, v in results.items()})

    return result_data

```

6.7 Calculating alphas

We then load the list of PERMNOs that pass the liquidity filters and run the loop.

```

permnos = np.load("data/permnos_liquid.npy")

from src import alphas_mp

if False: # Run only once
    result_1min = alphas_mp.alphas_mp(
        permnos, "1min",
        max_workers = 6, chunksize = 5
    )
    result_1min.write_parquet("data/alphas_1min.parquet")

if False: # Run only once
    result_5min = alphas_mp.alphas_mp(
        permnos, "1min",
        max_workers = 6, chunksize = 5
    )
    result_5min.write_parquet("data/alphas_5min.parquet")

```

6.8 Visualizing results

First, let us select some random stocks to visualize and print the datasets.

```
vis_permnos = (  
    pl.read_csv("data/stocks_raw/counts_permno.csv")  
    ["permno"].sample(4).to_numpy()  
)
```

```
result_1min = pl.read_parquet("data/alphas_1min.parquet")  
result_5min = pl.read_parquet("data/alphas_5min.parquet")
```

First, a look at the data statistics:

```
result_1min[["alpha", "linear_alpha"]].drop_nans().describe()
```

statistic str	alpha f64	linear_alpha f64
"count"	2.698958e6	2.698958e6
"null_count"	0.0	0.0
"mean"	0.00011	0.000015
"std"	0.009438	0.002108
"min"	-0.62898	-0.101278
"25%"	-0.000722	-0.00055
"50%"	0.000248	0.000154
"75%"	0.001124	0.00077
"max"	0.651504	0.077865

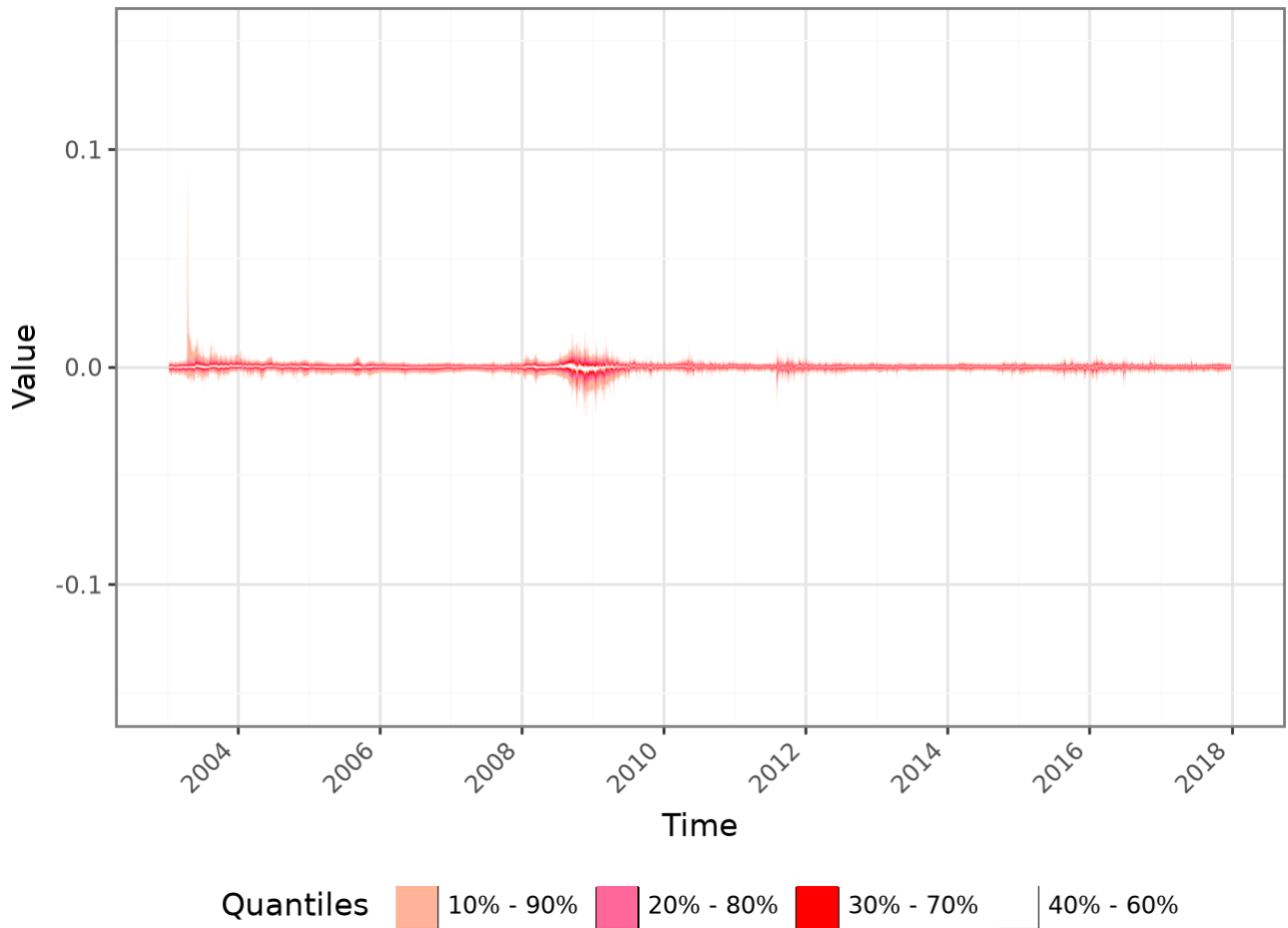
```
result_5min[["alpha", "linear_alpha"]].drop_nans().describe()
```

statistic str	alpha f64	linear_alpha f64
"count"	2.698958e6	2.698958e6
"null_count"	0.0	0.0
"mean"	-0.000244	-0.000263
"std"	0.010352	0.002265
"min"	-0.697051	-0.108568
"25%"	-0.001011	-0.000882
"50%"	0.000026	-0.000062
"75%"	0.000948	0.000646
"max"	0.683863	0.093873

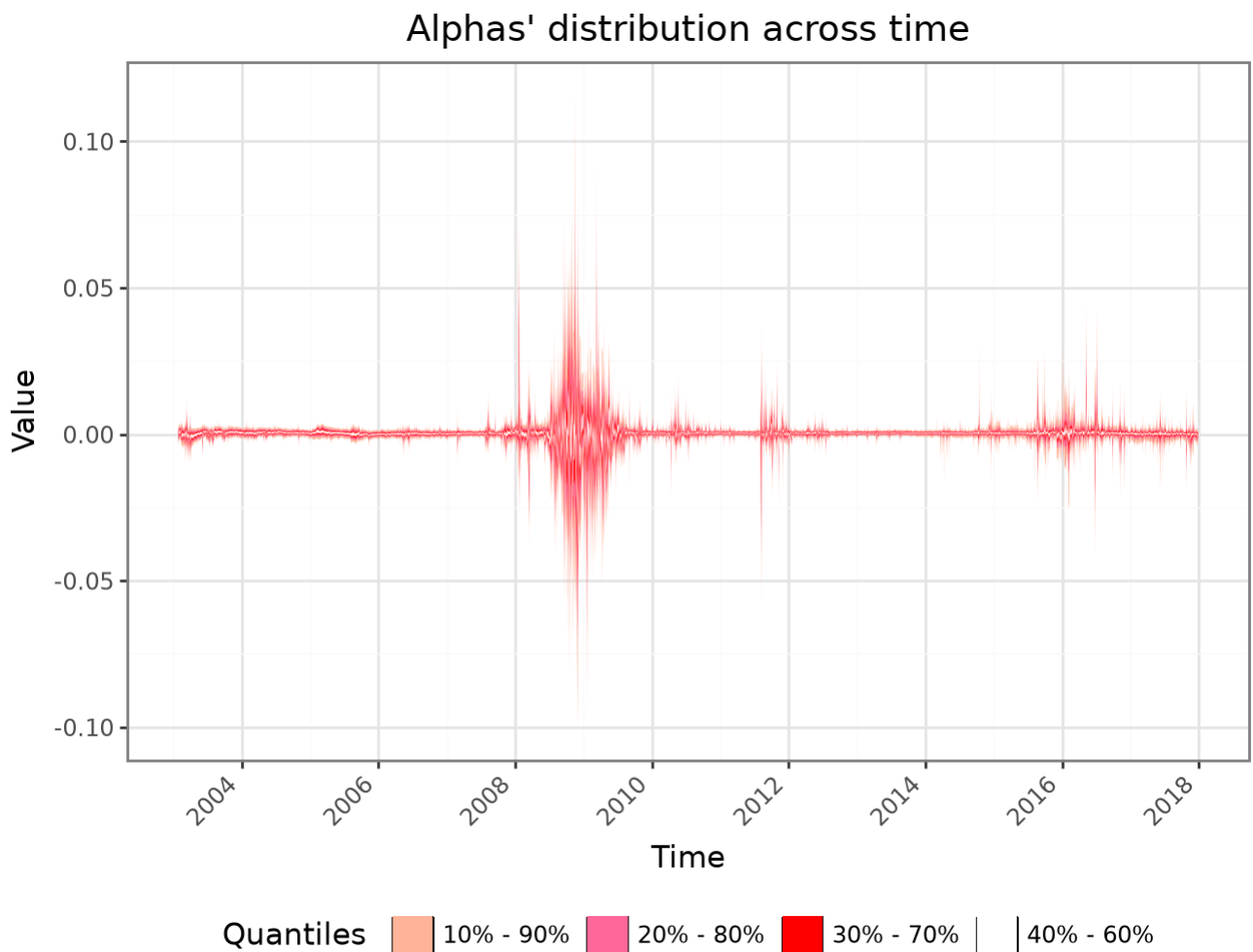
Then, the alpha time series for some stocks:

```
# plot.alphas_river("data/alphas_5min.parquet", alpha_col = "linear_alpha")  
plot.alphas_river(alpha_col = "linear_alpha")
```

Alphas' distribution across time



```
# plot.alphas_river("data/alphas_5min.parquet", alpha_col = "alpha")  
plot.alphas_river(alpha_col = "alpha")
```



6.9 Appendix

6.9.1 Saving the results as .mat files

For the rest of the paper's results, MATLAB code is used. Thus, the alphas are saved as .mat files, accompanied by some stock information (volume, market cap) from `data/stocks_raw/PermRetBetas_CRSP_v07.mat`.

This is run for every parameter that was used to calculate the alphas:

```
freq = "5min"
scaler = 2
```

```
permnos = np.load("data/permnos_liquid.npy")
data_betas = pl.scan_parquet(f"data/betas_{freq}.parquet")
data_alphas = pl.scan_parquet(f"data/alphas_{freq}.parquet")
data_adjusted = pl.scan_parquet(f"data/stocks_{freq}_adjusted.parquet")
data_permnos = pl.scan_parquet("data/stocks_1min_returns.parquet")

output1_list = []
output_num_list = []

with h5py.File("data/stocks_raw/PermRetBetas_CRSP_v07.mat", "r") as f:
    # 1. Parse VarNmHead (Cell array of strings)
```

```

VarNmHead = []
for ref in f["VarNmHead"][:].flatten():
    # MATLAB strings in HDF5 are stored as UTF-16 integer arrays
    # We dereference the object, read the chars, and join them
    var_name = "".join(chr(c[0]) for c in f[ref][:])
    VarNmHead.append(var_name)

# Apply Correction: VarNmHead{17} = 'Bss5mOC'
if len(VarNmHead) >= 17:
    VarNmHead[16] = 'Bss5mOC'

# Map column names to integer indices for fast lookup
col_idx = {name: idx for idx, name in enumerate(VarNmHead)}

# 2. Iterate through allPerms
# In MATLAB, allPerms is (N x 8). In h5py, the shape is transposed to (8 x N).
#n_stocks = f["allPerms"].shape[1]
#for i in range(n_stocks):
permnos_mat = np.array(
    [f[ref][0, 0] for ref in f["allPerms"][0, :]],
    dtype = np.uint32
)
for permno in tqdm(permnos):
    i = np.searchsorted(permnos_mat, permno)
    if permno != f[f["allPerms"][0, i]][0, 0]:
        raise ValueError(f"PERMNO {permno} not found in allPerms at index {i}")

# Extract BetasRets matrix (MATLAB Column 4 -> Python Index 3)
betas_rets_ref = f["allPerms"][3, i]

# READ AND TRANSPOSE to convert back to standard row-major matrix
BetasRets = f[betas_rets_ref][:].T

# Extract columns using our mapping
retAsset = BetasRets[:, col_idx['ret']]
p_close = BetasRets[:, col_idx['p.close']]

V1 = BetasRets[:, col_idx['prc']]
V2 = BetasRets[:, col_idx['vol']]
V3 = BetasRets[:, col_idx['MarketCap']]
V4 = BetasRets[:, col_idx['t.close']] - BetasRets[:, col_idx['t.open']]
V5 = BetasRets[:, col_idx['numtrd2']]

# Get dates:
data_dates = (
    pl.DataFrame({"date" : BetasRets[:, col_idx['date']].astype(np.int64)})
    .select(
        ts_day_ny = pl.col("date").cast(pl.String).str.to_date("%Y%m%d")
        .dt.epoch("d").cast(pl.UInt32)
    )
)

```

```

intt = data_dates["ts_day_ny"].to_numpy()# - 719_529

# Get our betas:
B = (
    data_dates.join(
        data_betas.filter(pl.col("permno") == permno).collect(),
        on = "ts_day_ny", how = "left"
    )
    .select([f"beta_{i}" for i in range(1, 7)]).to_numpy()
)

# Get our resid:
resid = (
    data_dates.join(
        data_adjusted.filter(pl.col("permno") == permno).collect(),
        on = "ts_day_ny", how = "left"
    )
    ["adjusted_return"].to_numpy()
)

# Get our ret* (which is group_by permno,day sum log_return)
ret = (
    data_permnos
    .filter(pl.col("permno") == permno)
    .group_by(ts_day_ny = pl.col("ts_min_ny") // (60 * 24))
    .agg(pl.col("log_return").sum())
    .collect()
    .join(data_dates, on = "ts_day_ny", how = "right")
    ["log_return"].to_numpy()
)

# Get our condalpha and linalpha:
alphas = (
    data_dates.join(
        data_alphas.filter(pl.col("permno") == permno).collect(),
        on = "ts_day_ny", how = "left"
    )
    .select(["alpha", "linear_alpha"])
)
condalpha = alphas["alpha"].to_numpy()
linalpha = alphas["linear_alpha"].to_numpy()

# Reconstruct output arrays (omitting int, resid, condalpha, linalpha as requested)
# np.column_stack forces 1D arrays into vertical columns side-by-side
output0 = np.column_stack([intt, B, resid, retAsset, p_close, ret, condalpha, linalpha])
output00 = np.column_stack([intt, B, V1, V2, V3, V4, V5])

# Slicing equivalent to MATLAB's (window:end, :)
out0_sliced = output0[PARS_ALPHAS.window_warmup:, :]
out00_sliced = output00[PARS_ALPHAS.window_warmup:, :]

# Append to our tracking lists

```

```

        output1_list.append([permno, out0_sliced, out00_sliced])
        output_num_list.append(out0_sliced)

# 3. Format into strict MATLAB-compatible structures
# Create a NumPy "object" array so SciPy saves it as a proper MATLAB Cell Array
output1_cell = np.empty((len(output1_list), 3), dtype=object)
for i, row in enumerate(output1_list):
    output1_cell[i, 0] = row[0]
    output1_cell[i, 1] = row[1]
    output1_cell[i, 2] = row[2]

# Replicate cell2mat(output1(:,2)) by stacking vertically
output_mat = np.vstack(output_num_list)

# 4. Save the outputs
sio.savemat(
    f"data/final/realized_alpha_oos_{freq}_cell_sc{scaler}.mat",
    {"output1": output1_cell}
)
sio.savemat(
    f"data/final/realized_alpha_oos_{freq}_num_sc{scaler}.mat",
    {"output": output_mat}
)

```

7 References

- Aït-Sahalia, Yacine, and Dacheng Xiu. 2015. “Principal Component Analysis of High Frequency Data.” Working Paper 21584. Working Paper Series. National Bureau of Economic Research. <https://doi.org/https://doi.org/10.3386/w21584>.
- Aleti, Saketh. 2022. “The High-Frequency Factor Zoo.” <https://doi.org/http://dx.doi.org/10.2139/ssrn.4021620>.
- Corradi, Valentina, Walter Distaso, Marcelo Fernandes, and Asger Lunde. 2013. “Conditional Alphas and Realized Betas.” In. <https://hdl.handle.net/10438/11330>.